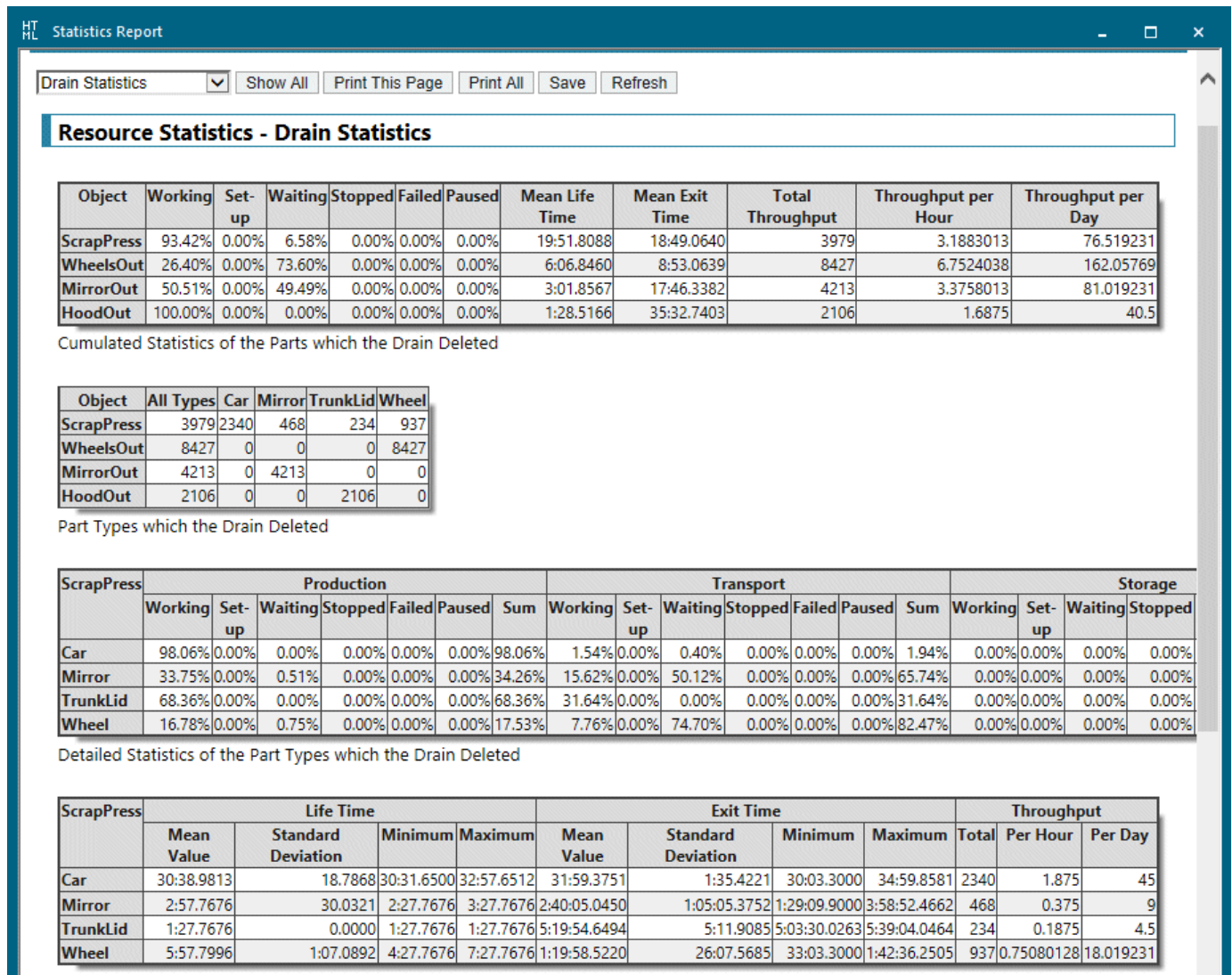




You can extend this model at will, for example to remove more parts, such as the engine, doors, bumpers, fenders, etc. You can also add *Workers*, who carry parts from station to station and who test and sell parts as we did in the scrap yard model in the **examples collection**.

[Back to Remove Parts with the Dismantle Station](#)

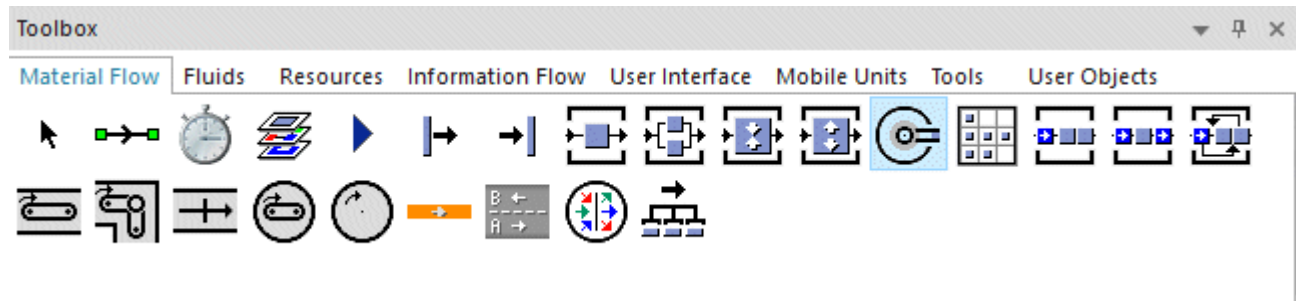
## Picking and Placing Parts with the PickAndPlace Robot

### Pick and Place Parts with the PickAndPlace Robot

The *PickAndPlace robot* picks a part up at one station, rotates to another station and places it there. It can pick up and deliver one or more parts.

To deliver several parts, enter the **Capacity**.

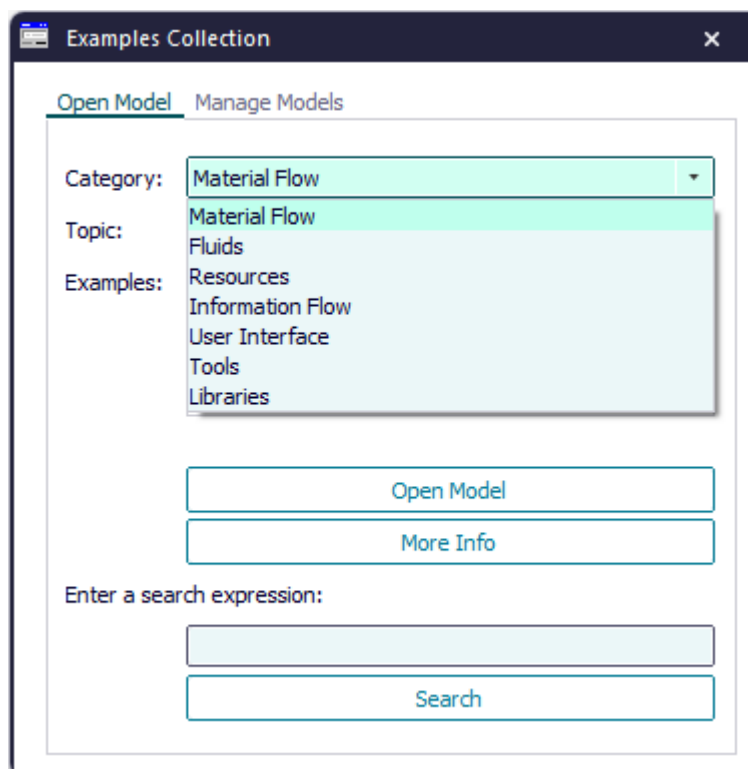
You can insert the **PickAndPlace Robot**  into your simulation model from the folder **MaterialFlow** in the *Class Library* or from the toolbar **Material Flow** in the *Toolbox*.



In our examples we will demonstrate how to:

- **Pick Up Parts and Place Them with the Robot**
- **Pick Up Several Parts and Place Them with the Robot**
- **Place Parts with a Target Control**
- **Load Parts with an Exit Control**
- **Load and Unload Parts at the Sensor of a Conveyor**
- **Set the Conveying Direction of the Parts**
- **Unload Stacked Parts**
- **Animate the Robot Arm**

Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.



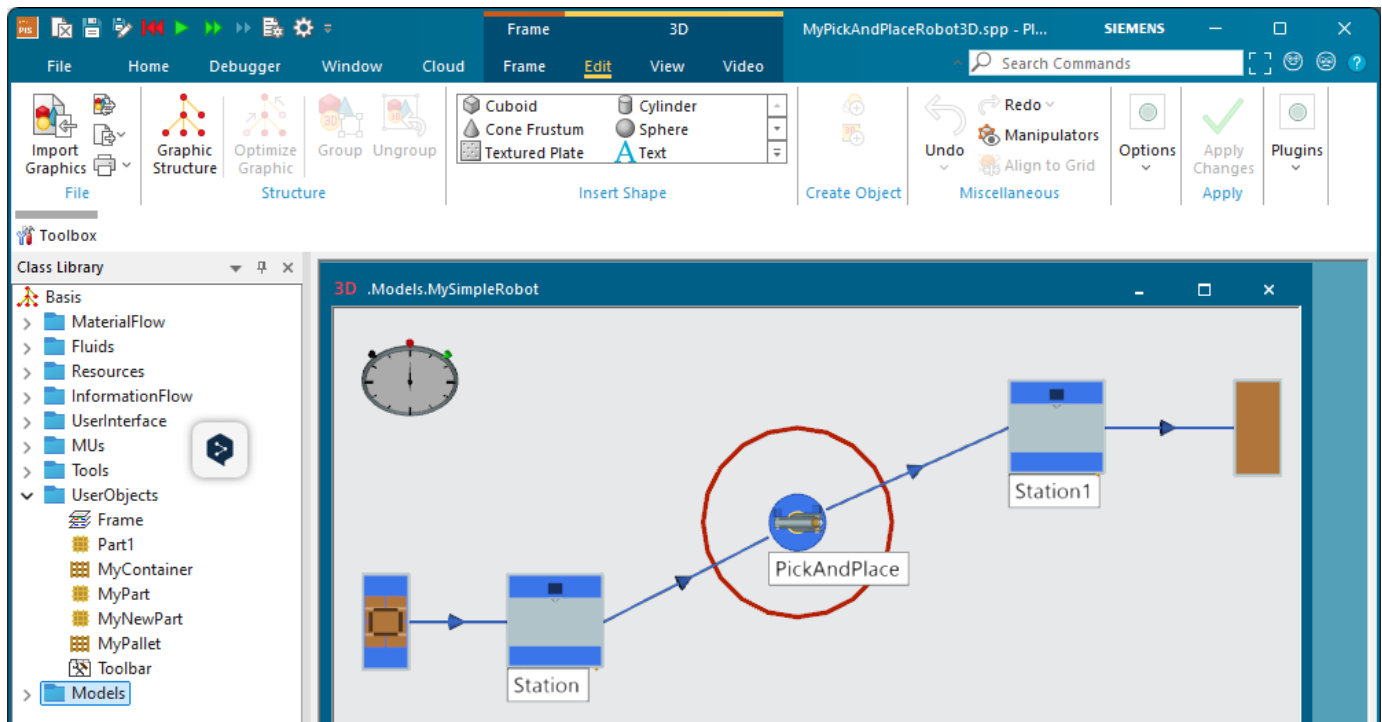
Back to [Modeling the Material Flow](#)

## Pick Up Parts and Place Them with the Robot

In this sample model we demonstrate how you can use the *PickAndPlace robot* with its default settings, i.e., without changing anything.

The *Source* produces parts of type *MyPart* and moves it on to the processing station *Station*, where the *PickAndPlace robot* picks it up. It rotates toward the target station, *Station1* in our case, and deposits the part there. After processing the part, the target station moves it on to the *Drain*.

The finished simulation model looks like this:

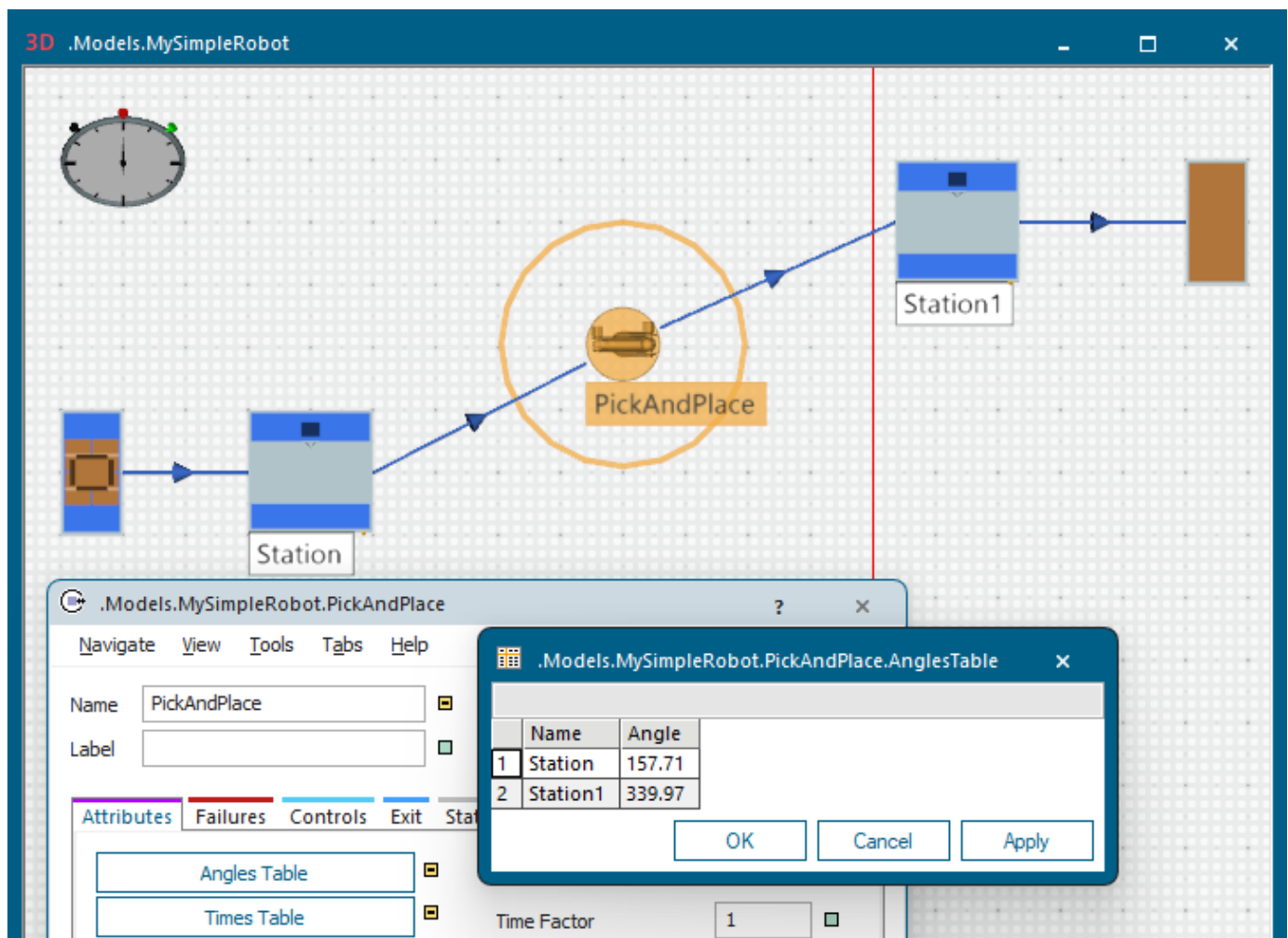


To create the simulation model, insert the objects so that they match the screenshot above. You can use the objects as you inserted them, i.e., without changing any of their settings.

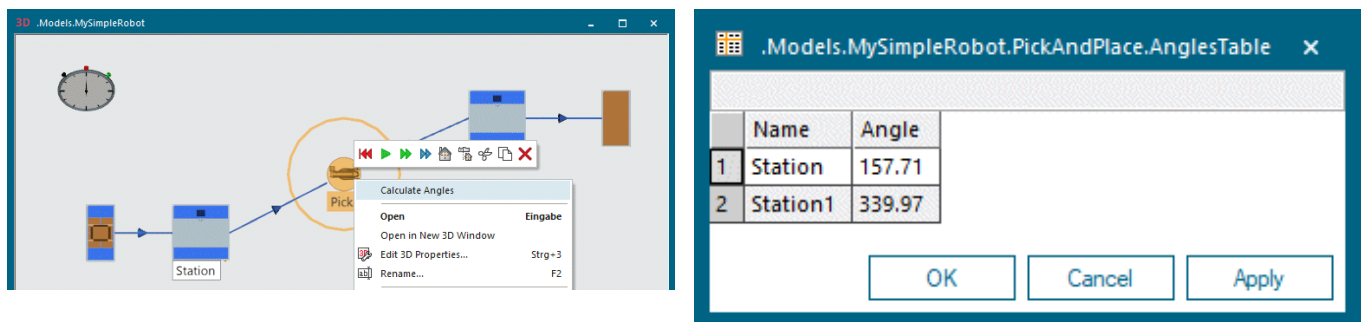
Start the simulation and watch how the *PickAndPlace* robot picks the part up at the station named *Station*, rotates to the target station named *Station1*, and deposits the part there.

You can also move the position of the target station at will and make the *PickAndPlace* robot recompute the angle at which it connects to the station named *Station1*.

With the stations at their original position, the **Angles Table** looks like this:



Then, move the target station *Station1*, right-click the *PickAndPlace* robot and select **Calculate Angles**. Check the new target angle.



Back to **Pick and Place Parts with the PickAndPlace Robot**

## Pick Up Several Parts and Place Them with the Robot

### Pick Up Several Parts and Place Them with the Robot

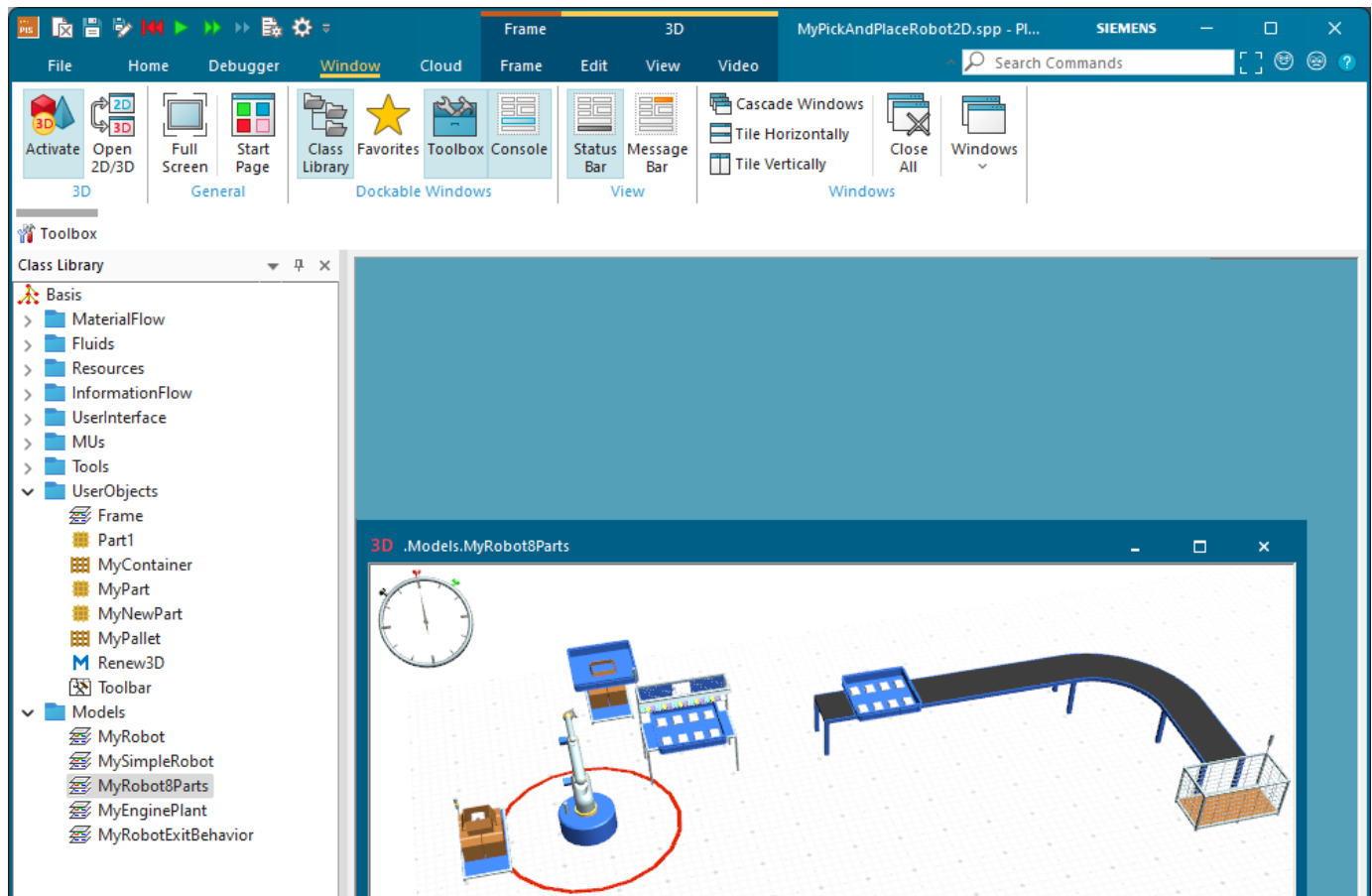
In our sample model the *Sources* create the parts. The object *SourceContainers* moves the containers directly on to the *AssemblyStation*. The object *SourceParts* moves the parts to the *PickAndPlace* robot, which picks them up, rotates, and delivers them to the *AssemblyStation*.

After the *AssemblyStation* has loaded the parts onto the container, it moves the loaded container onto the *Conveyor*, which delivers it to the *Drain*.

In our examples we will:

- [Configure Sources, Containers, and Parts to be Produced](#)
- [Configure the Robot to Pick Up Several Parts](#)
- [Configure the AssemblyStation \[robot\]](#)

The finished simulation model looks like this:

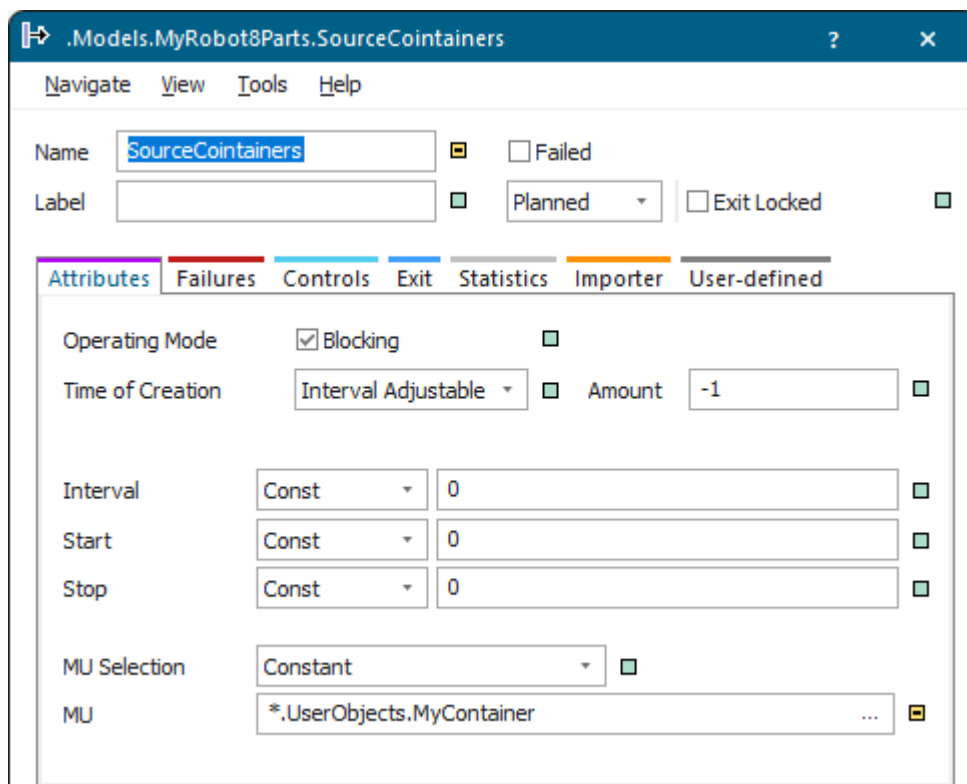


Back to [Pick and Place Parts with the PickAndPlace Robot](#)

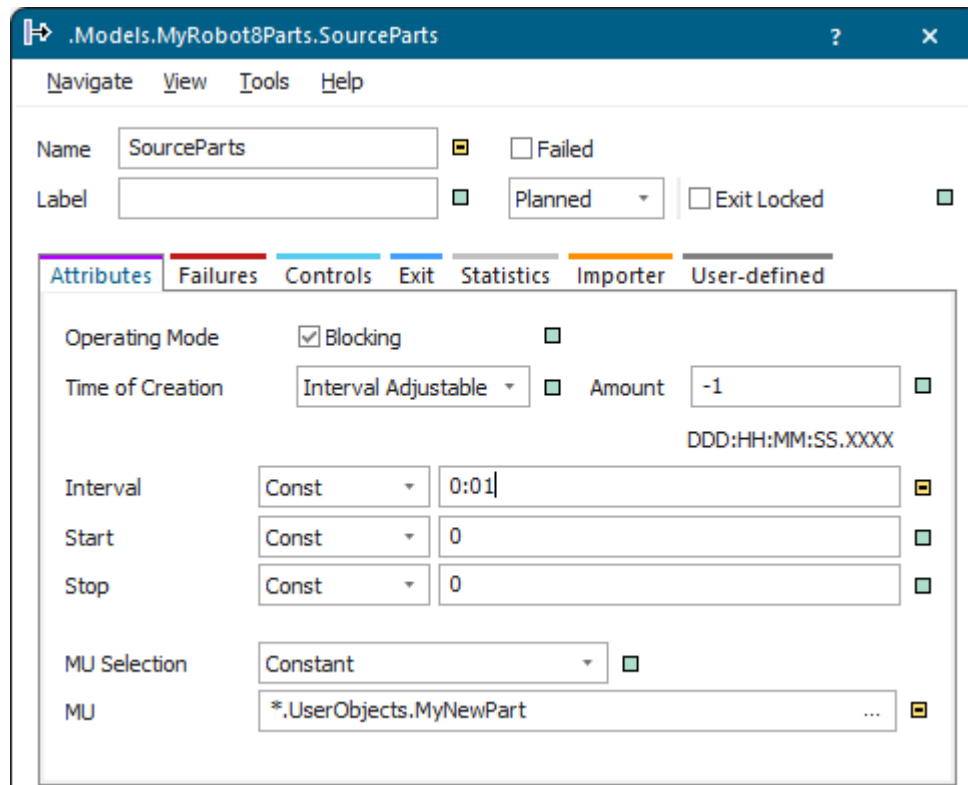
## Configure Sources, Containers, and Parts to be Produced

Insert and configure the *Sources* which produce the parts:

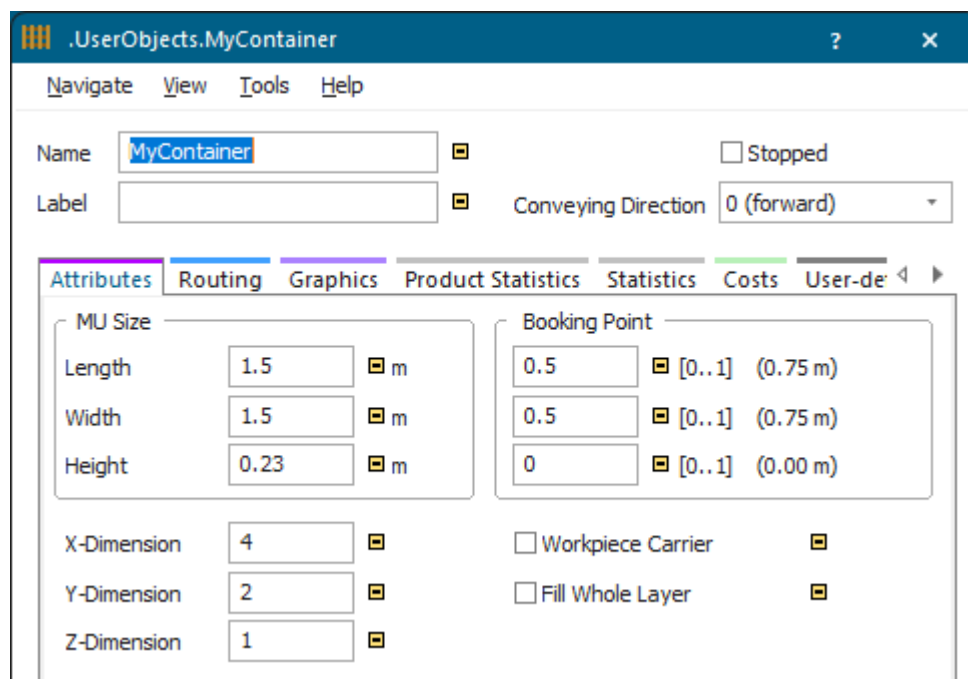
- Select the type of part which the object *SourceContainers* is to produce. We selected *MyContainer* in our example. We did not change any other settings on this or the other tabs.



- Select the type of part which the object *SourceParts* is to produce. We selected *MyNewPart* in our example. We also want this *Source* to produce a part every second, so we entered 0 : 01 as the **Interval**.



- Configure the *Container*, which the *SourceContainers* is going to produce. We entered the settings shown in the screenshot below. We want our parts to be placed in two rows of four each, so we entered 4 as the **x-dimension**, 2 as the **y-dimension**, and 1 as the **z-dimension**.



- Configure the *part* which the *SourceParts* is going to produce.

## Back to [Pick Up Several Parts and Place Them with the Robot](#)

### Configure the Robot to Pick Up Several Parts

Insert and configure the *PickAndPlace* robot. You can use it with its default settings except for entering its **Capacity**.

The screenshot shows the configuration window for the *PickAndPlace* robot. The 'Attributes' tab is selected, displaying the following settings:

- Name:
- Label:
- ☐ Failed
- ☐ Entrance Locked
- ☐ Exit Locked
- ☐ Planned
- Time Factor:
- Default Angle:
- Blocking Angle:
- Capacity:
- MU Conveying Direction:
- Loading Time:
- Unloading Time:

- Enter the number of parts which the *PickAndPlace* robot is to move at the same time into the text box **Capacity**. We want it pick up eight parts, so we entered 8.
- Connect the *PickAndPlace* robot with the *SourceParts* and with the *AssemblyStation*. *Plant Simulation* then computes and enters the respective angles and times into the **Angles Table** and into the **Times Table**.

|   | Name        | Angle  |
|---|-------------|--------|
| 1 | SourceParts | 180.00 |
| 2 | Assembly    | 303.69 |

| Full\Empty    | Default Angle | SourceParts | Assembly |
|---------------|---------------|-------------|----------|
| Default Angle | 0.0000        | 2.0000      | 0.6257   |
| SourceParts   | 0.0000        | 0.0000      | 1.3743   |
| Assembly      | 0.0000        | 1.3743      | 0.0000   |

- As we want to see how the *PickAndPlace* robot picks up eight parts, we type in 2 each into the text boxes **X-Dimension** and **Z-Dimension**.

**.UserObjects.MyContainer** ? x

Navigate View Tools Help

Name  ☐ Stopped

Label  ☐ Conveying Direction 0 (forward) ▾

Attributes Routing Product Statistics Statistics Costs User-defined

MU Size

|        |                                     |
|--------|-------------------------------------|
| Length | <input type="text" value="0.85"/> m |
| Width  | <input type="text" value="0.55"/> m |
| Height | <input type="text" value="0.23"/> m |

Booking Point

|                                                  |
|--------------------------------------------------|
| <input type="text" value="0.5"/> [0..1] (0.42 m) |
| <input type="text" value="0.5"/> [0..1] (0.27 m) |
| <input type="text" value="0"/> [0..1] (0.00 m)   |

X-Dimension  ☐

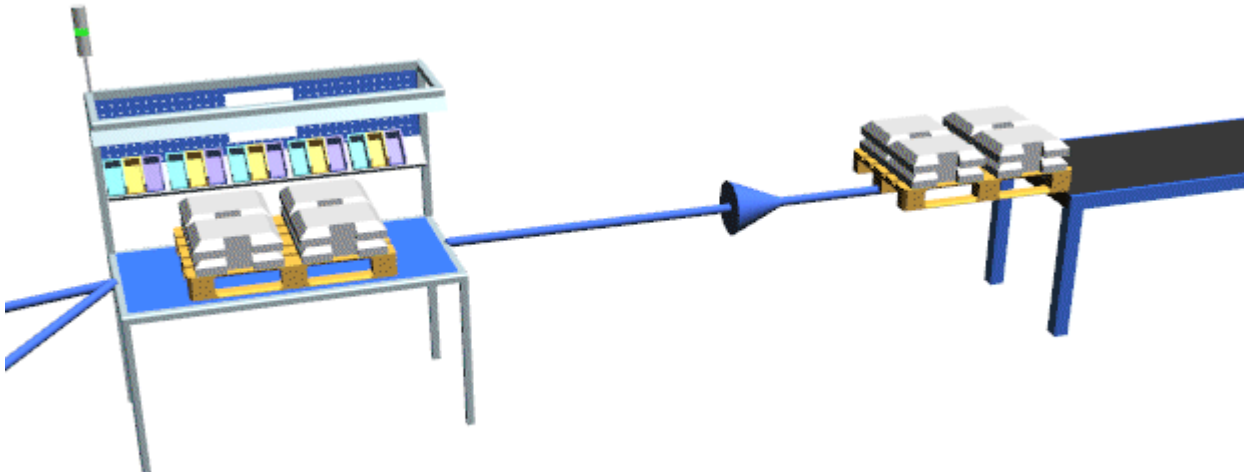
Y-Dimension  ☐

Z-Dimension  ☐

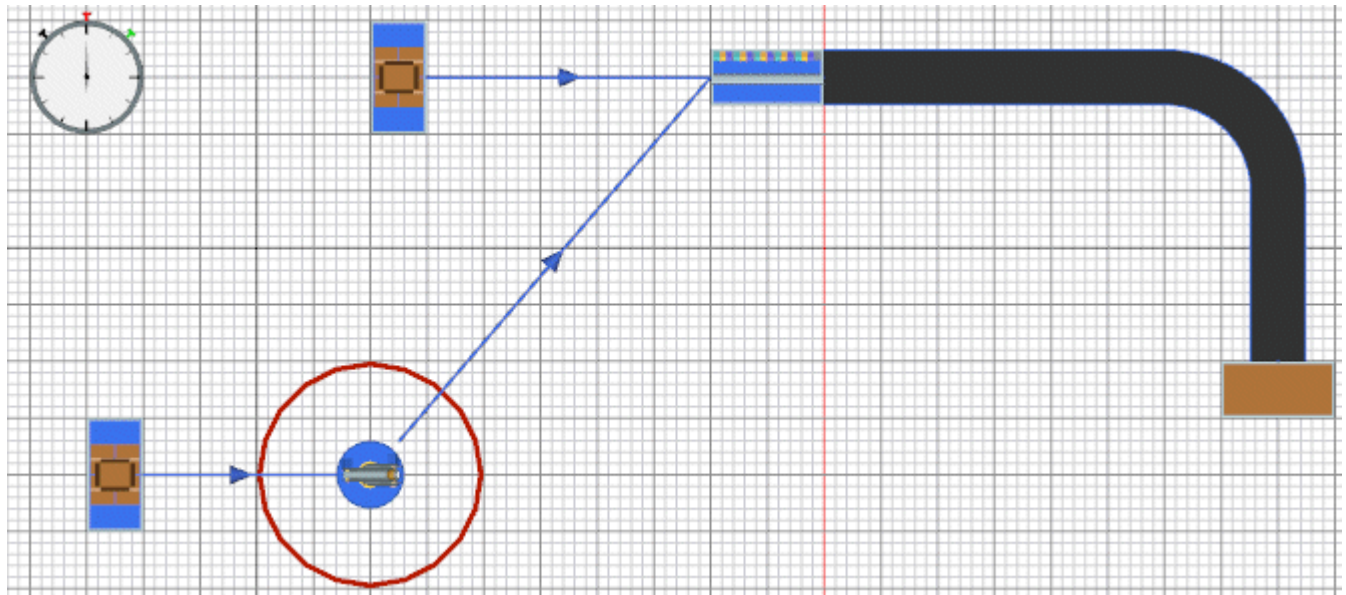
☐ Workpiece Carrier ☐

☐ Fill Whole Layer ☐

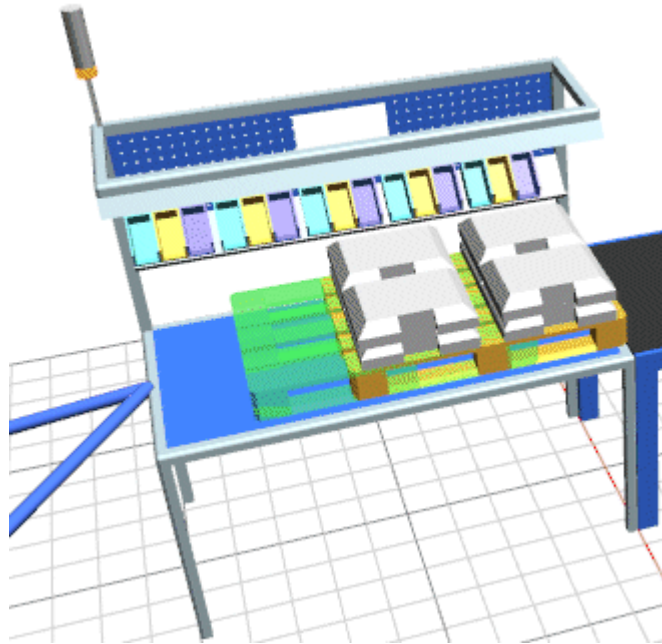
- Now *Plant Simulation* places two parts each along the width, depth, and height of the pallet, meaning that it also stacks parts in the height.



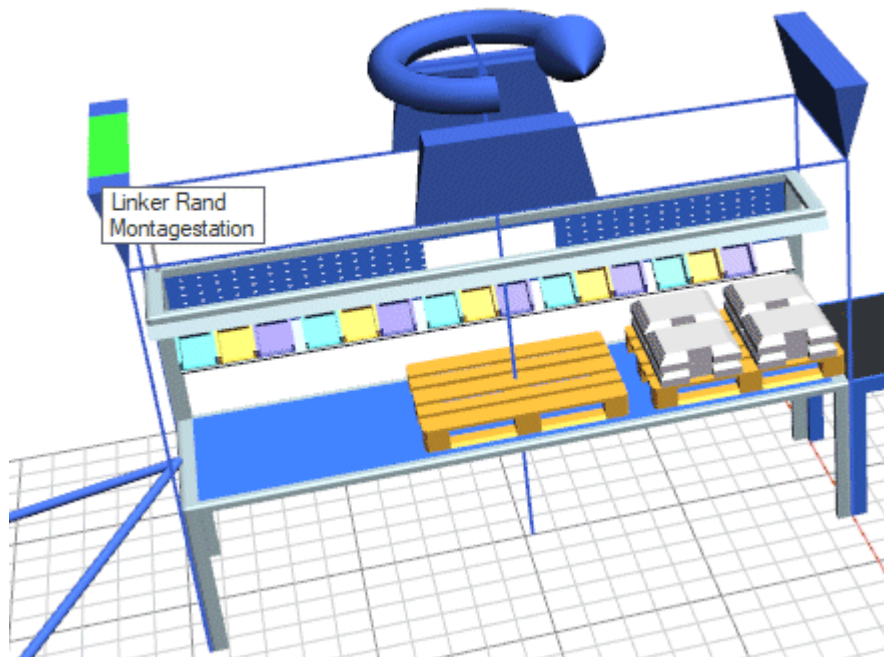
- Next, we move the *Conveyor* to the left and the *Drain* up so that these objects directly join each other.



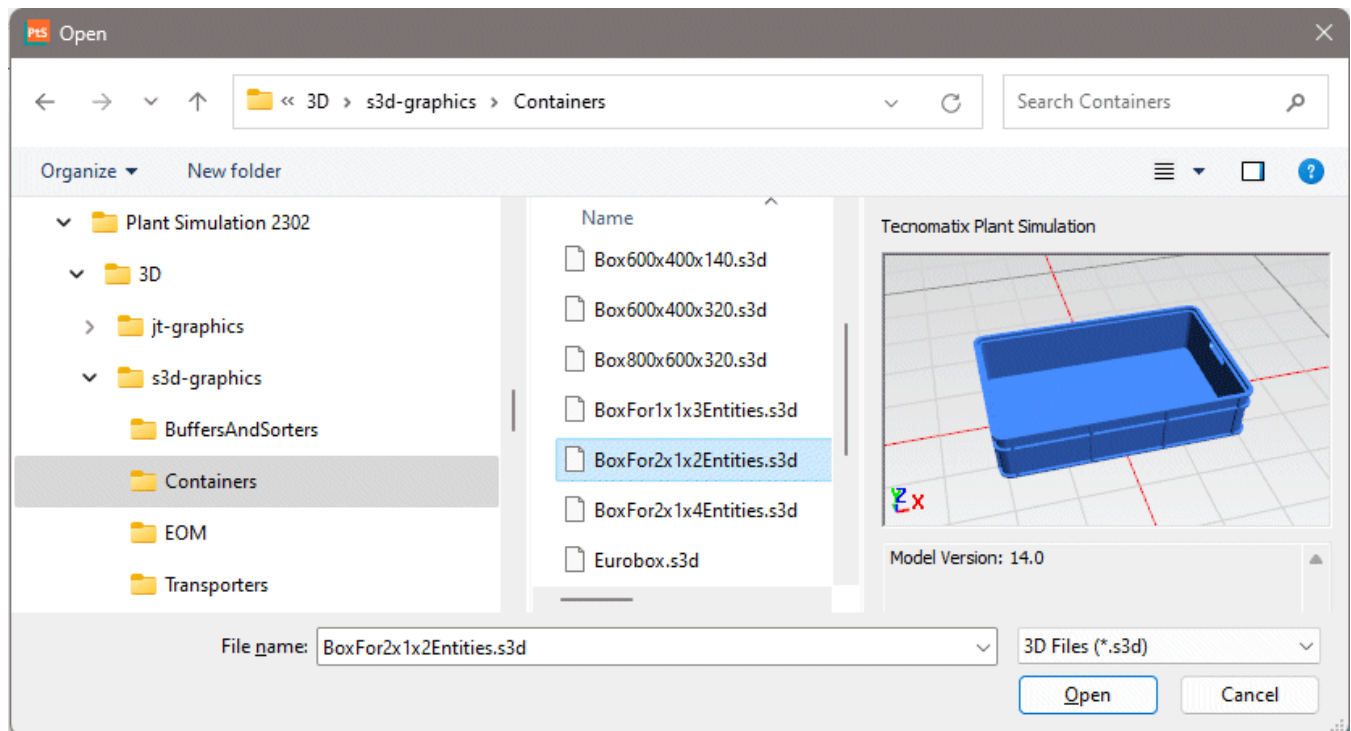
- When we run the simulation, we see that an empty *pallet*, which moves to the *AssemblyStation*, overlaps the loaded *pallet* that is still located there.



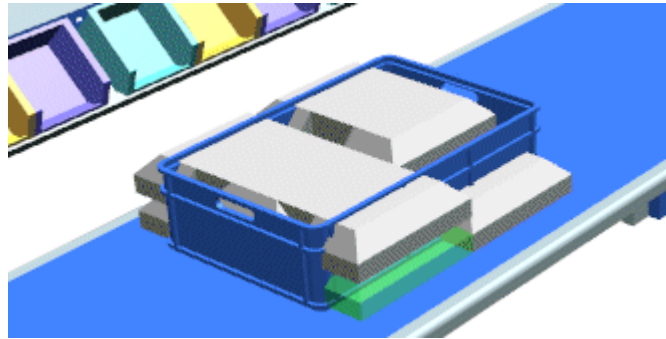
To fix this problem, click the *AssemblyStation* and press the **M** key, **Show Manipulators**. Click the left border of the *AssemblyStation* and press the left arrow key until the *pallets* do not overlap any more. Instead you can also set the settings for the scale on the tab **Transformation** of the *AssemblyStation*.



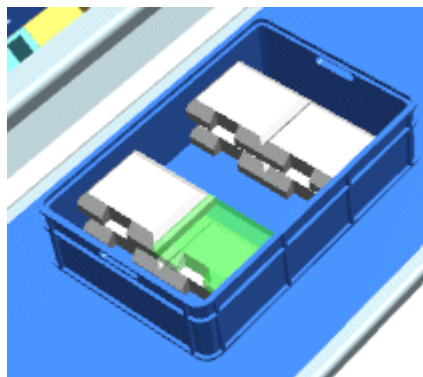
- Click our *Container*, named *MyContainer*, with the right mouse button in the *Class Library* and select **Open in 3D**.
- Click in the background of the window and select **Exchange Graphics**.
- We selected the transport box **BoxFor2x1x2Entities.s3d** and clicked **Yes** in the dialog that was opened.



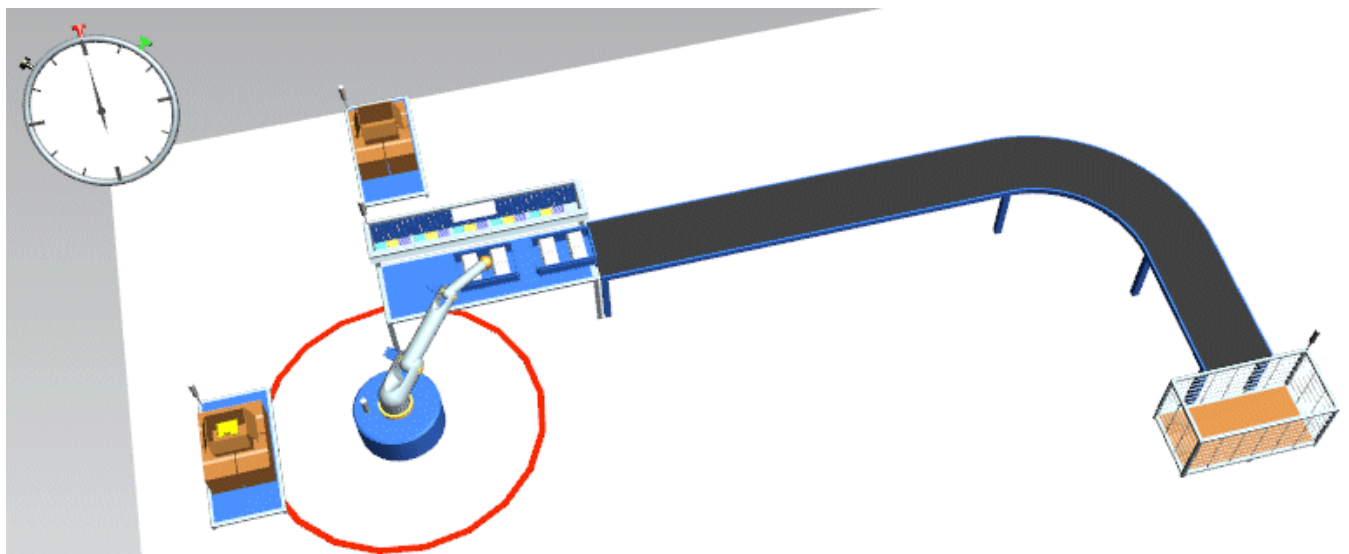
- When we run the simulation, we see that the displayed parts protrude the border of the transport box.



To fix this, we scale the part *MyNewPart* on the tab **Transformation** in the dialog **Edit 3D Properties**. Click the **down arrow** below **Scale** and then click **Apply** to show the changed value in the model. In our case the value 0.55 is the correct scale.



As our *AssemblyStation* now is a little bit too long, we can adjust it with again with the **manipulator**. We finally move the remaining objects so that they directly join the neighboring objects and hide the **connections**.

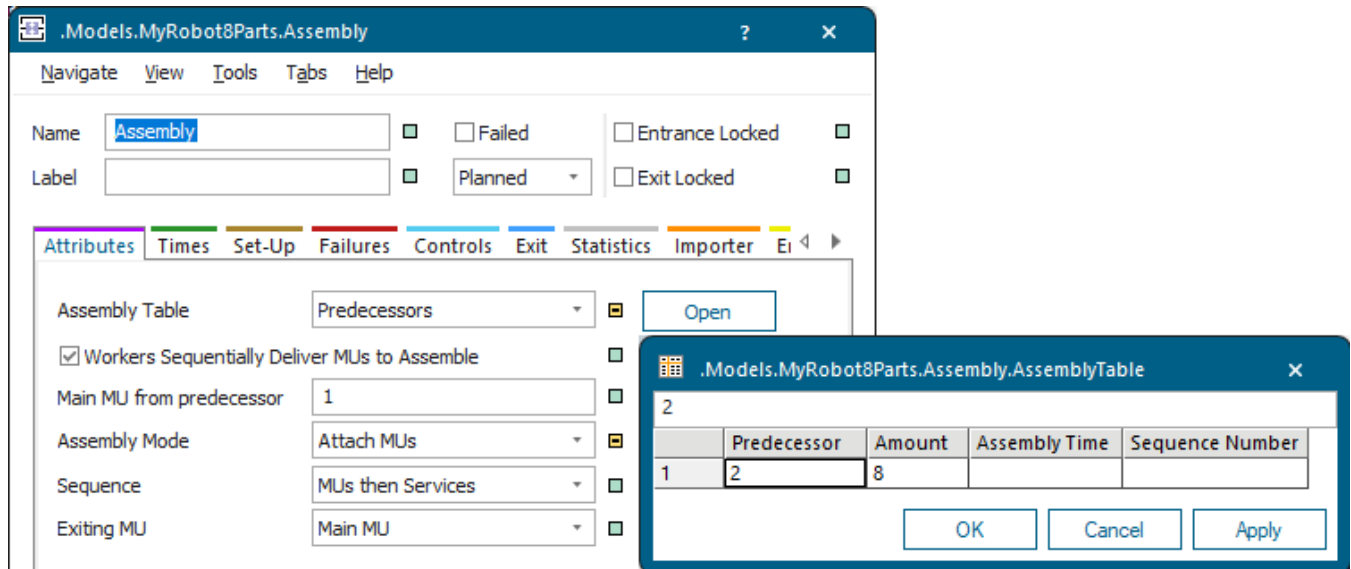


Back to **Pick Up Several Parts and Place Them with the Robot**

## Configure the AssemblyStation [robot]

Insert and configure the *AssemblyStation* which loads the parts onto the containers.

- As we want to load eight parts arriving from the *PickAndPlace robot*, we select **Assembly Table > Predecessors** and click **Open**. Enter the number of the **Predecessor**, namely 2, and the **Number** of parts we want to load onto the containers, namely 8.



- Start the simulation and watch how the *SourceParts* produces parts and how it moves them to the *PickAndPlace robot*. It then picks up eight parts one after the other, rotates, and loads them one by one onto the container on the *AssemblyStation* in the same order as it picked them up (FIFO). To better see how this is done, you can reduce the simulation speed.

Back to [Pick Up Several Parts and Place Them with the Robot](#)

## Place Parts with a Target Control

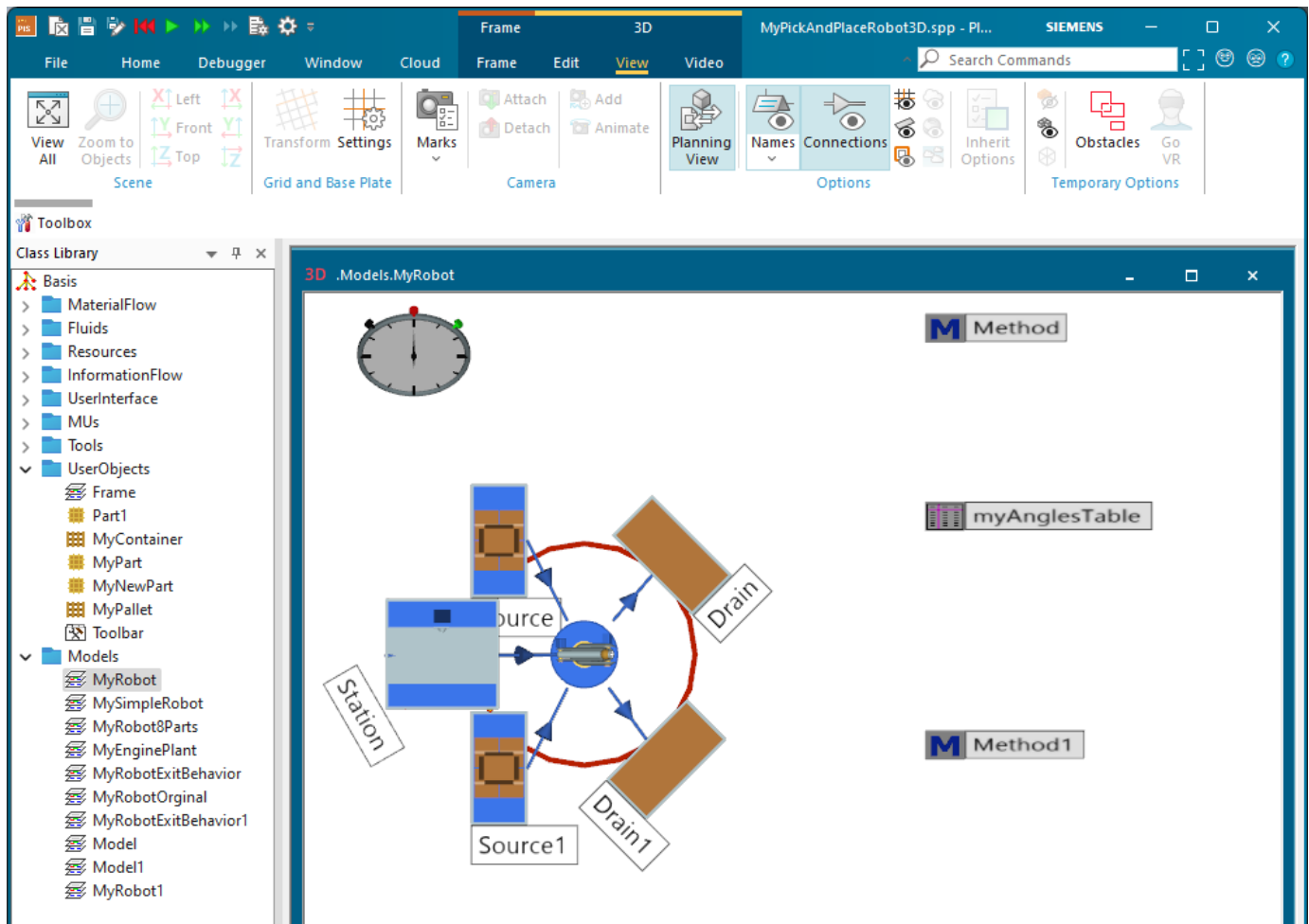
### Place Parts with a Target Control

In our sample model the *Sources* create the parts. The object *Source* moves its part directly on to the *PickAndPlace robot*, which picks it up, rotates, and delivers it to the *Drain*. The object *Source1* moves its part to the *PickAndPlace robot*, which picks it up, rotates, and delivers it to the processing station named *Station*. After the processing station has processed the part, it feeds it back to the *PickAndPlace robot*, which picks it up and delivers it to *Drain1*.

In our example we will demonstrate how to:

- [Configure the Sources and the Parts to be Produced](#)
- [Configure the Processing Station](#)
- [Configure the Robot](#)

The finished simulation model looks like this:

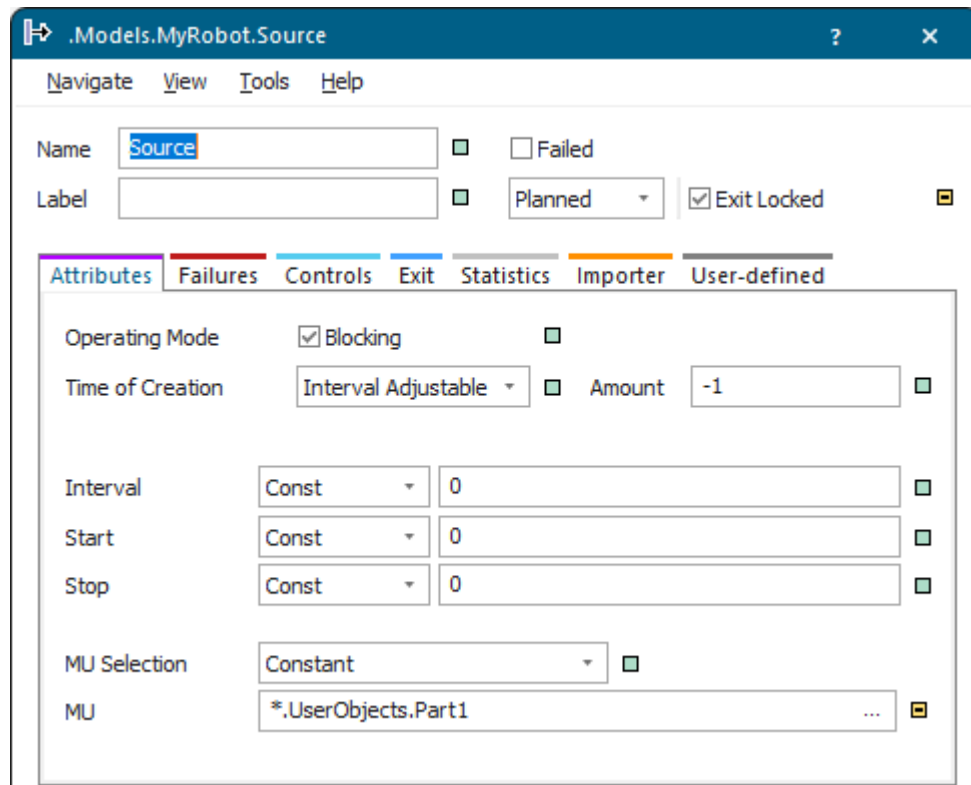


### Back to Pick and Place Parts with the PickAndPlace Robot

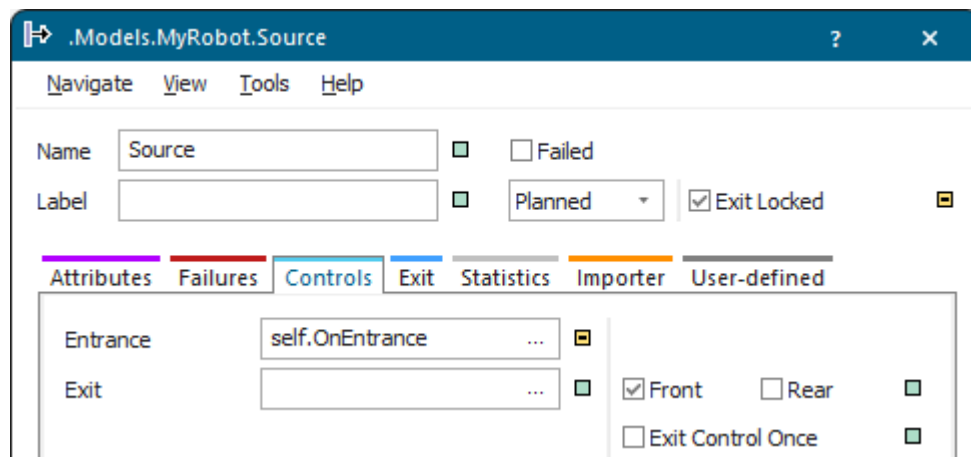
## Configure the Sources and the Parts to be Produced

Insert and configure the *Sources* which produce the parts:

- Select the type of part which the object *Source* is to produce. We selected *Part1* in our example.



- Define the **Entrance Control**, which sets the last station on which the part is located. To do so, right-click into the text box and select **Create Control**.



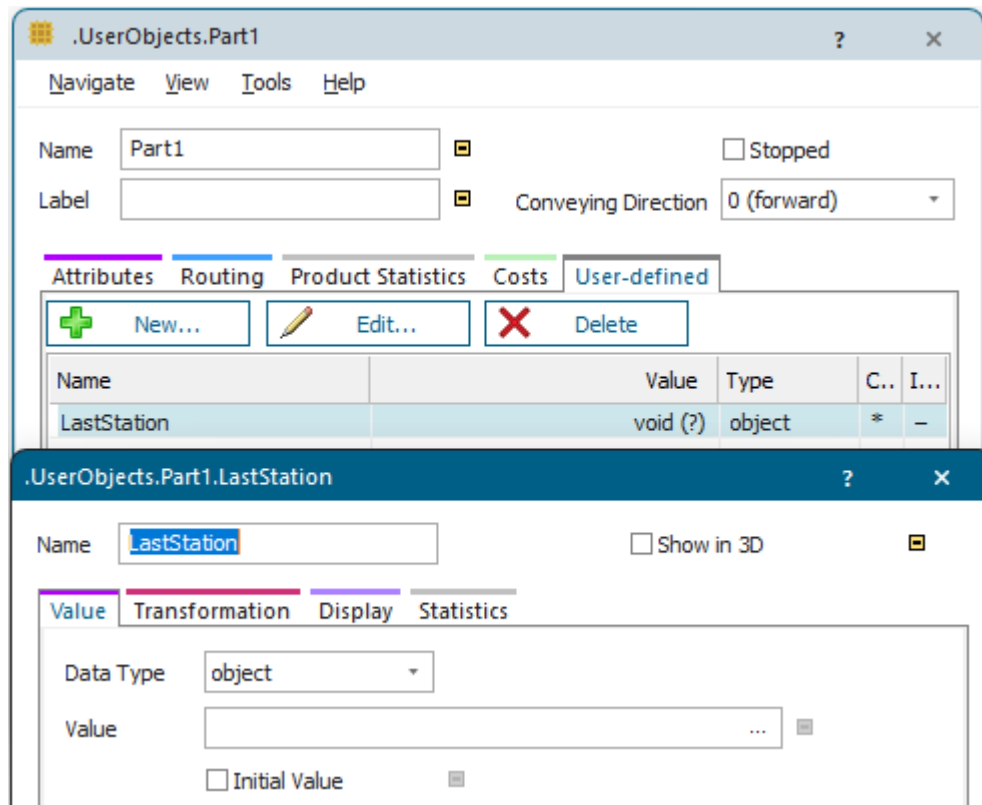
- Copy the following source code and paste it. The anonymous identifier @ designates the part, while the anonymous identifier ? designates the station, i.e., the material flow object, on which the part is located.

```
@.LastStation := ?
```

- Select the type of part which the object *Source1* is to produce. We selected *Part1* as well.
- Create an **Entrance Control** for the *Source1* and enter the same source code as above.

Create the user-defined attribute `LastStation` for the part named *Part1*. This attribute is evaluated in the **Target Control** of the *PickAndPlace* robot to select the new target station.

- Double-click the object *Part1* and click the tab **User-defined**.
- Click **New**, enter the **Name** of the user-defined attribute and select its **Data Type**. We entered `LastStation` and selected the **Data Type > object**.



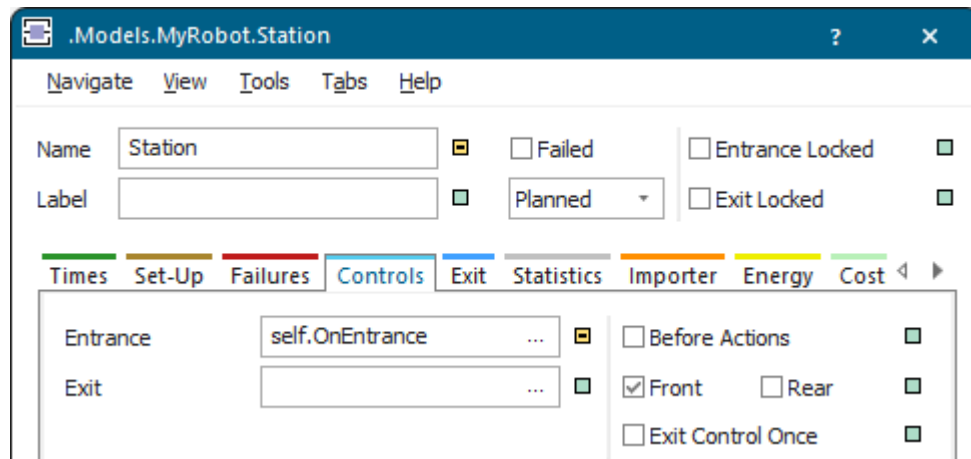
### Back to Place Parts with a Target Control

## Configure the Processing Station

Insert and configure the station which processes the part named *Part1*, which the *Source1* produces.

- Enter a **Processing Time** of your choice.
- Define the **Entrance Control**, which sets the last station on which the part is located. To do so, right-click into the text box and select **Create Control**.
- Copy the following source code and paste it. The anonymous identifier `@` designates the part, while the anonymous identifier `?` designates the station, i.e., the material flow object, on which the part is located.

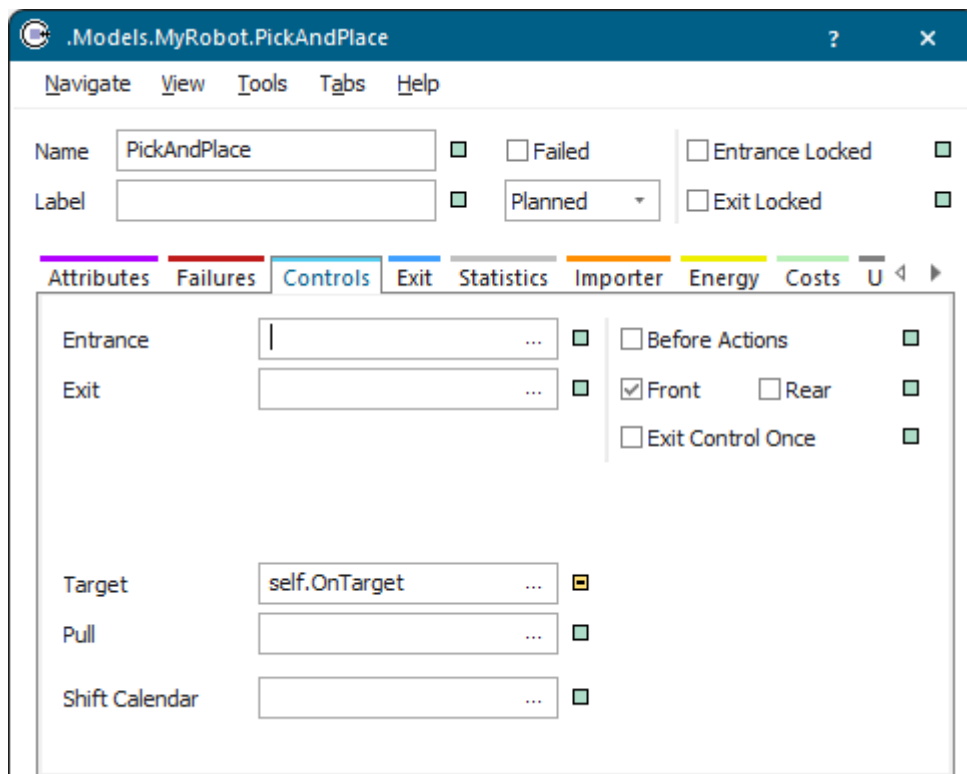
```
@.LastStation := ?
```



Back to [Pick Up Several Parts and Place Them with the Robot](#)

## Configure the Robot

Insert and configure the *PickAndPlace* robot. You can use it with its default settings except for entering a **Target Control**.



- Define the **Target Control**, which determines to which station or *Drain* the *PickAndPlace* robot delivers the two parts. To do so, right-click into the text box and select **Create Control**.

- Copy the following source code and paste it. The anonymous identifier @ designates the part, while the anonymous identifier ? designates the station, i.e., the material flow object, on which the part is located.

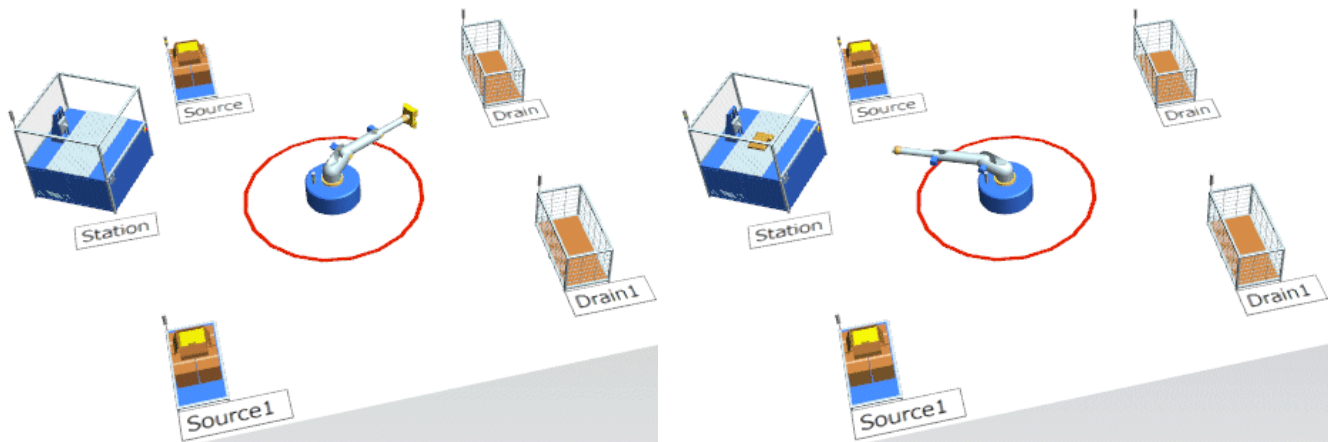
```

if ?.empty
  return
end

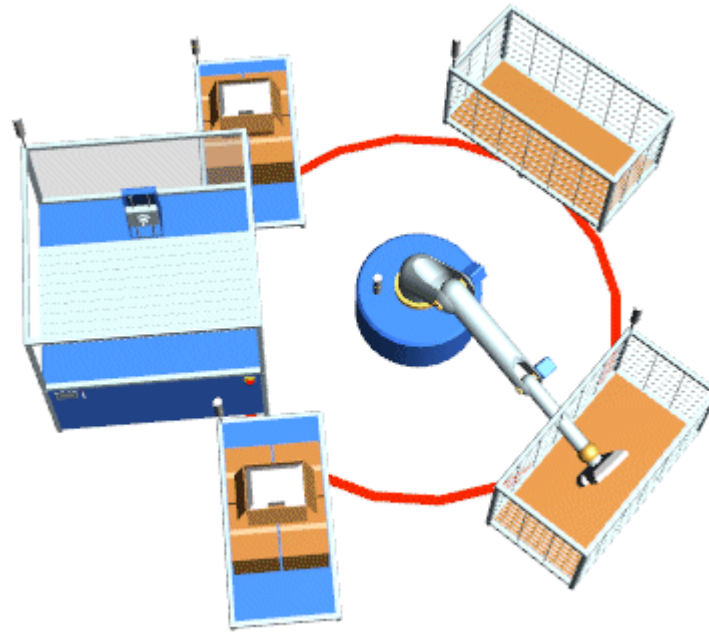
if @.LastStation = Source
  ?.setDestination(Drain)
elseif @.LastStation = Source1
  ?.setDestination(Station, true)
else
  ?.setDestination(Drain1)
end

```

- Start the simulation and watch how the *Source* moves *Part1* directly on to the *PickAndPlace* robot, which picks it up, rotates, and delivers it to the *Drain*. The *Source1* moves the part to the *PickAndPlace* robot, which picks it up, rotates, and delivers it to the processing station *Station*. After the processing station has processed the part, it feeds it back to the *PickAndPlace* robot, which picks it up and delivers it to *Drain1*. To better see how this is done, you can reduce the simulation speed.



- Finally, we can move the stations closer together so that the robot picks the parts up at the stations and deposits them there.



Compare [Animate the Robot Arm](#)

Back to [Pick and Place Parts with the PickAndPlace Robot](#)

## Load Parts with an Exit Control

### Load Parts with an Exit Control

In this sample model we will solve this modeling task:

- The *PickAndPlace* robot picks a *Part* up from a *Source* and places it directly into a *Basket*.
- *Conveyor1* transports the *Basket* to the *Station* which processes the *Part*.
- *Conveyor2* transports the *Basket* containing the *Part* in the opposite direction.
- Once the *Basket* stopped, the *PickAndPlace* robot picks up the *Part* and places it in the *Drain*.

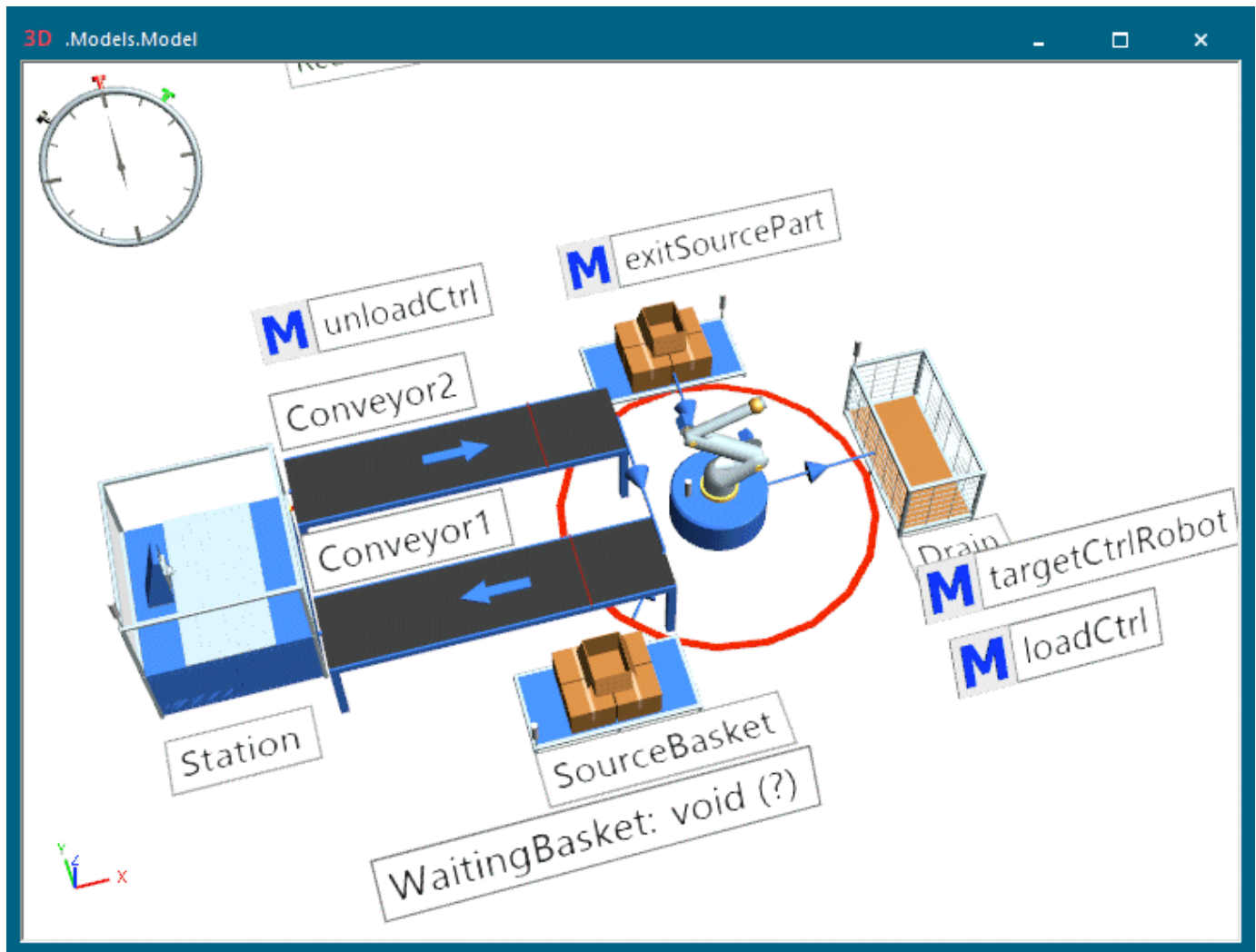
The *Variable* named *WaitingBasket* ensures that the part will always be put into the correct basket and that the *PickAndPlace* robot only picks up a part when a basket is available to place the part.

- We accomplish this with the **Exit Control** and the **Target Control** of the *PickAndPlace* robot.

In our example we will:

- [Model the Material Flow and Program the Controls](#)
- [Configure the Robot and Program Exit and Target Control](#)

The finished simulation model looks like this:



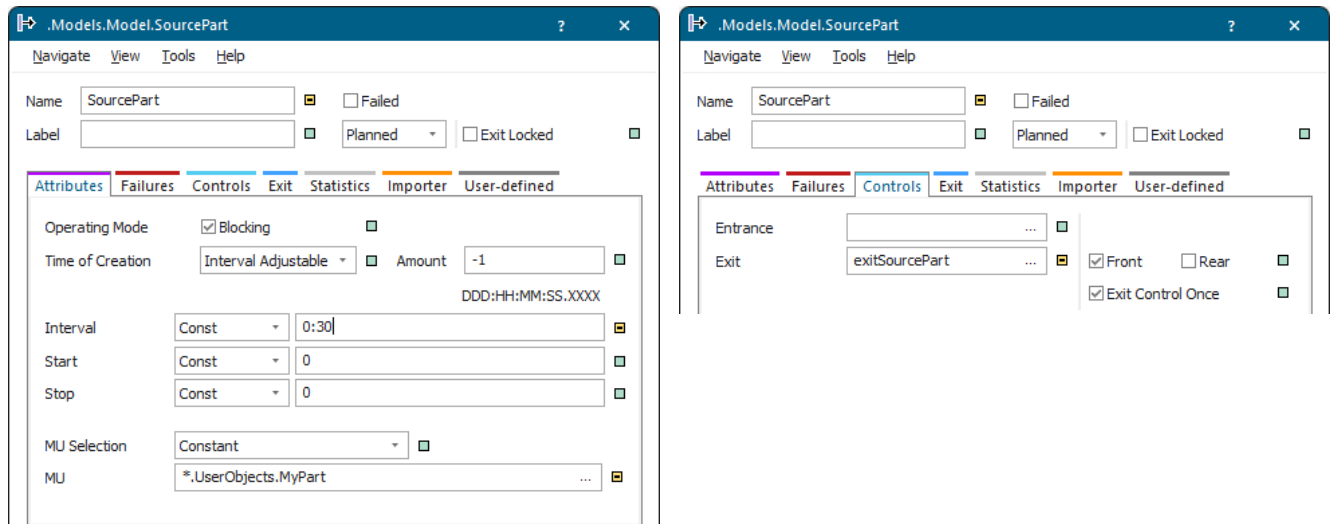
### Back to **Pick and Place Parts with the PickAndPlace Robot**

#### Model the Material Flow and Program the Controls

As we do not want to modify the built-in MU classes, we duplicate the *Part* and the *Container* in the folder *MUs*. We renamed the duplicated *Part* to *MyPart* and the *Container* to *Basket*.

Then we insert the material flow objects.

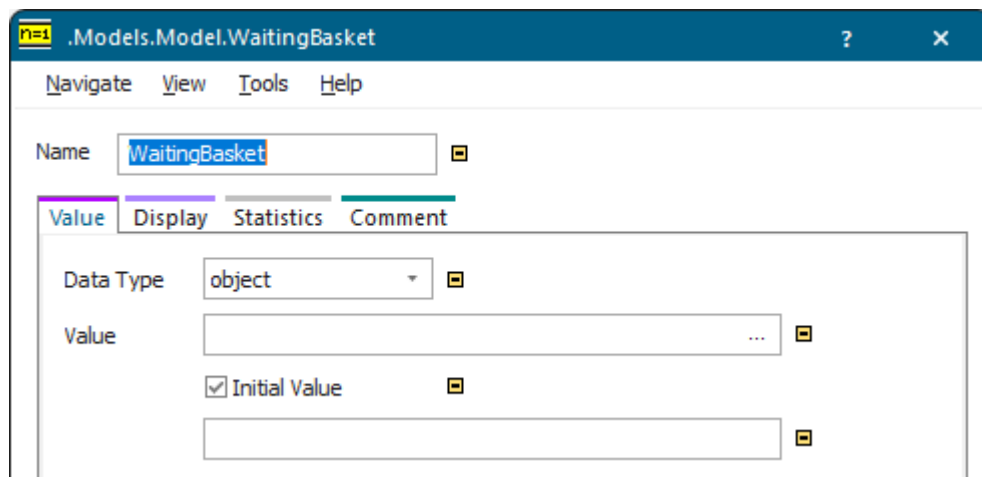
- The *SourceParts* produces a part of type *MyPart* every 30 seconds.



In the **Exit Control** of the *SourcePart* we set the destination of *MyPart*. We named our control *exitSourcePart* and typed in this source code:

```
// exit control of the SourceParts
var Destination: object := Robot.getDestination
stopuntil WaitingBasket /= void and Destination /= WaitingBasket
@.Destination := WaitingBasket
@.move
```

- We selected these settings in the *Variable* named *WaitingBasket*.



The *Variable* named *WaitingBasket* ensures that the part will always be put into the correct basket and that the *PickAndPlace* robot only picks up a part when a basket is available to place the part.

- The *SourceBasket* produces 4 baskets in an interval of 10 seconds.

**.Models.Model.SourceBasket**

Navigate View Tools Help

Name: SourceBasket ☐ Failed

Label:  ☐ Planned ☐ Exit Locked

**Attributes** Failures Controls Exit Statistics Importer User-defined

Operating Mode: ☒ Blocking ☐

Time of Creation: Interval Adjustable ☐ Amount: 4 ☐  
DDD:HH:MM:SS.XXXX

Interval: Const  0:10 ☐

Start: Const  0 ☐

Stop: Const  0 ☐

MU Selection: Constant ☐

MU: \*.UserObjects.Basket ... ☐

- The conveyors transport the *Basket* with the loaded part to type *MyPart* through the plant. We inserted *Conveyor1* from right to left and *Conveyor2* from left to right. In addition we created a **sensor** each with the respective control.

For the **sensor** on *Conveyor1* we selected these settings.

**Sensors.ID1**

Position: Length

1 ☐ [m]

☒ Front ☐

☐ Rear ☐

☐ Booking Point ☐

☐ Light Barrier Mode ☐

Activate: Always ☐

Control: self.OnSensor1 ... ☐

OK Cancel Apply

We typed in this source code into the **Sensor Control**. It sets how *MyPart* is unloaded from the *Basket* and ensures that the *Basket* waits at the sensor until a new part is loaded, i.e. it is not empty any more.

```
param SensorID: integer, Front: boolean, BookPos: boolean
@.Stopped := true
WaitingBasket := @

stopuntil not @.Empty
WaitingBasket := void
@.Stopped := false
```

For the sensor on *Conveyor2* we selected these settings and selected our *unloadCtrl* as the control.

In the sensor control of *Conveyor2* we set how the part will be unloaded from the *Basket*. We typed in this source code:

```
param SensorID: integer, Front: boolean, BookPos: boolean

var MU : object
@.Stopped := true // stop

var Destination:object := Robot

MU := @.Cont
MU.Destination := Drain
if not MU.move(Destination)
    stopuntil MU.Location /= @ and not Destination.IsLoading
end

@.Stopped := false
```

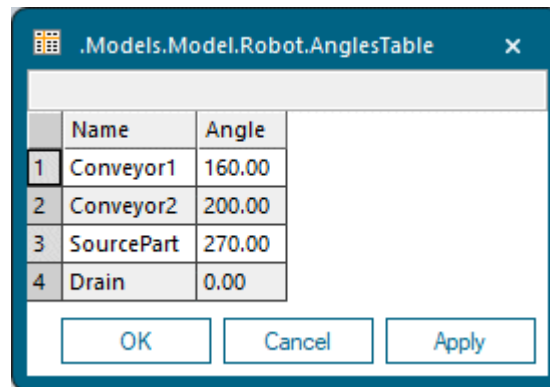
## Back to Load Parts with an Exit Control

### Configure the Robot and Program Exit and Target Control

In this step we configure the *PickAndPlace robot* and program the **Exit Control** and the **Target Control**.

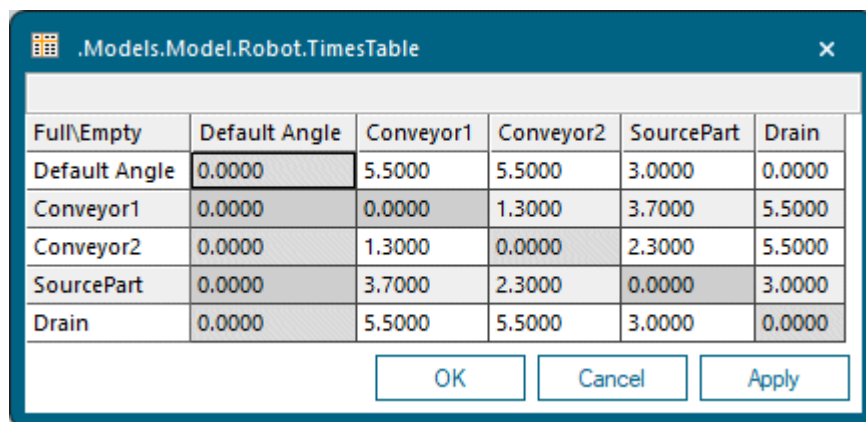
First, we set the angles at which the *PickAndPlace robot* picks the parts up, places them, and deposits them:

- Drag *Conveyor1* first and then *Conveyor2* onto the picture of the *PickAndPlace robot* and drop it there.
- Connect the *SourceParts* with a *Connector* with the *PickAndPlace robot*. Then connect the *PickAndPlace robot* with the *Drain*.
- Double-click the picture of the *PickAndPlace robot* and open the **Angles Table**. You can adjust the angles if they are not precise enough.



|   | Name       | Angle  |
|---|------------|--------|
| 1 | Conveyor1  | 160.00 |
| 2 | Conveyor2  | 200.00 |
| 3 | SourcePart | 270.00 |
| 4 | Drain      | 0.00   |

- Open the **Angles Table**. We use these time. You can adjust them if need be.



| Full\Empty    | Default Angle | Conveyor1 | Conveyor2 | SourcePart | Drain  |
|---------------|---------------|-----------|-----------|------------|--------|
| Default Angle | 0.0000        | 5.5000    | 5.5000    | 3.0000     | 0.0000 |
| Conveyor1     | 0.0000        | 0.0000    | 1.3000    | 3.7000     | 5.5000 |
| Conveyor2     | 0.0000        | 1.3000    | 0.0000    | 2.3000     | 5.5000 |
| SourcePart    | 0.0000        | 3.7000    | 2.3000    | 0.0000     | 3.0000 |
| Drain         | 0.0000        | 5.5000    | 5.5000    | 3.0000     | 0.0000 |

Then, we select the controls which define how the parts are loaded and to which destination they move:

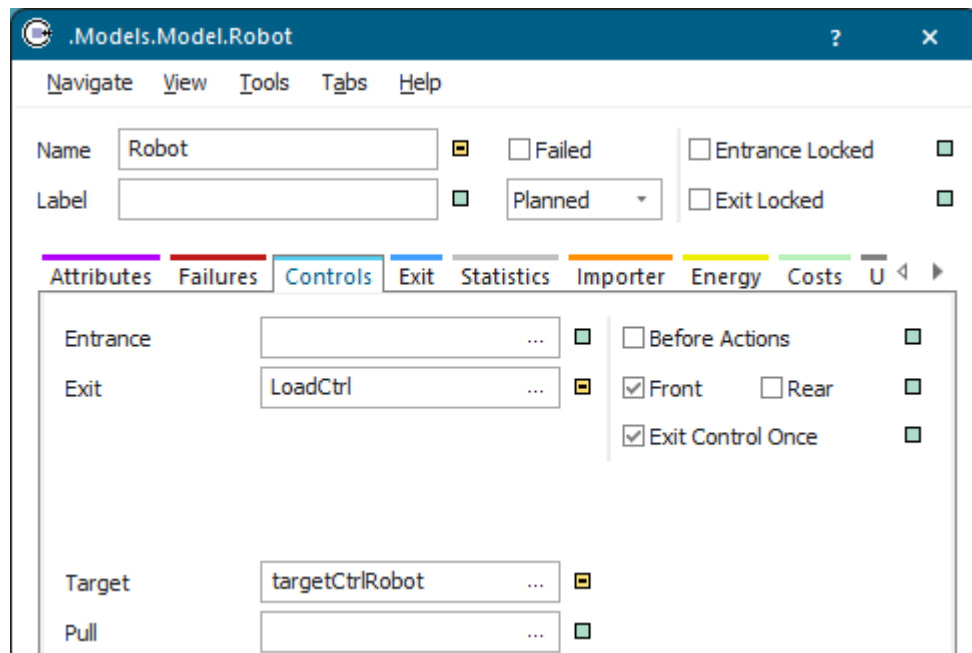
- We selected our **loadCtrl** as the **Exit Control**.

In the **Exit Control** we set how the *PickAndPlace robot* loads the part onto *Conveyor1* if that is its destination.

We typed in this source code:

```
// exit control of the Robot
if @.Destination = Conveyor1
    stopuntil WaitingBasket /= void
    @.move(WaitingBasket)
else
    @.move
end
```

- As **Target Control** we selected the targetCtrlRobot.

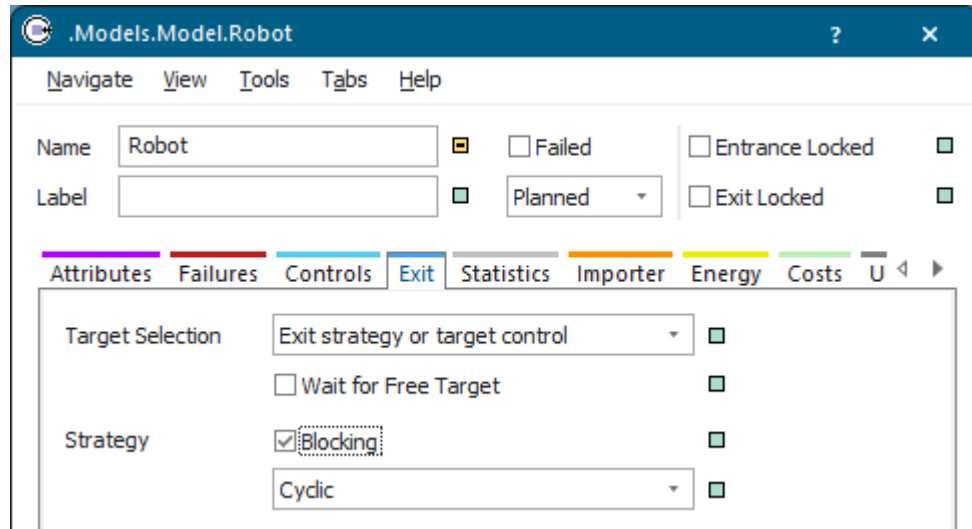


In the **Target Control** we set the target of the part.

We typed in this source code:

```
// target control of the Robot
if @.Destination = Conveyor1 and WaitingBasket /= void
    ?.setDestination(WaitingBasket)
else
    ?.setDestination(@.Destination)
end
```

- We selected **Exit strategy or target control** as the **Target Selection**.

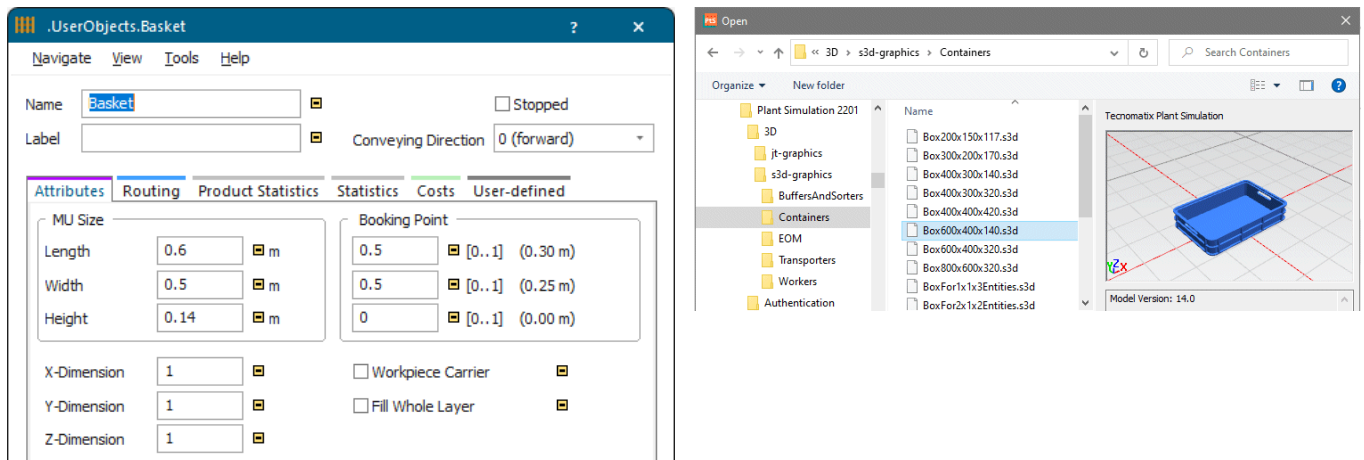


When we run the simulation, we see that the *PickAndPlace* robot picks *MyPart* up at the *SourceParts*, rotates toward *Conveyor1* and places it in the *Basket* at the sensor.

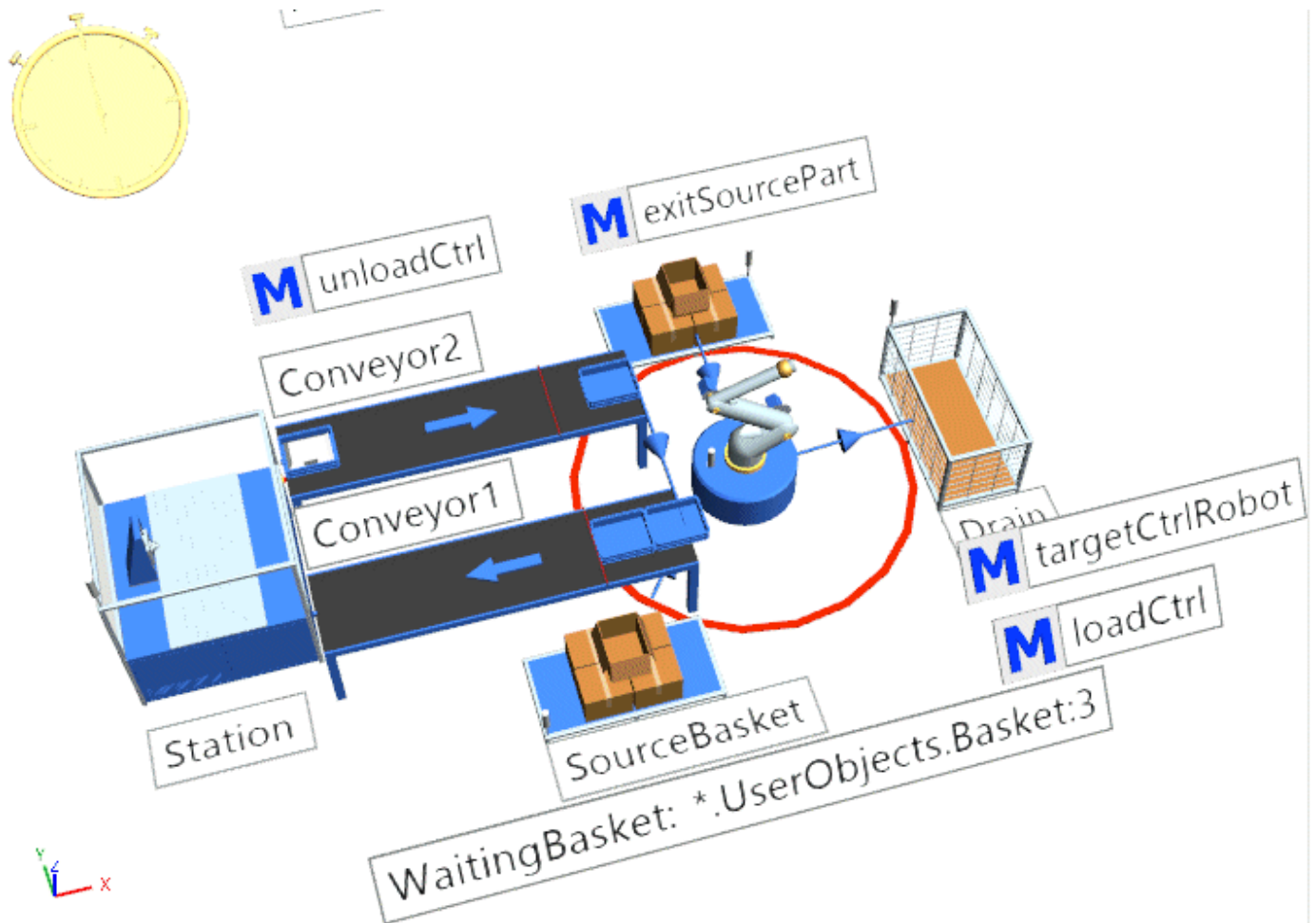
*Conveyor1* transports the *Basket* with the loaded part to the *Station* where it is processed for 10 seconds.

It then moves the *Basket* with the loaded part to *Conveyor2* where the *PickAndPlace* robot unloads it, rotates toward the *Drain*, and deposits it there.

After we ran the simulation for the first time, we changed the settings of the *Basket* and its graphic.



We finally moved the objects closer to each other so that the model looks more realistic.



**Configure the Movement of the Robot Arm** shows how to adjust the movement of the robot arm with an animation path so that it does not collide with the cage of the *Drain*.

Back to **Load Parts with an Exit Control**

## Automatically Load and Unload Parts at a Sensor

### Load and Unload Parts at the Sensor of a Conveyor

In our example we will demonstrate how to place parts from a processing station onto a *Conveyor* with the *PickAndPlace* robot. A second *PickAndPlace* robot then loads the parts onto pallets on a second *Conveyor*. The *pallets*, which the *Conveyor* transports, are then unloaded with the third *PickAndPlace* robot onto another processing station. The *Drain* finally removes the parts from the plant.

We show how to configure the *PickAndPlace* robot and the *Conveyors*:

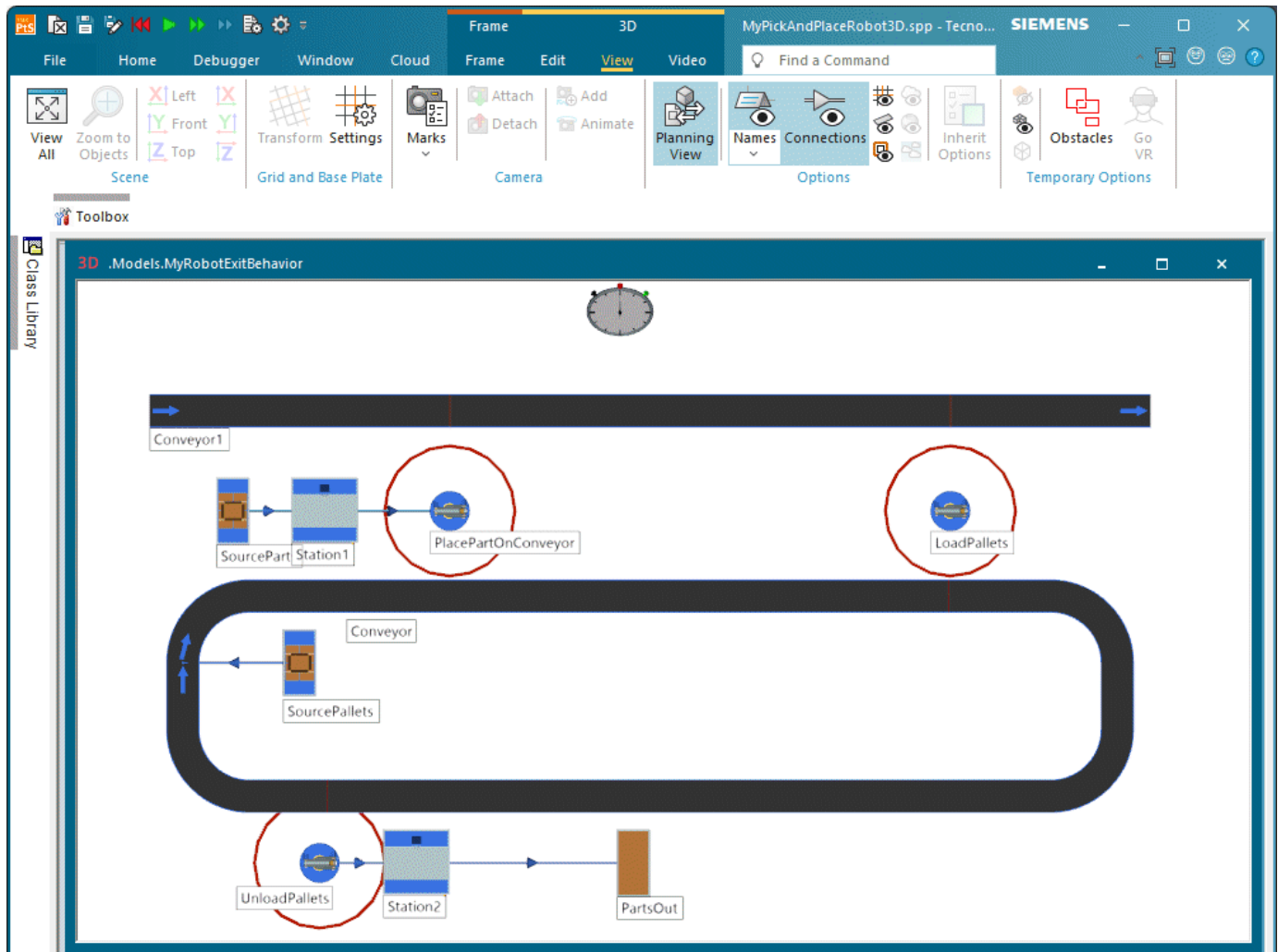
- With drag-and-drop. Here the *PickAndPlace* robot automatically creates the required sensors and controls and enters the corresponding angle into the **Angles Table**.

- Manually on the tab **Exit** in the list **Target Selection**. Here you have to create the required sensors and program the controls yourself.

In our example we will:

- Configure the Material Flow Objects**
- Configure the Robots**

The finished simulation model looks like this:



Back to **Pick and Place Parts with the PickAndPlace Robot**

## Configure the Material Flow Objects

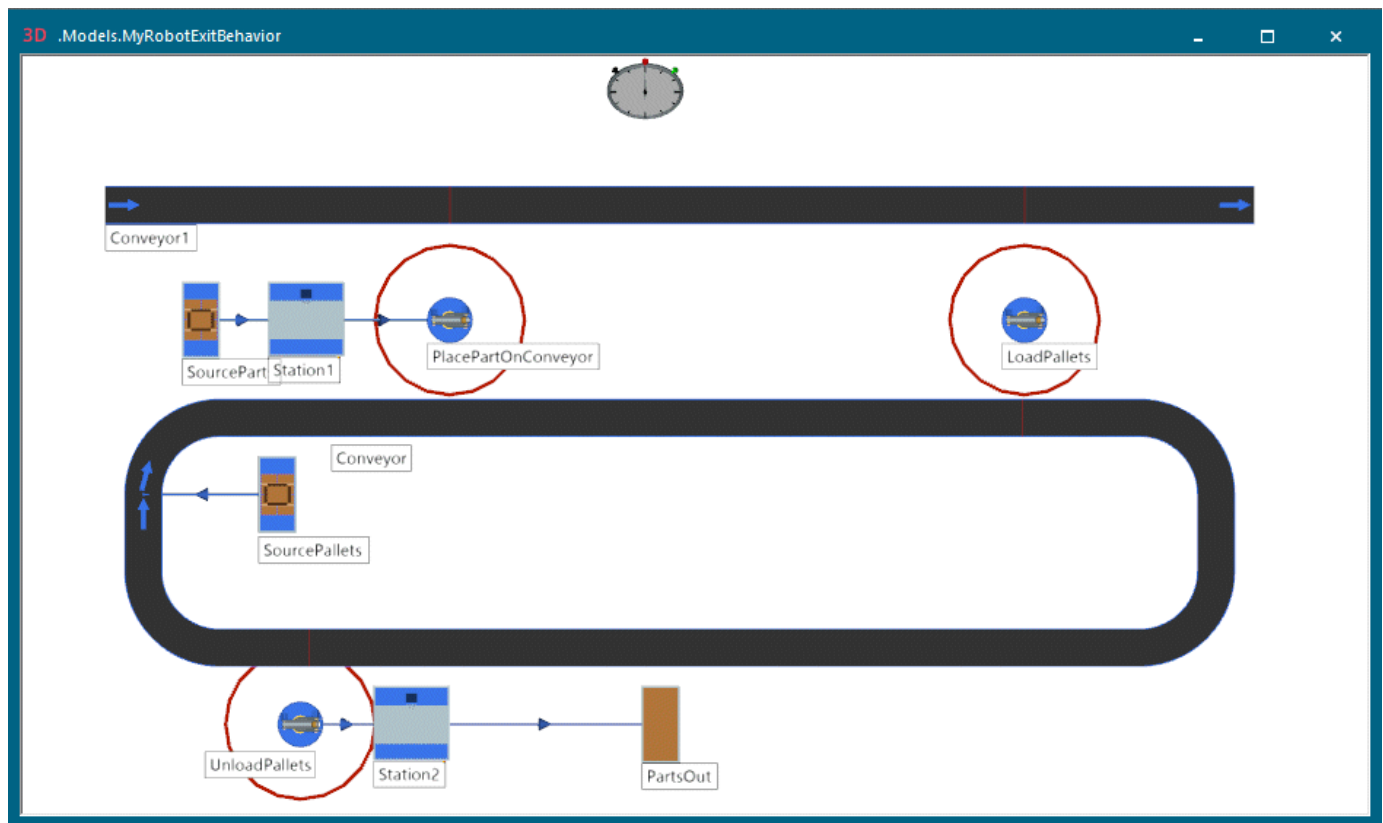
The *Source* named *SourceParts* produces parts and moves them to the *Station* named *Station1*. The *PickAndPlace* robot named *PlacePartOnConveyor* picks the part up, rotates to the *Conveyor* and places it onto the *Conveyor*. To model this, drag the *PickAndPlace* robot onto the *Conveyor* and select the

**Operation > Place part.** *Plant Simulation* then automatically creates the respective sensor on the *Conveyor*.

The *Conveyor* transports the placed part over a certain distance. A second *Source* named *SourcePallets* produces pallets for a second *Conveyor*. The *PickAndPlace* robot named *LoadPallets* loads the parts from the *Conveyor* onto the pallets. To model this, drag the *PickAndPlace* robot onto the *Conveyor* and select the **Operation > Load part.** *Plant Simulation* then automatically creates the respective sensor on the *Conveyor*.

The *PickAndPlace* robot named *UnloadPallets* unloads the parts from the pallets onto the *Station2*. To model this, drag the *PickAndPlace* robot onto the *Conveyor* and select the **Operation > Unload part.** *Plant Simulation* creates the corresponding control for unloading the pallet at the respective sensor. The corresponding exit behavior here is **Exit strategy or target control.** The fully processed parts are then moved from *Station2* to the *Drain* named *PartsOut* which removes them from the plant.

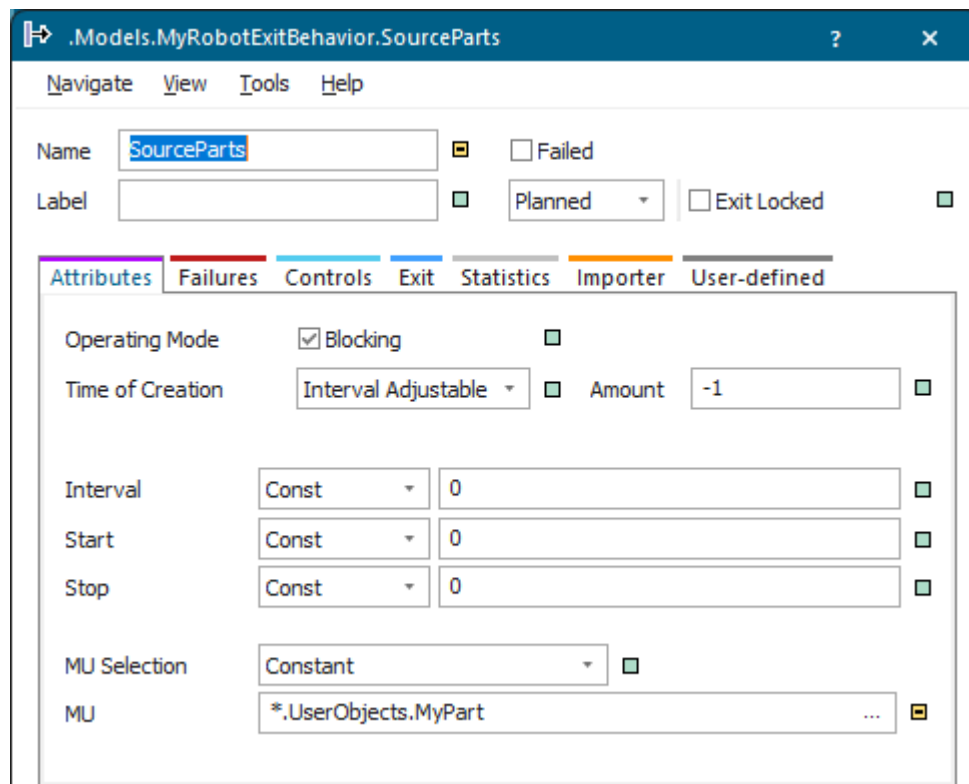
Insert the required objects so that they match the figure below:



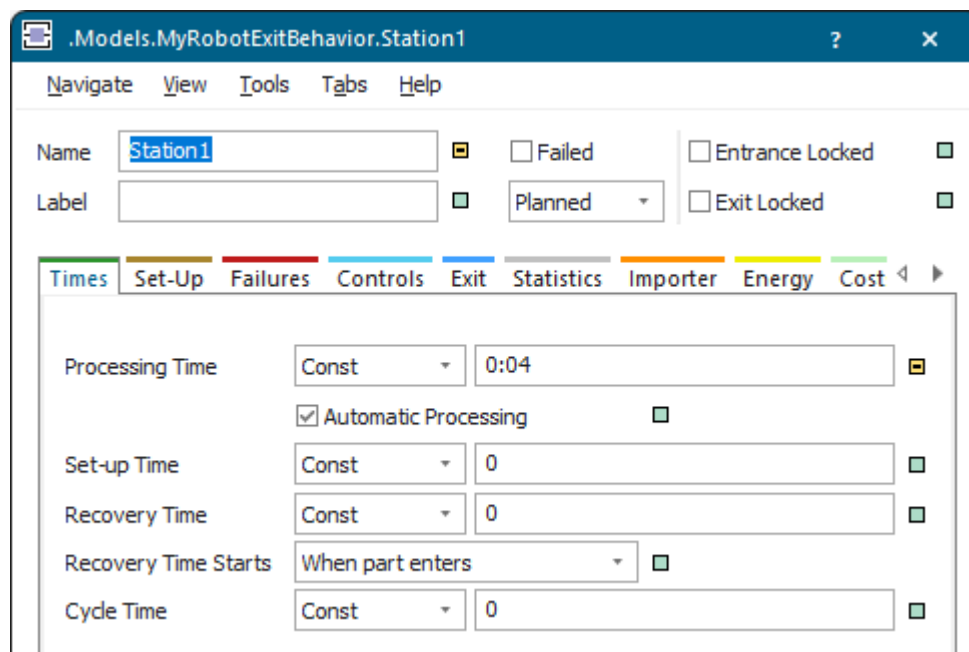
#### Note

Connect the *robots* with the *Stations* with *Connectors*. Make sure that the starting point of the conveyor is connected with its end point with a *Connector* so that the parts can move in a circle.

We use the *SourceParts* with the built-in settings. It produces an unlimited number of parts of type *MyPart*.



We entered a **Processing Time** of 4 seconds into the processing station named *Station1*.



For the *SourcePallets* we use these settings:

.Models.MyRobotExitBehavior.SourcePallets

Name:  ☐ Failed  
 Label:  ☐ Planned ☐ Exit Locked

Attributes | Failures | Controls | Exit | Statistics | Importer | User-defined

Operating Mode: ☒ Blocking ☐  
 Time of Creation:  ☐ Amount:  ☐  
 Interval:   ☐  
 Start:   ☐  
 Stop:   ☐  
 MU Selection:  ☐  
 MU:  ☐

- We produce 5 pallets of type *MyPallet*.
- For the *pallets* we use a **Length** of 0.55 meters, a **Width** of 0.45 meters, a **Height** of 0.33 meters, an **X-Dimension** of 1, a **Y-Dimension** of 1, and a **Z-Dimension** of 3.

.UserObjects.MyPallet

Name:  ☐ Stopped  
 Label:  ☐ Conveying Direction:

Attributes | Routing | Product Statistics | Statistics | Costs | User-defined

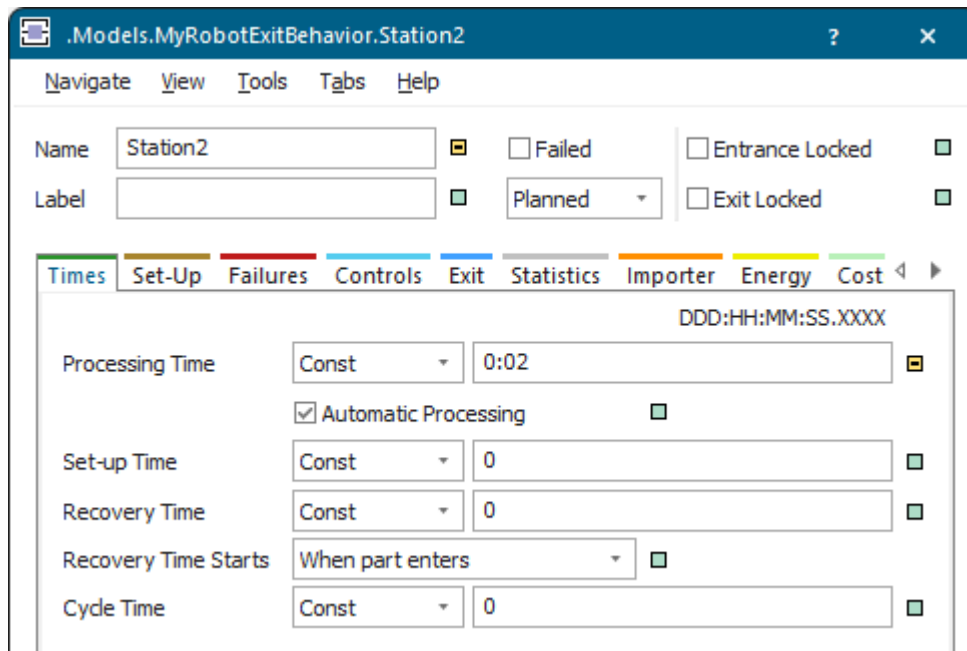
MU Size  
 Length:  ☐ m  
 Width:  ☐ m  
 Height:  ☐ m

Booking Point  
 0.5 ☐ [0..1] (0.27 m)  
 0.5 ☐ [0..1] (0.22 m)  
 0 ☐ [0..1] (0.00 m)

X-Dimension:  ☐  
 Y-Dimension:  ☐  
 Z-Dimension:  ☐

☐ Workpiece Carrier ☐  
☐ Fill Whole Layer ☐

For the processing station named *Station2* we entered a **Processing Time** of 2 seconds.



We use the *Conveyors* with the built-in settings.

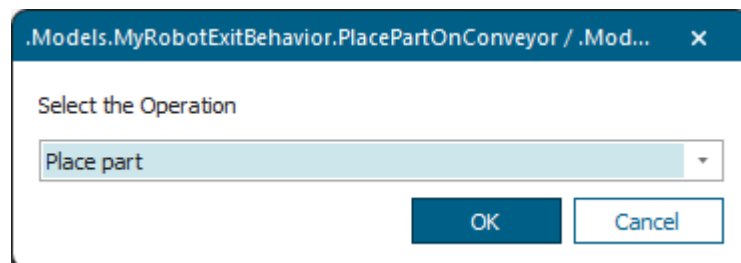
Go to [Configure the Robots](#)

Back to [Load and Unload Parts at the Sensor of a Conveyor](#)

## Configure the Robots

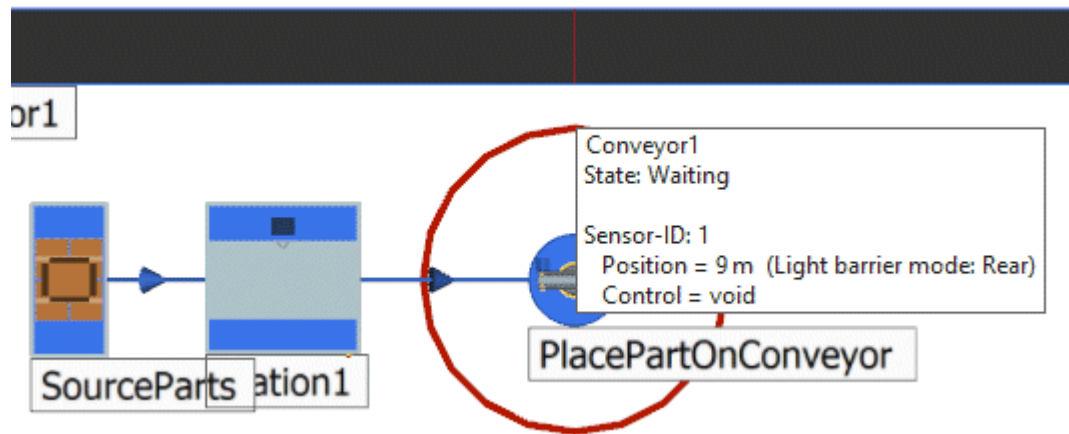
We used drag-and-drop to configure the individual *robots*.

- The robot *PlacePartOnConveyor* is to place parts onto *Conveyor1*. To model this, drag it onto a position of your choice on *Conveyor1*, select the **Operation > Place part**, and click **OK**.



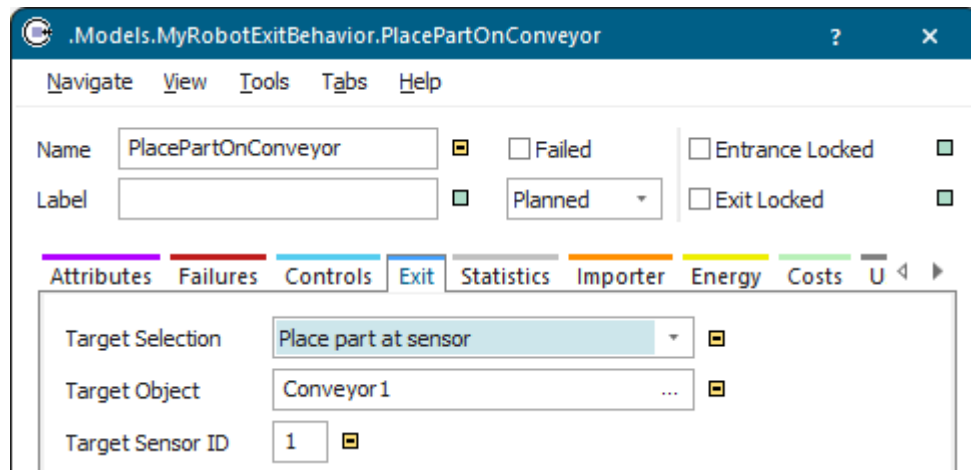
*Plant Simulation* automatically does the following:

- It creates a sensor on *Conveyors* with these settings:

**Note**

Zoom in the *Frame* to show the sensors more clearly.

- It enters these settings on the tab **Exit** of the robot *PlacePartOnConveyor*.



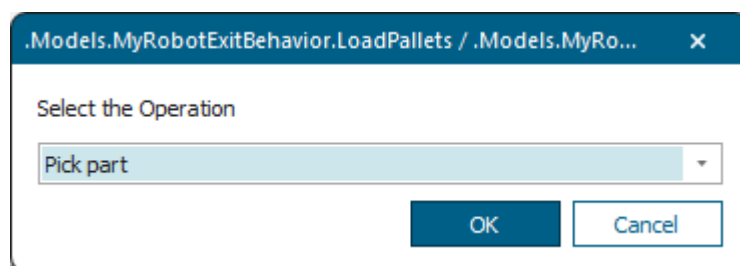
- It enters these settings into the **Angles Table** on the tab **Attributes** of the robot *PlacePartOnConveyor*.

The screenshot shows the 'AnglesTable' dialog box. It contains a table with the following data:

|   | Name      | Angle  |
|---|-----------|--------|
| 1 | Station1  | 180.00 |
| 2 | Conveyor1 | 268.94 |

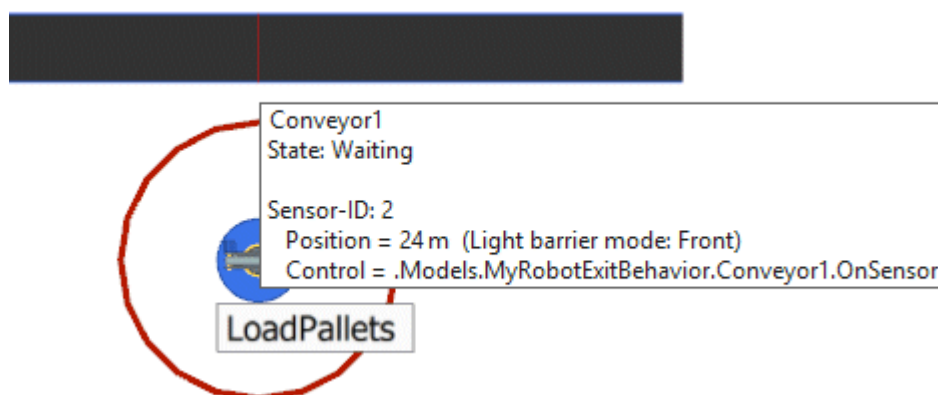
Buttons: OK, Cancel, Apply

- In a first step the robot *LoadPallets* is to pick the parts from *Conveyor1*. To model this, drag it to a position of your choice on the *Conveyor*, select the **Operation > Pick part**, and click **OK**.

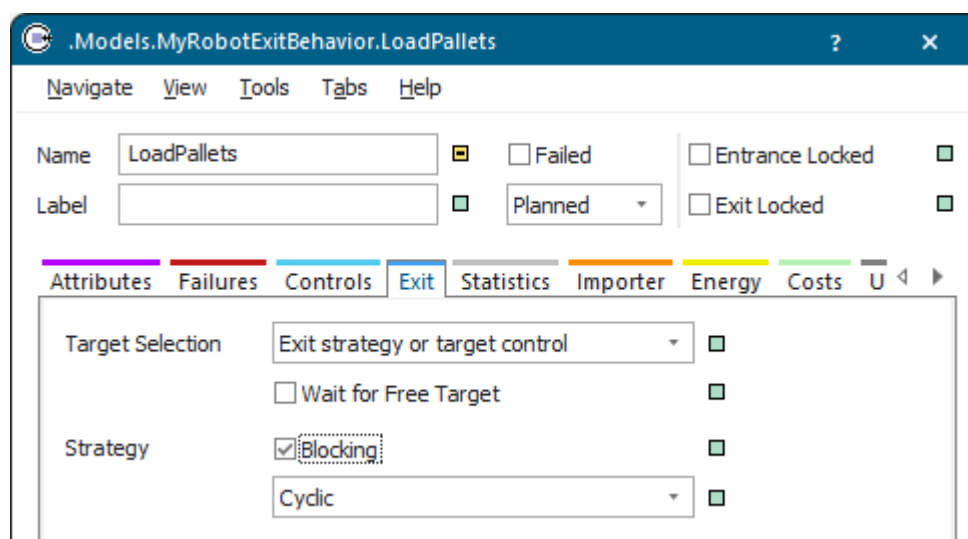


*Plant Simulation* automatically does the following:

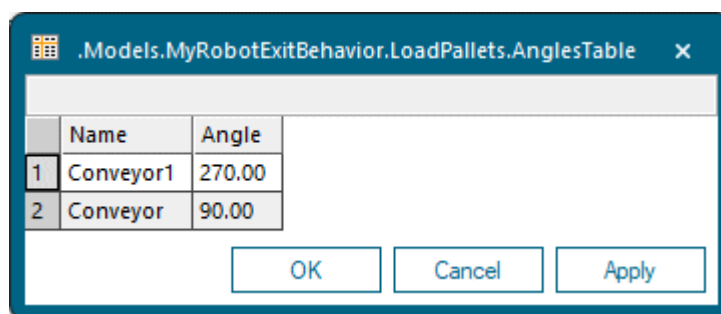
- It creates a sensor on *Conveyor* with these settings:



- It creates a **Sensor Control** as a *user-defined attribute* of *Conveyor1*.
- It enters these settings on the tab **Exit** of the robot named *LoadPallets*.

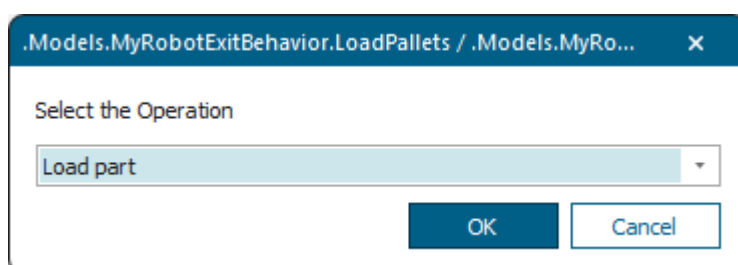


- It enters these settings into the **Angles Table** on the tab **Attributes** of the robot named *LoadPallets*.

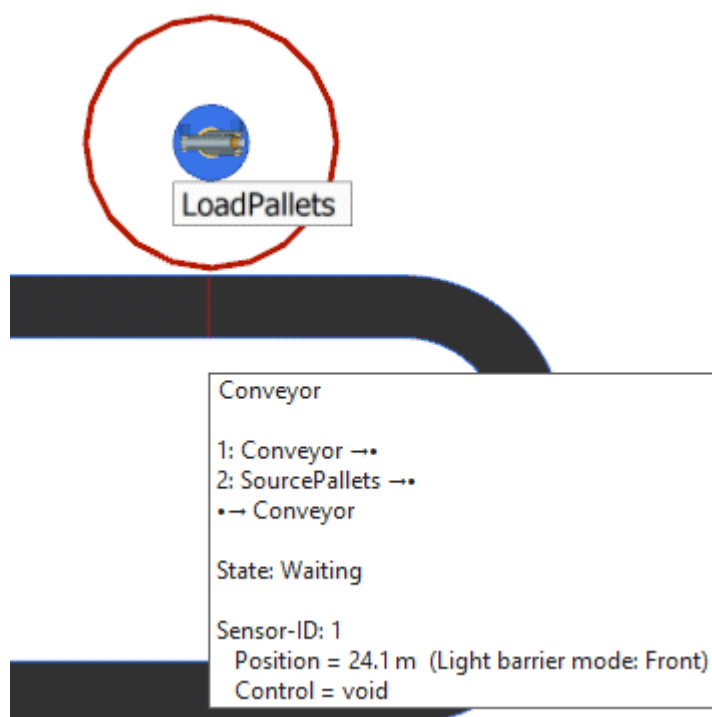


- In a second step the robot *LoadPallets* is to load the pallets which move on the *Conveyor*. To model this, drag the robot onto a position of your choice on the *Conveyor*, select the **Operation > Load part**, and click **OK**.

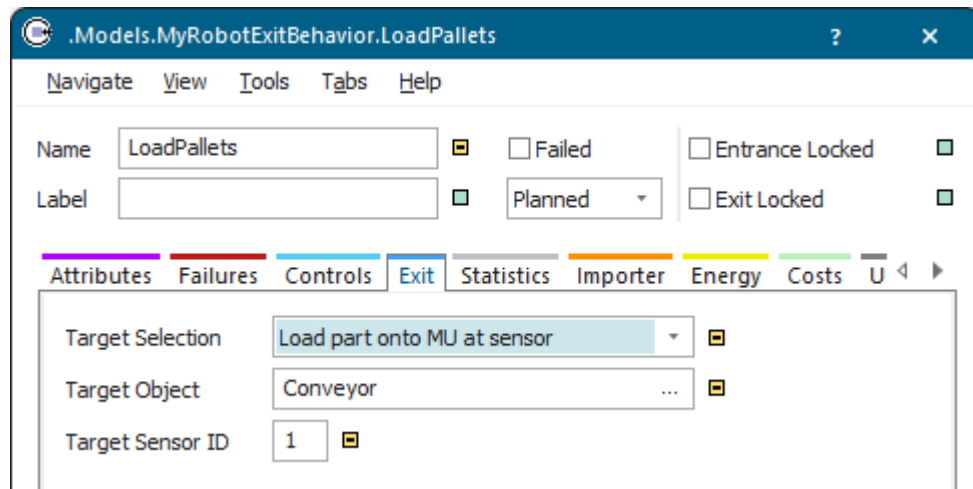
*Plant Simulation* automatically does the following:



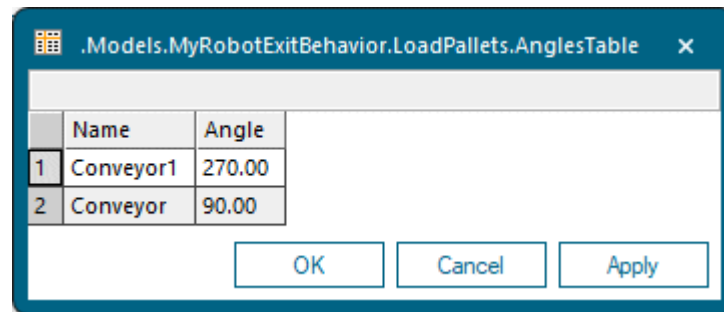
- It creates a sensor on the *Conveyor* with these settings:



- It creates a **Sensor Control** as a *user-defined attribute* of the *Conveyor*.
- It enters these settings on the tab **Exit** of the robot named *LoadPallets*.



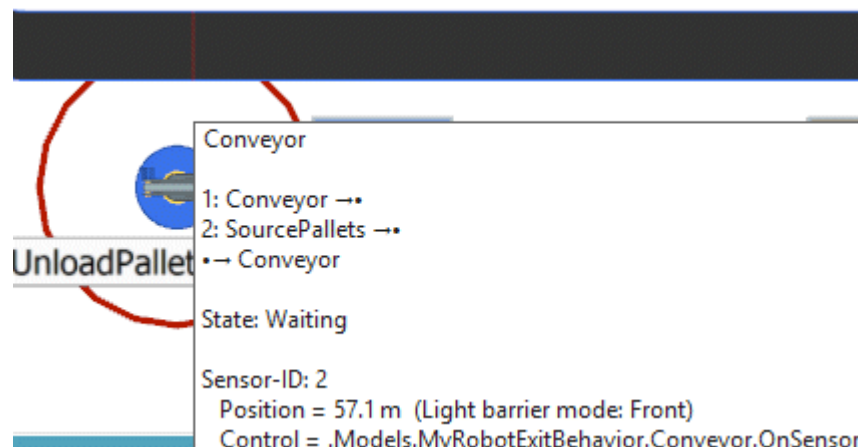
- It enters these settings into the **Angles Table** on the tab **Attributes** of the robot named *LoadPallets*.



- As the last step the robot *UnloadPallets* unloads the parts from the pallets and loads them onto the station named *Station2*. To model this, drag the robot onto a position of your choice on the *Conveyor*, select the **Operation > Unload part**, and click **OK**.

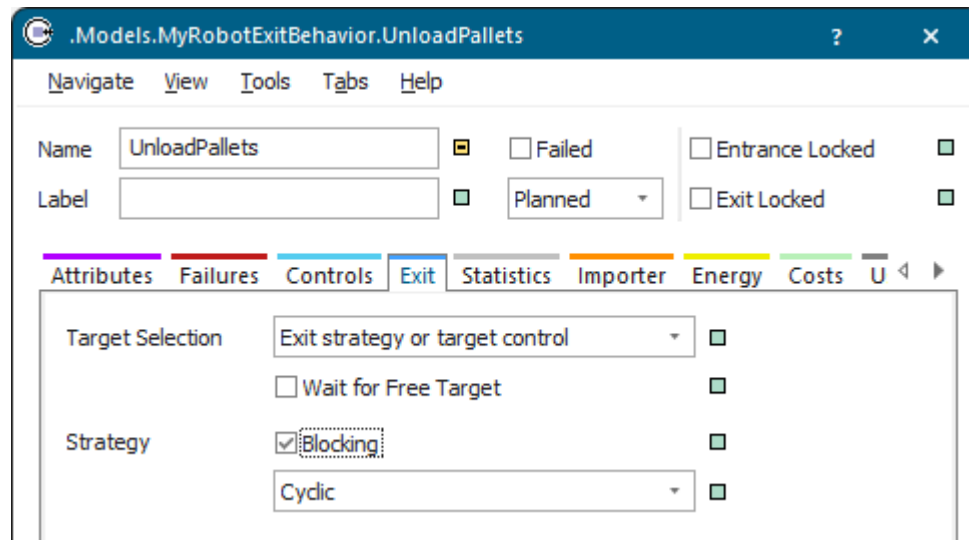
*Plant Simulation* automatically does the following:

- It creates a sensor on the *Conveyor* with these settings:

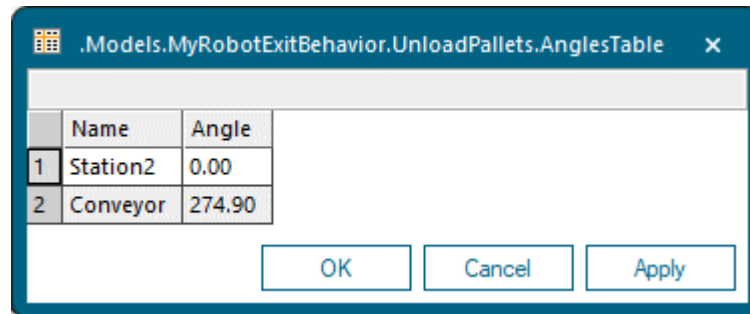


It creates a **Sensor Control** as a *user-defined attribute* of the *Conveyor*.

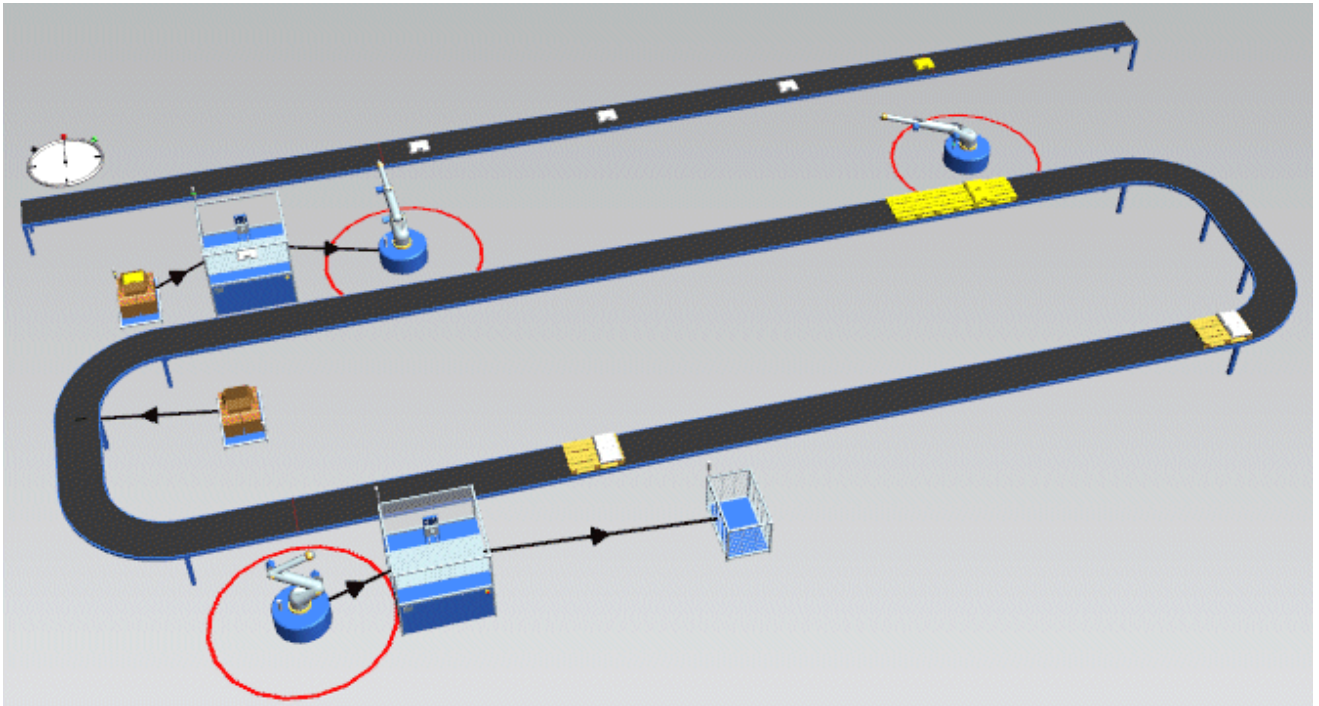
- It enters these settings on the tab **Exit** of the robot named *UnloadPallets*.



- It enters these settings into the **Angles Table** on the tab **Attributes** of the robot named *UnloadPallets*.



- Run the simulation and watch what's happening.



During the simulation *Plant Simulation* loads three parts onto the pallet.

**.UserObjects.MyPart**

Navigate View Tools Help

Name: MyPart ☐ Stopped

Label:  Conveying Direction: 0 (forward)

Attributes Routing Product Statistics Costs User-defined

MU Size

Length: 0.3 m

Width: 0.3 m

Height: 0.1 m

Booking Point

0.5 [0..1] (0.25 m)

0.5 [0..1] (0.20 m)

0 [0..1] (0.00 m)

**.UserObjects.MyPallet**

Navigate View Tools Help

Name: MyPallet ☐ Stopped

Label:  Conveying Direction: 0 (forward)

Attributes Routing Product Statistics Statistics Costs User-defined

MU Size

Length: 1 m

Width: 1 m

Height: 0.1 m

Booking Point

0.5 [0..1] (0.27 m)

0.5 [0..1] (0.22 m)

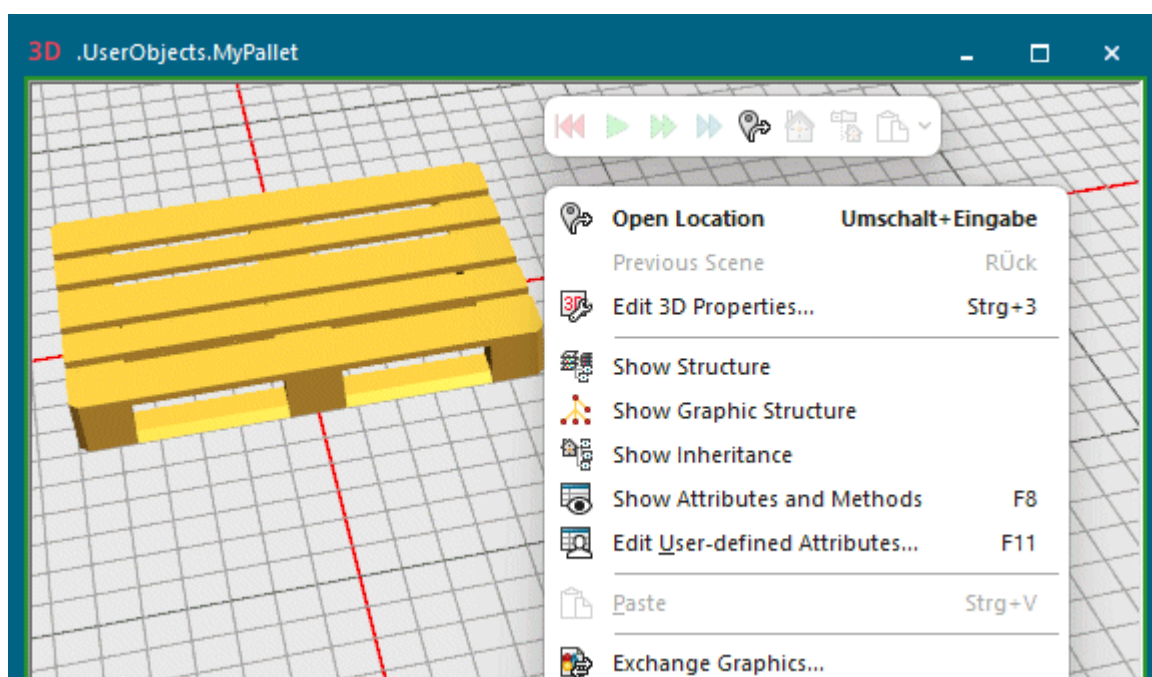
0 [0..1] (0.00 m)

X-Dimension: 1 ☐ Workpiece Carrier ☐

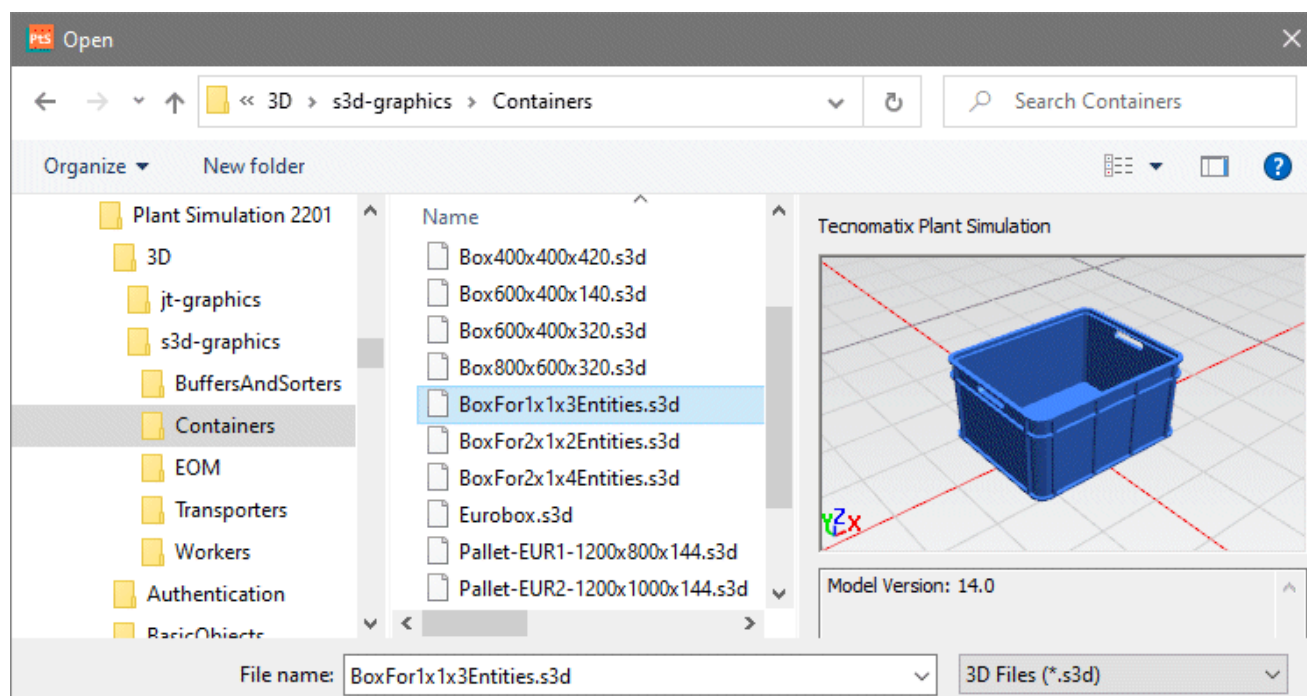
Y-Dimension: 3 ☐ Fill Whole Layer ☐

Z-Dimension: 1 ☐

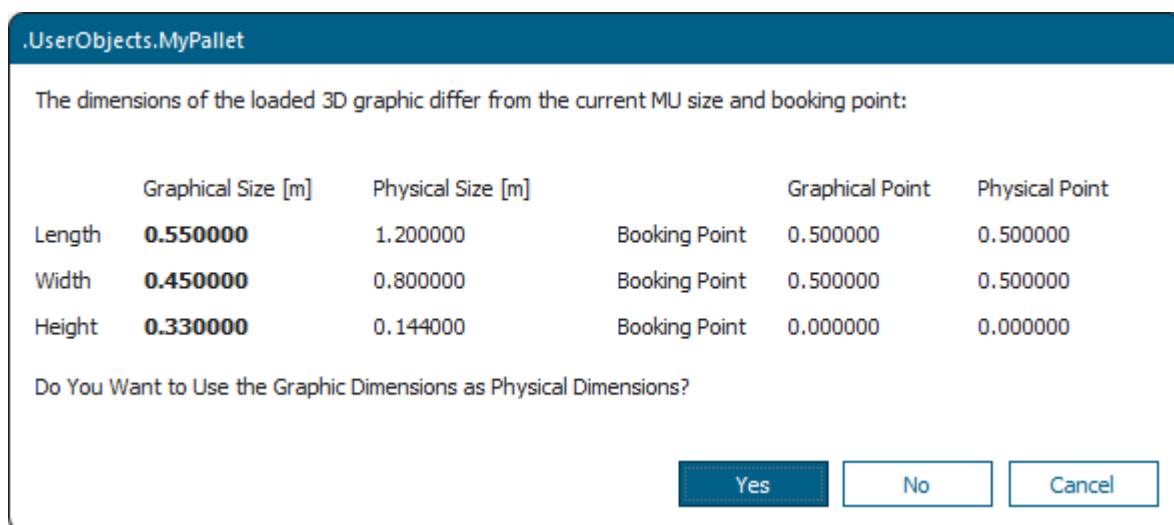
In 3D, on the other hand, only two parts are loaded onto the pallet. This is because the standard pallet in 3D receives four parts on the predefined loading spaces of the animation area. To fix this, we are going to exchange the 3D graphic of the pallet with a box that accommodates three parts stacked onto each other. Open the pallet in 3D, click the right mouse button into the background of the window and select **Exchange Graphics**.



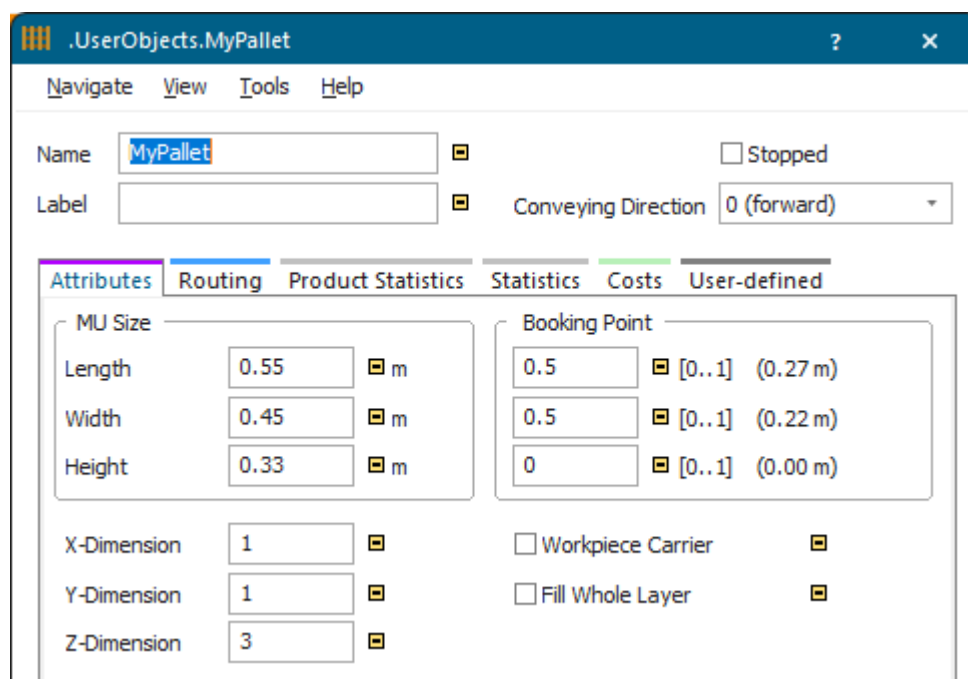
Select **BoxFor3Entities.s3d** in the folder that is going to be opened.



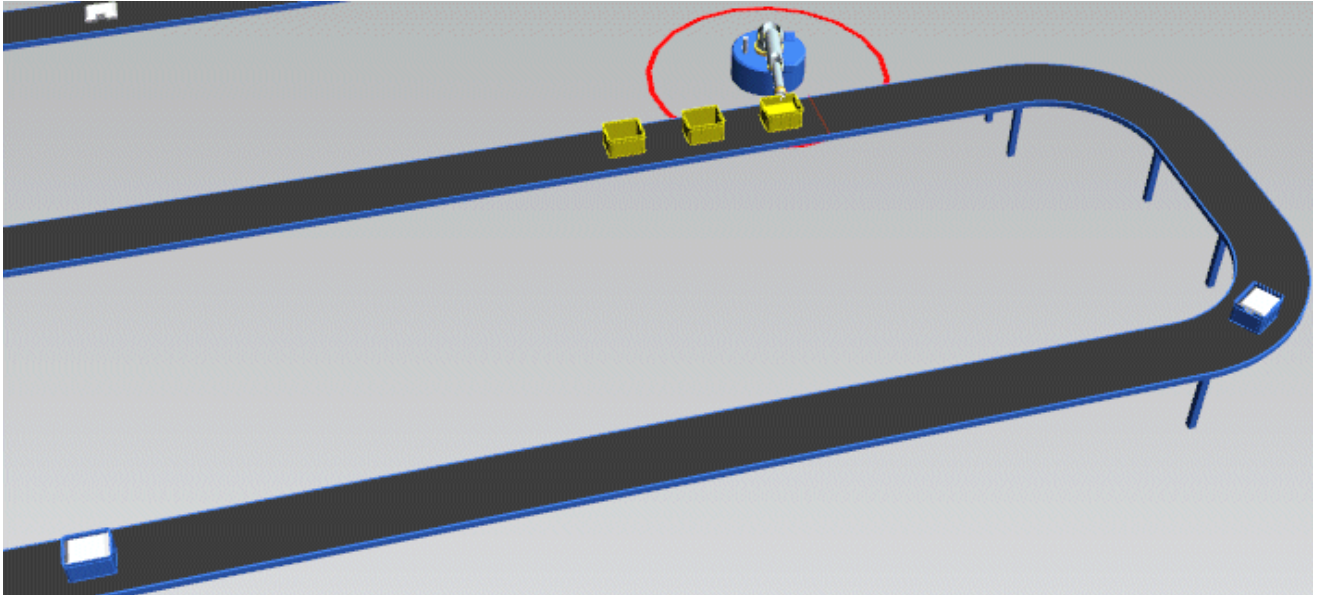
As we want to use the graphic dimensions of the transport box in 3D, we click **Yes** in the message dialog that is shown.



To stack the parts in the transport box, we type in 3 as the **Z-Dimension**.



Now the animation also looks good in 3D. Three parts each are placed into a box, one stacked on the other, and are then moved on.



Back to [Load and Unload Parts at the Sensor of a Conveyor](#)

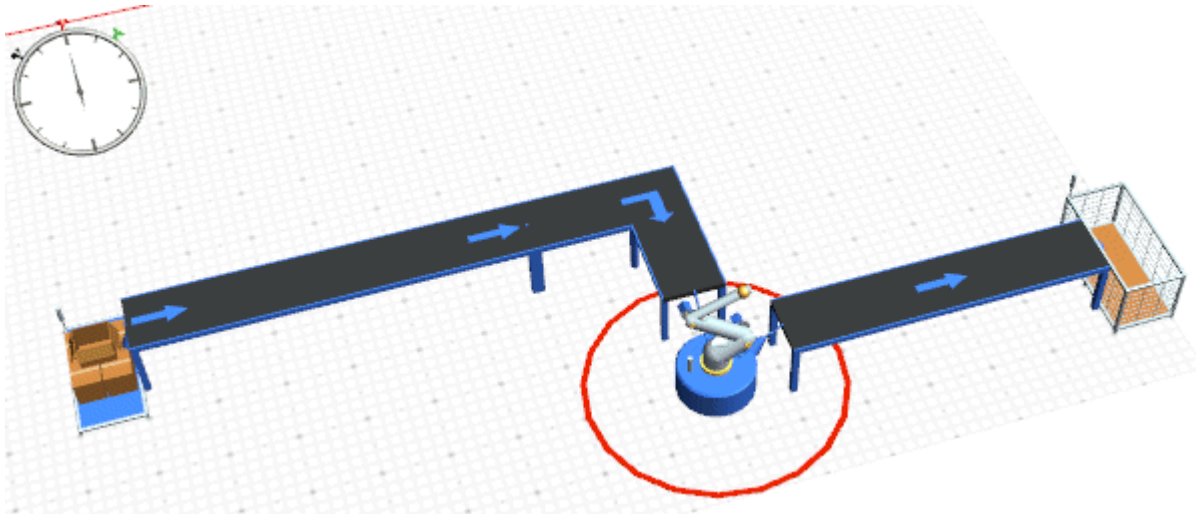
## Set the Conveying Direction of the Parts

In this model we show how you can set the **MU conveying direction** in the *PickAndPlace robot*. The conveying direction is of importance because the objects *Turnplate*, *Turntable*, *AngularConverter*, and *Conveyor* change the conveying direction of the MUs in the simulation model.

As soon as they do this, *Plant Simulation* uses the previous length of the *MU* as its new width and the previous width as the new length. The *PickAndPlace robot* can also pick the part up and rotate it arbitrarily and then place it as it is required for further processing/conveying, just as a robot does in a real factory.

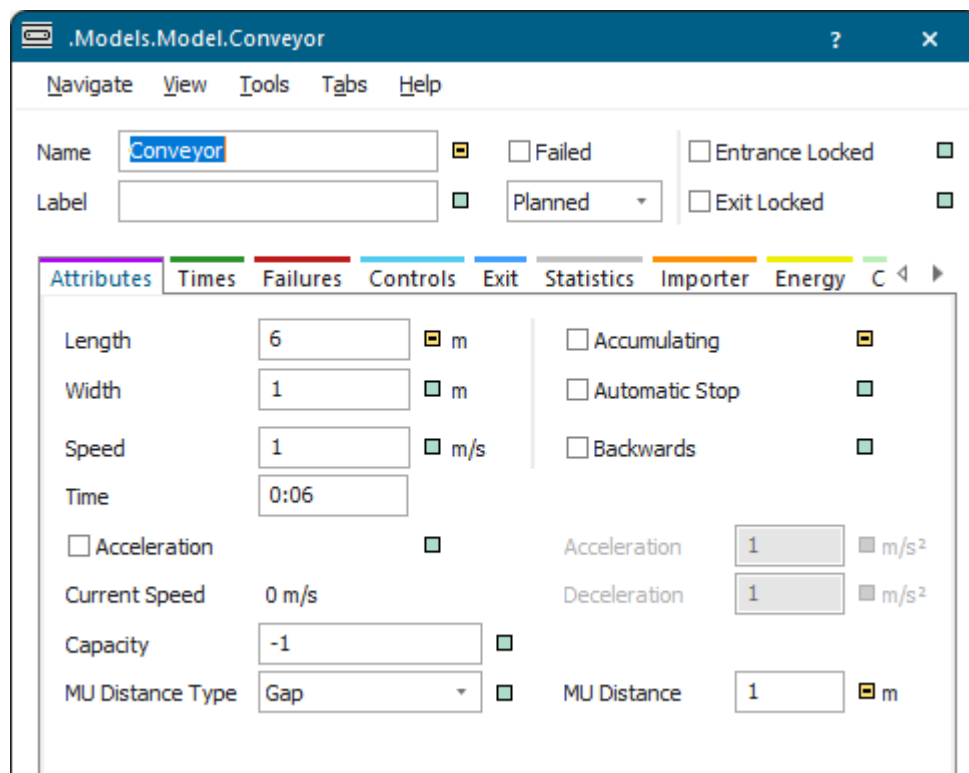
In our model the *Source* produces parts and moves them on to a *Conveyor*. The *AngularConverter* conveys the part around the corner and changes the conveying direction of the part from **Forward** to **Lateral right**. The *PickAndPlace robot* picks the part up and places it onto the *Conveyor* in the direction, which we selected as the **MU Conveying Direction** for the *PickAndPlace robot*. The *Conveyor* moves the part to the *Drain*, which removes it from the installation.

The finished simulation model looks like this:

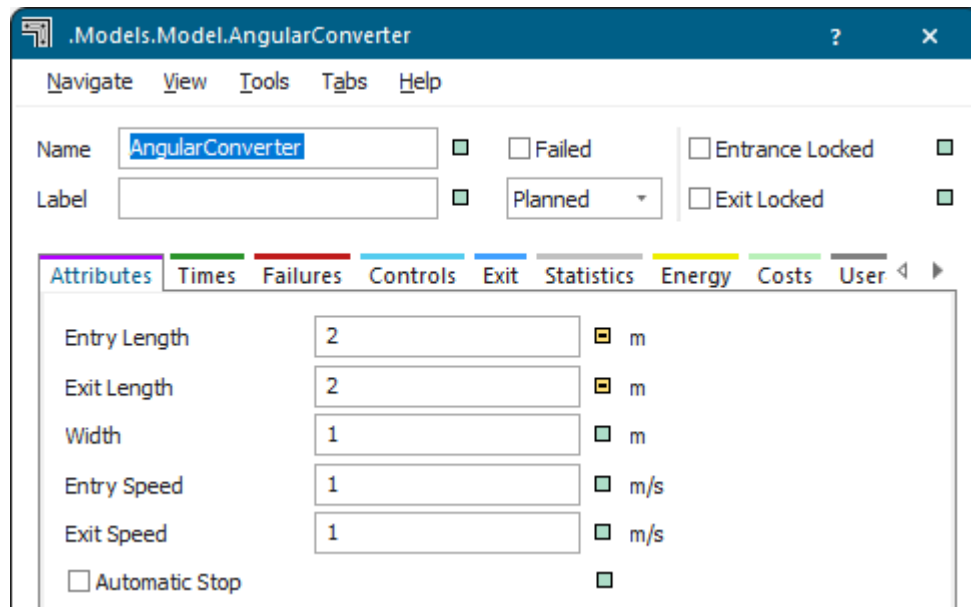


To create the simulation model, insert the objects so that they match the figure above.

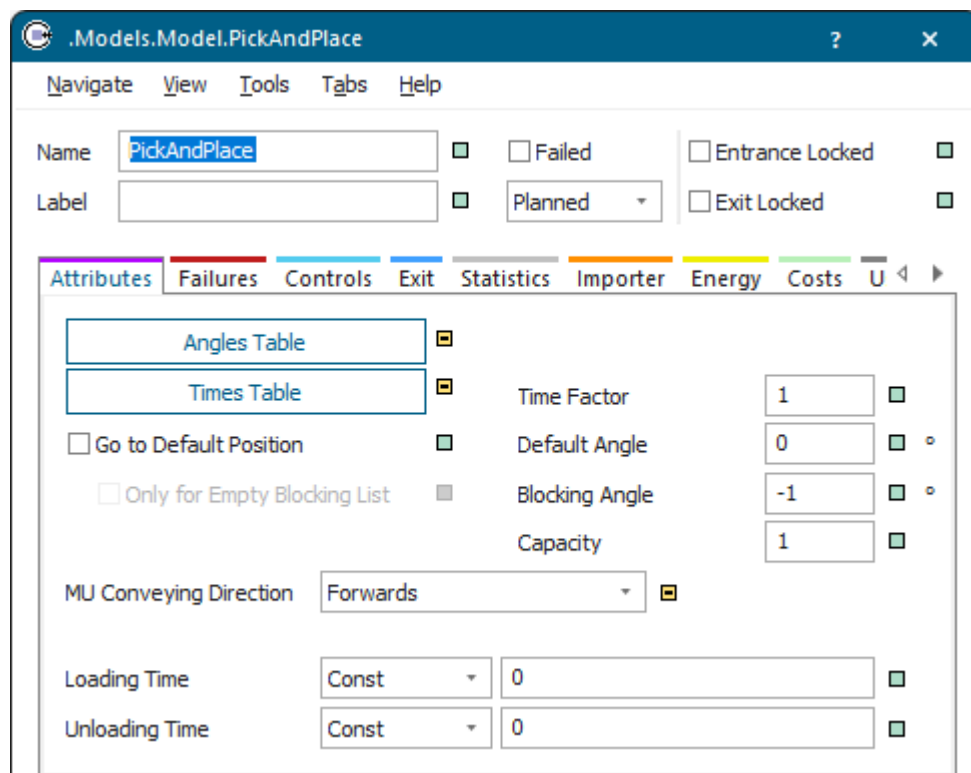
- Our *Source* uses the default settings.
- Our *Conveyor* is 6 meters long and is non-accumulating. The gap between the individual parts that are conveyed is 1 meter.



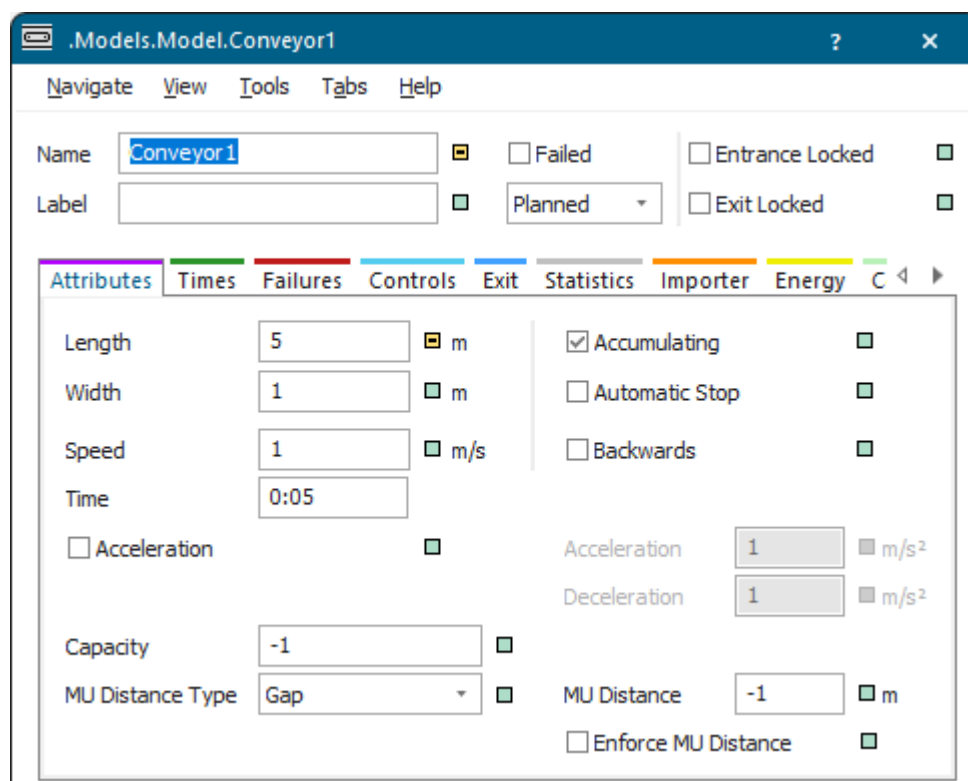
- Our *AngularConverter* has an **Entry Length** and an **Exit Length** of 2 meters each. We did not change any other settings.



- In the *PickAndPlace* robot we selected the **MU Conveying Direction > Forwards**.

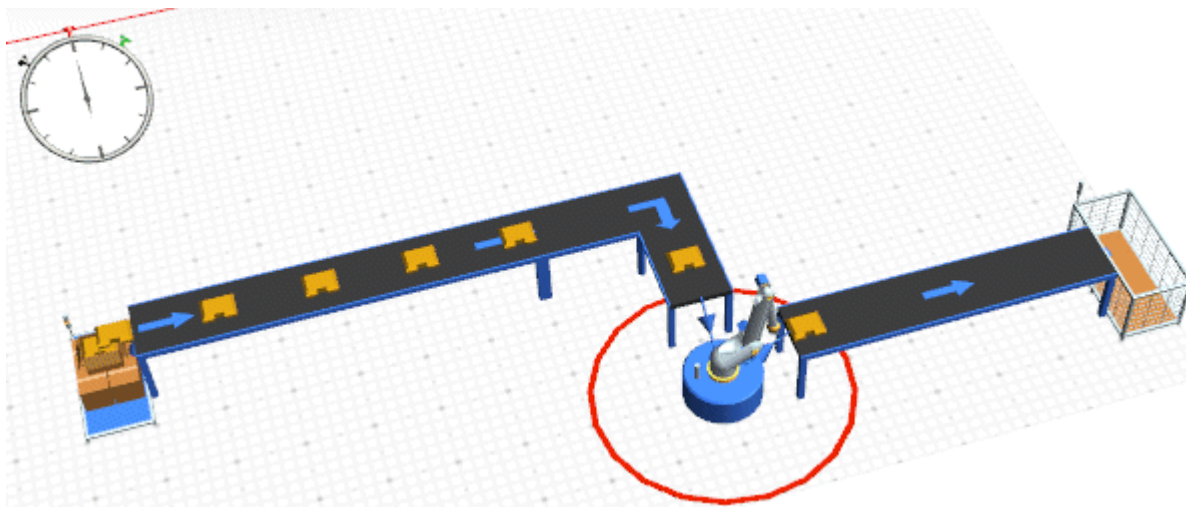


- Our second Conveyor, named *Conveyor1* is 5 meters long and is **Accumulating**.



- Our *Drain* uses the default settings.

Start the simulation and watch how the *PickAndPlace* robot picks the parts from the *AngularConverter*, how it rotates to *Conveyor1*, and places the part there with the **MU Conveying Direction > Forwards**. Here the robot rotates the part so that it will be placed forward on the successor and then places it on the successor.



You can run further experiments with the different settings for the **MU Conveying Direction** in the model and thus find the setting that best fits your modeling needs.

Back to [Pick and Place Parts with the PickAndPlace Robot](#)

## Unload Stacked Parts

### Unload Stacked Parts

The **Z-Dimension** of the *Store*, the **Z-Dimension** of the *Transporter*, and the **Z-Dimension** of the *Container* allow you to stack parts on them.

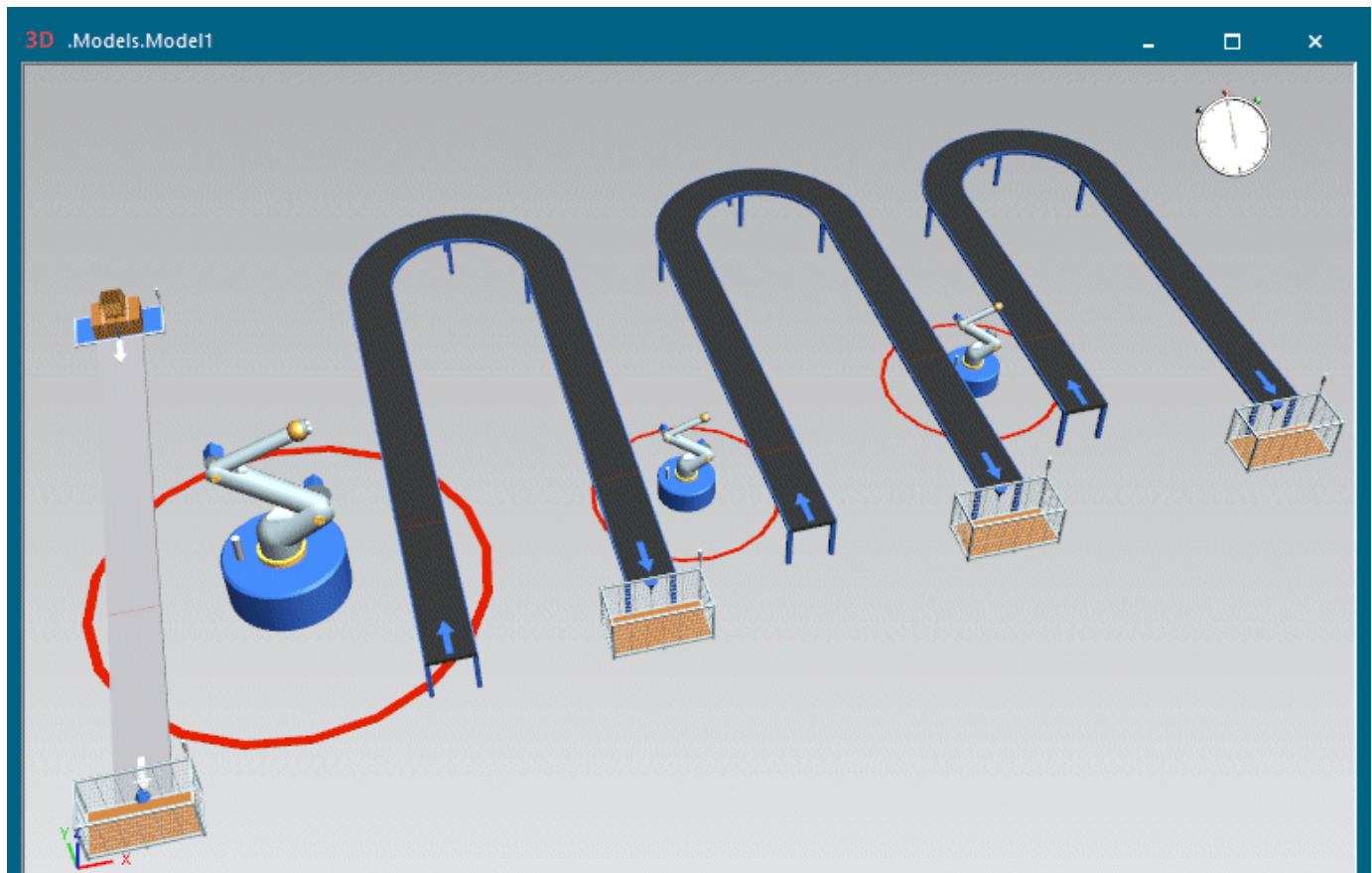
In our sample model a *Transporter* transports pallets on which boxes are stacked. The boxes themselves contain three stacked parts.

The first *PickAndPlace robot* unloads the pallets from the *Transporter* and places them onto a conveyor.

The second *PickAndPlace robot* unloads the boxes from the pallets and places them onto the second conveyor.

The third *PickAndPlace robot* unloads the stacked parts from the boxes and places them onto the third conveyor which moves them on. A *Drain* at the respective end of the three conveyors removes the parts from the plant.

The finished simulation model looks like this:



To let the robots know where to unload and load parts, drag the respective predecessor and successor onto the respective robot and drop it there. The **Angles Tables** of the robots then show the respective predecessor and successor.

| .Models.Model1.PickAndPlace.AnglesTable |          |        |
|-----------------------------------------|----------|--------|
|                                         | Name     | Angle  |
| 1                                       | Track    | 202.17 |
| 2                                       | Conveyor | 333.43 |

OK Cancel Apply

In the following you will:

- Configure the Transporter, the Pallet, and the Box
- Configure the Source and the Track
- Configure the Conveyors
- Configure the Robots Unloading the Stacks

## Back to [Pick and Place Parts with the PickAndPlace Robot](#)

### Configure the Transporter, the Pallet, and the Box

As the *Transporter*, the *Pallets*, and the *boxes* are always going to be the same, we configure their classes in the *Class Library*.

- For the *Transporter* select the settings below:

The screenshot shows the configuration window for a transporter. The window has a title bar with a question mark and a close button. Below the title bar is a menu bar with 'Navigate', 'View', 'Tools', 'Tabs', and 'Help'. The main area is divided into several sections:

- Name:** 'Transporter' (highlighted in blue)
- Label:** (empty)
- Stopped:** ☐
- Failed:** ☐
- Planned:** ☒ (selected)

Below these are tabs for 'Attributes', 'Load Bay', 'Routing', 'Failures', 'Controls', 'Battery', and 'Product Statistic'. The 'Attributes' tab is active, showing the following settings:

- MU Size:**
  - Length: 1.6 m
  - Width: 0.8 m
  - Height: 0.4 m
- Booking Point:**
  - 0.5 [0..1] (0.80 m)
  - 0.5 [0..1] (0.40 m)
  - 0 [0..1] (0.00 m)
- Speed:** 1 m/s
- Acceleration:** ☐
- Current Speed:** 0 m/s
- Is Tractor:** ☐
- Start Delay Duration:** Const, 0
- Backwards:** ☐
- Acceleration:** 1 m/s<sup>2</sup>
- Deceleration:** 1 m/s<sup>2</sup>

Use the standard setting of 1 for the **Z-Dimension** of the storage area of type **Store**. Thus, the *Transporter* does not stack any *Pallets* one onto the other.

**.MUs.Transporter**

Navigate View Tools Tabs Help

Name:  ☐ Stopped

Label:  ☐ Failed

Attributes Load Bay Routing Failures Controls Battery Product Statistics

Type:  ☐ Z-Dimension:  ☐

X-Dimension:  ☐ Y-Dimension:  ☐

Length:  ☐ m Capacity:  ☐

Speed:  ☐ m/s ☒ Accumulating ☐

☐ Acceleration ☐ Backwards ☐

Acceleration:  ☐ m/s<sup>2</sup>

Deceleration:  ☐ m/s<sup>2</sup>

- For the *Container*, our *Pallet*, select the settings below. We entered 3 as the **Z-Dimension**; this way the *Pallet* can stack 3 *Boxes* onto each other.

**.MUs.Container**

Navigate View Tools Help

Name:  ☐ Stopped

Label:  ☐ Conveying Direction:

Attributes Routing Product Statistics Statistics Costs User-defined

MU Size

Length:  ☐ m

Width:  ☐ m

Height:  ☐ m

Booking Point

☐ [0..1] (0.60 m)

☐ [0..1] (0.40 m)

☐ [0..1] (0.00 m)

X-Dimension:  ☐

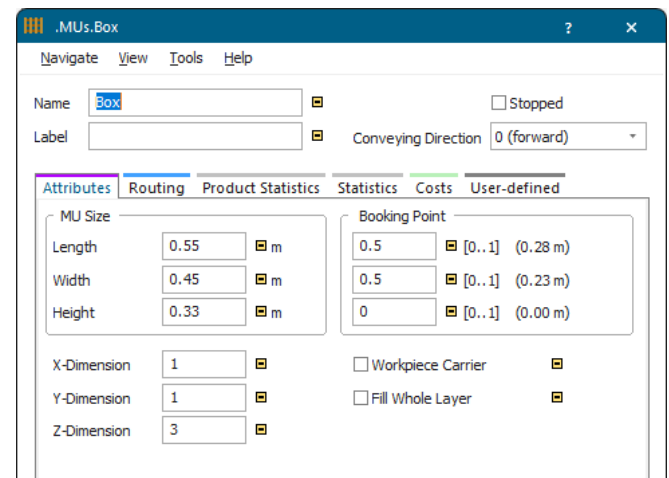
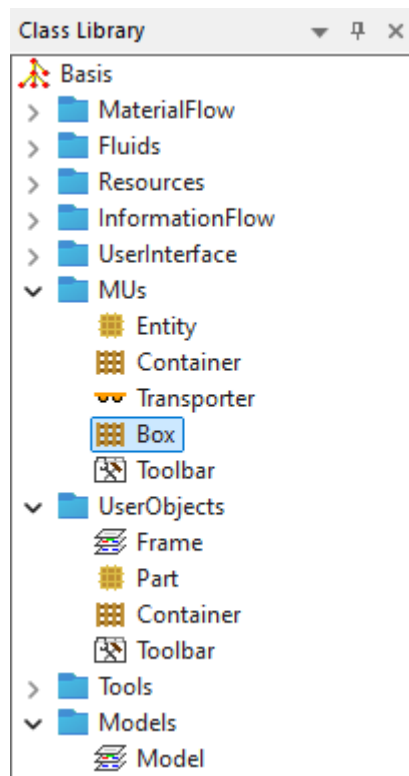
Y-Dimension:  ☐

Z-Dimension:  ☐

☐ Workpiece Carrier ☐

☐ Fill Whole Layer ☐

- To create the box, duplicate the *Container* in the *Class Library*. Then rename the duplicated object to *Box* and select the settings below. We entered 3 as the **Z-Dimension**, this way three parts can be stacked in the *Box*. The **Length** and **Width** make sure that four *Boxes* fit on a *Pallet*.



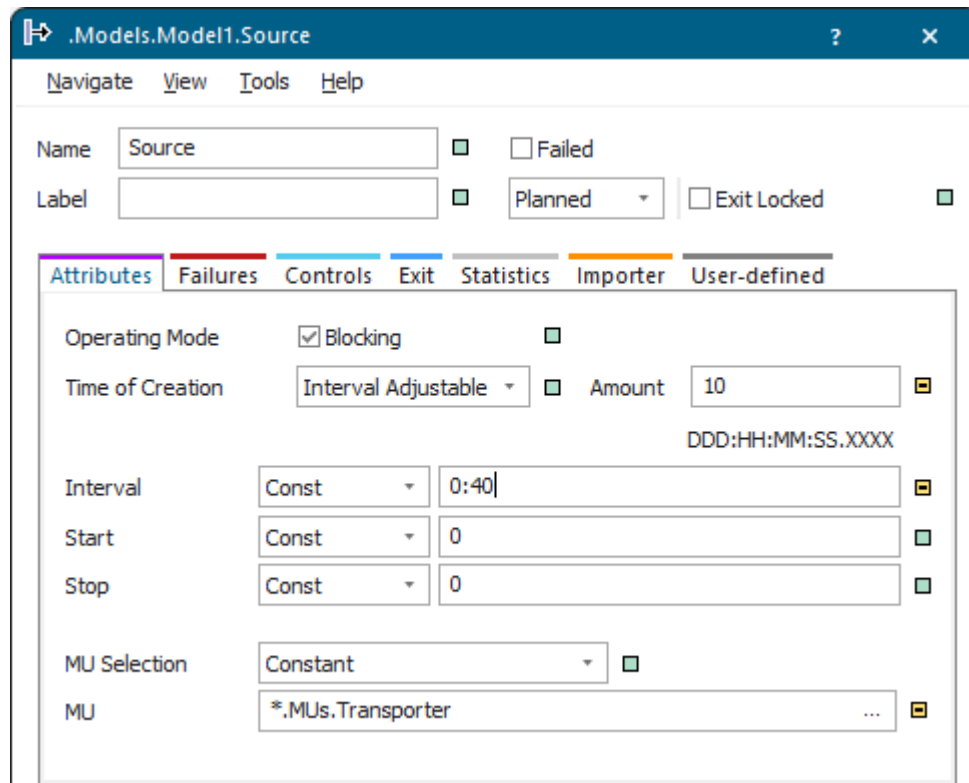
## Back to [Unload Stacked Parts](#)

### Configure the Source and the Track

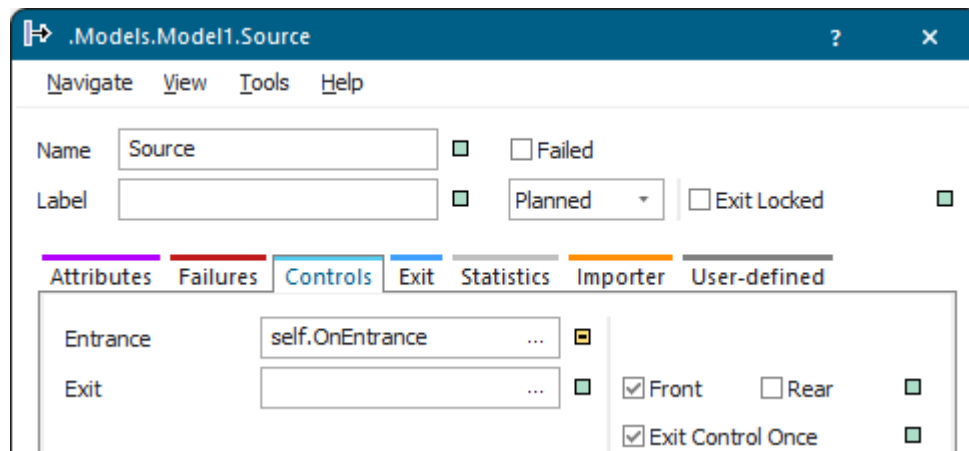
In this step we configure the *Source*, which creates the parts.

Proceed like this:

- Create 10 *Transporters* every 40 seconds.



- Determine the parts that are loaded onto the *Transporter* in the **Entrance Control**.



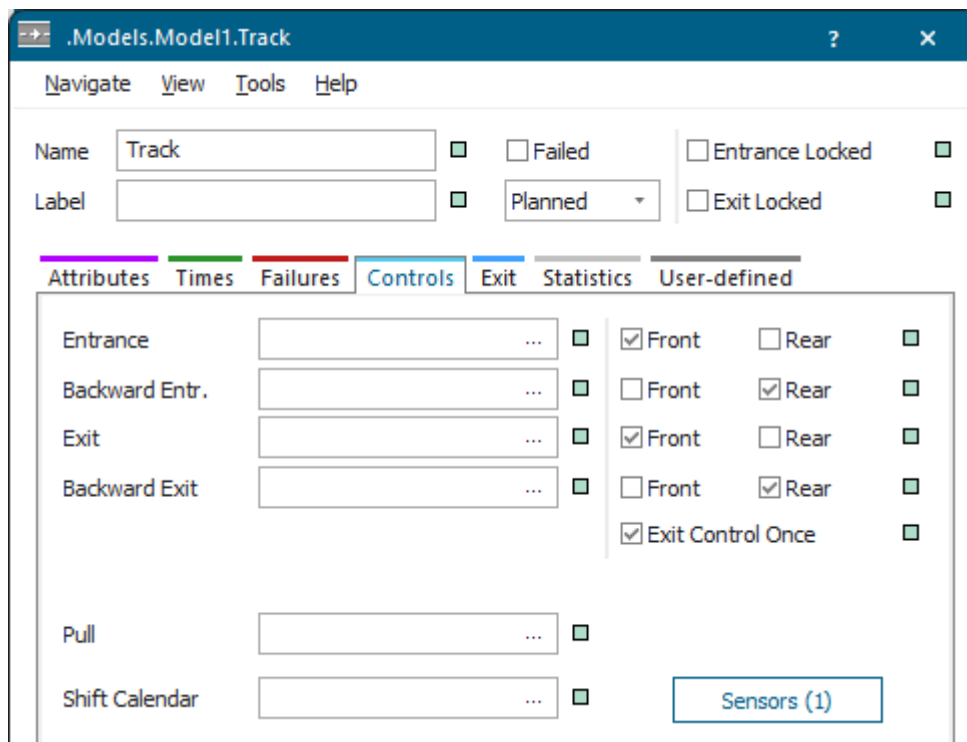
First, set the Length of the *Transporter* to 3.2 meters. You could also have done this in the class of the *Transporter* in the *Class Library*. We recommend to not change the standard settings of the built-in objects if possible. If you customize them and then merge models later on or if you update the class library later on, your customized settings are going to be discarded and replaced by the standard settings. You then have to make your customizations again.

Then, create a *Pallet* and continue creating *Boxes* on it until the *Pallet* is filled with *Boxes*. In the last step we are going to produce parts until the *Boxes* are full as well.

The respective source code looks like this:

```
@.Length := 3.2
while not @.Full
  var pallet := .MUs.Container.create(@)
  while not pallet.Full
    var box := .MUs.Box.create(pallet)
    while not box.Full
      .MUs.Part.create(box)
    end
  end
end
end
```

Then, configure the *Track* on which the *Transporter* drives.



- First, create a **sensor** at which the robot unloads the *Pallets* from the *Transporter*.

- Next, create the **Sensor Control** which stops the *Track* until the *Transporter* has been unloaded all the way. Then the *Transporter* drives on and is going to be removed from the installation in the *Drain*.

The respective source code looks like this:

```
param SensorID: integer, Front: boolean
@.stopped := true
var Destination:object := PickAndPlace
while not @.Empty
    var mu: object := @.cont
    if not mu.move(Destination)
        stopuntil mu.location /= @ and not Destination.isLoading
    end
end
@.stopped := false
```

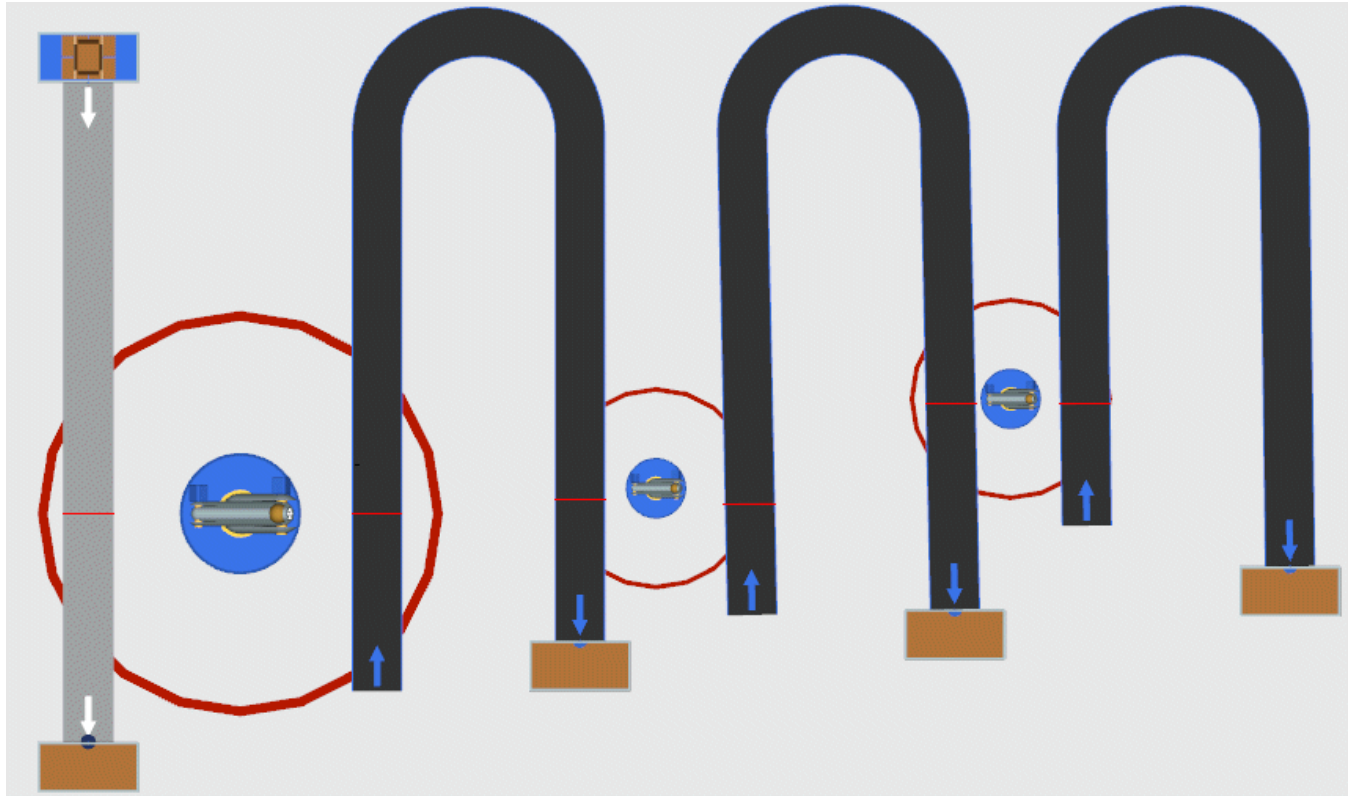
Back to [Unload Stacked Parts](#)

## Configure the Conveyors

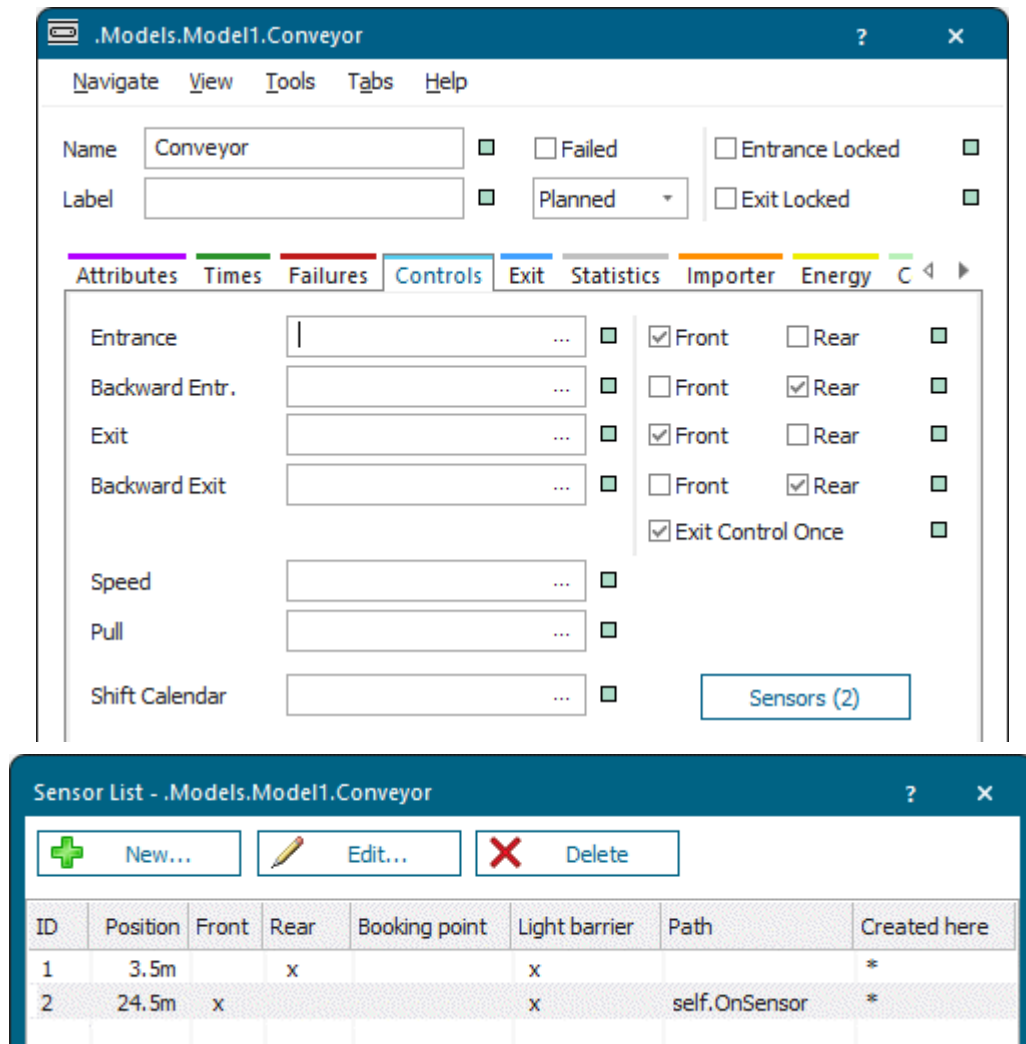
In this step we are going to configure the *Conveyors* that transport the *Pallets*, the *Boxes*, and the *Parts*.

- Insert three *Conveyors* next to each other.
- For the two *Conveyors* on the left create two sensors each. For the *Conveyor* on the right create one sensor. The robots unload parts at the first sensor from a *Conveyor* and load them onto the next

Conveyor at the second sensor. The sensors for unloading parts from a Conveyor are triggered by a light barrier by the rear of the part.



- Then, program the **Sensor Controls** at which the individual Conveyors load the parts onto the next Conveyor.



Except for the Destination the **Sensor Control** is the same for the three *Conveyors*. You only have to adjust the variable Destination, namely PickAndPlace1 and PickAndPlace2.

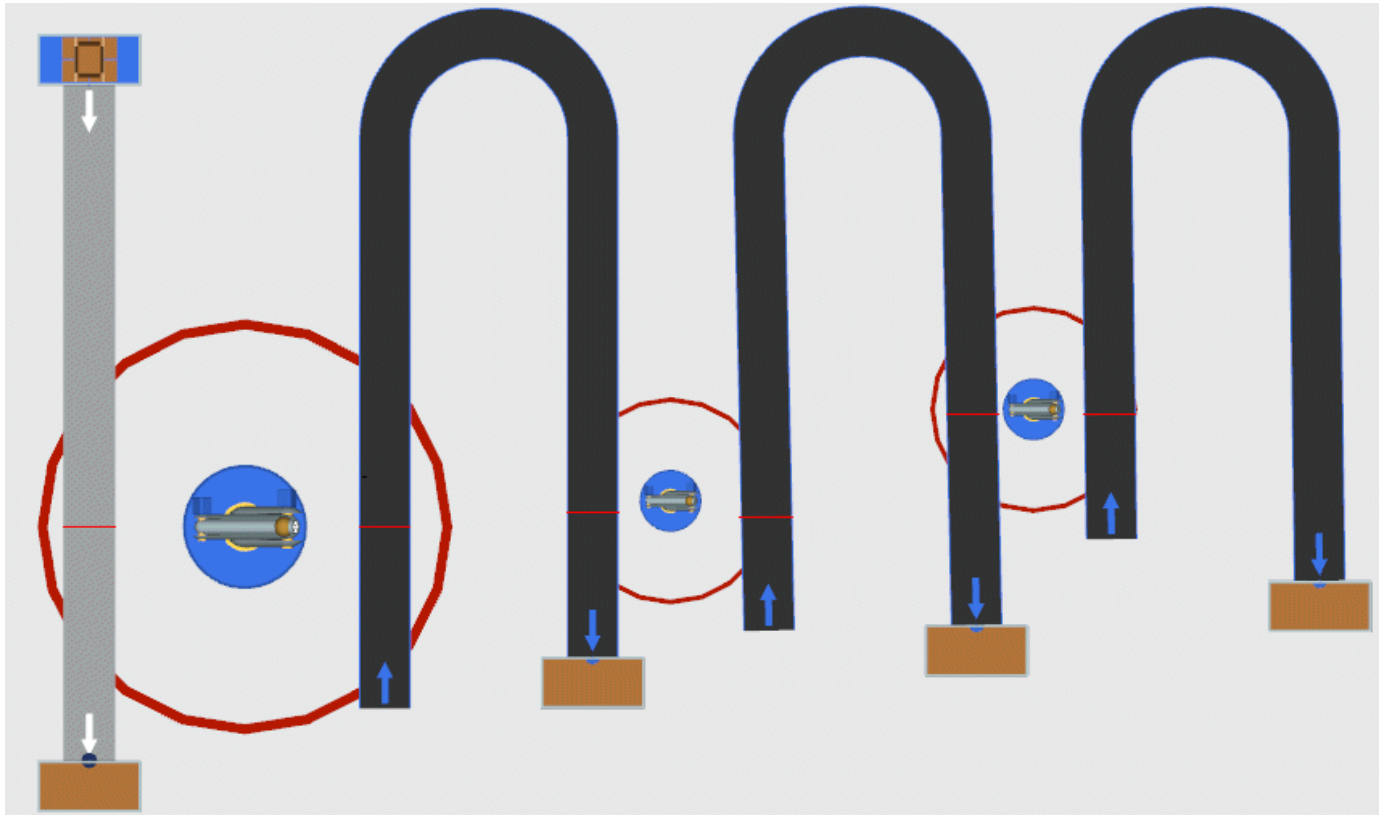
The respective source code looks like this:

```
param SensorID: integer, Front: boolean
@.stopped := true
var Destination:object := PickAndPlace1
while not @.Empty
  var mu: object := @.cont
  if not mu.move(Destination)
    stopuntil mu.location /= @ and not Destination.isLoading
  end
end
@.stopped := false
```

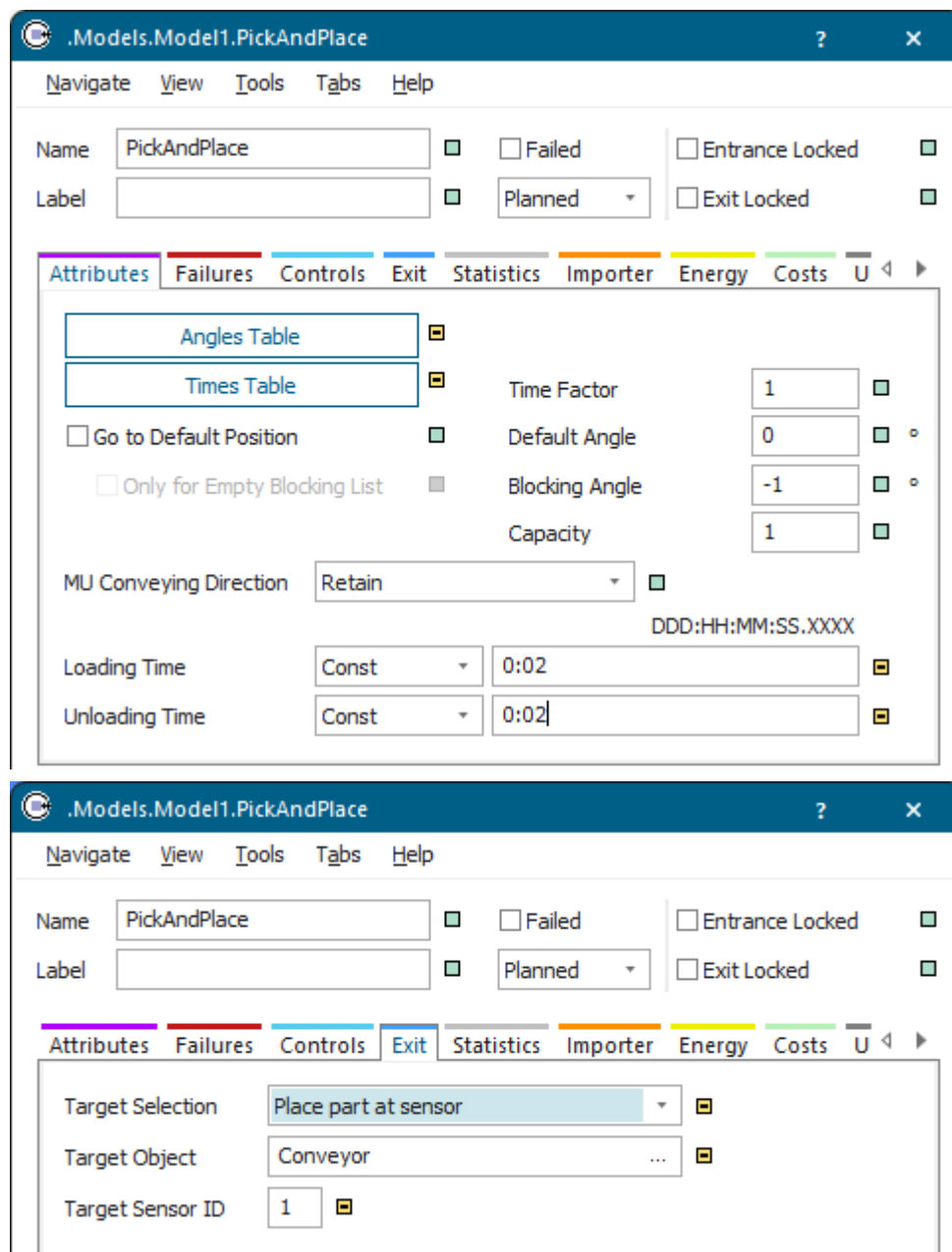
[Back to Unload Stacked Parts](#)

## Configure the Robots Unloading the Stacks

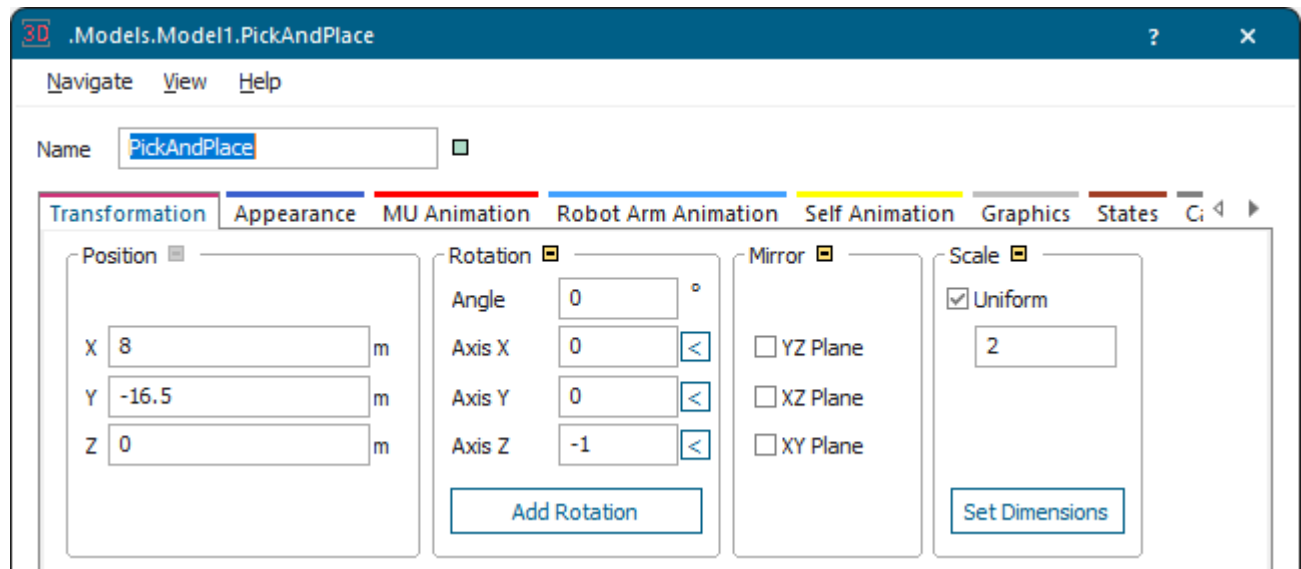
In this step we configure the robots that unload the stacks. Our three robots load the parts at the sensors from the *Track* onto the *Conveyor* and then onto the other two *Conveyors*.



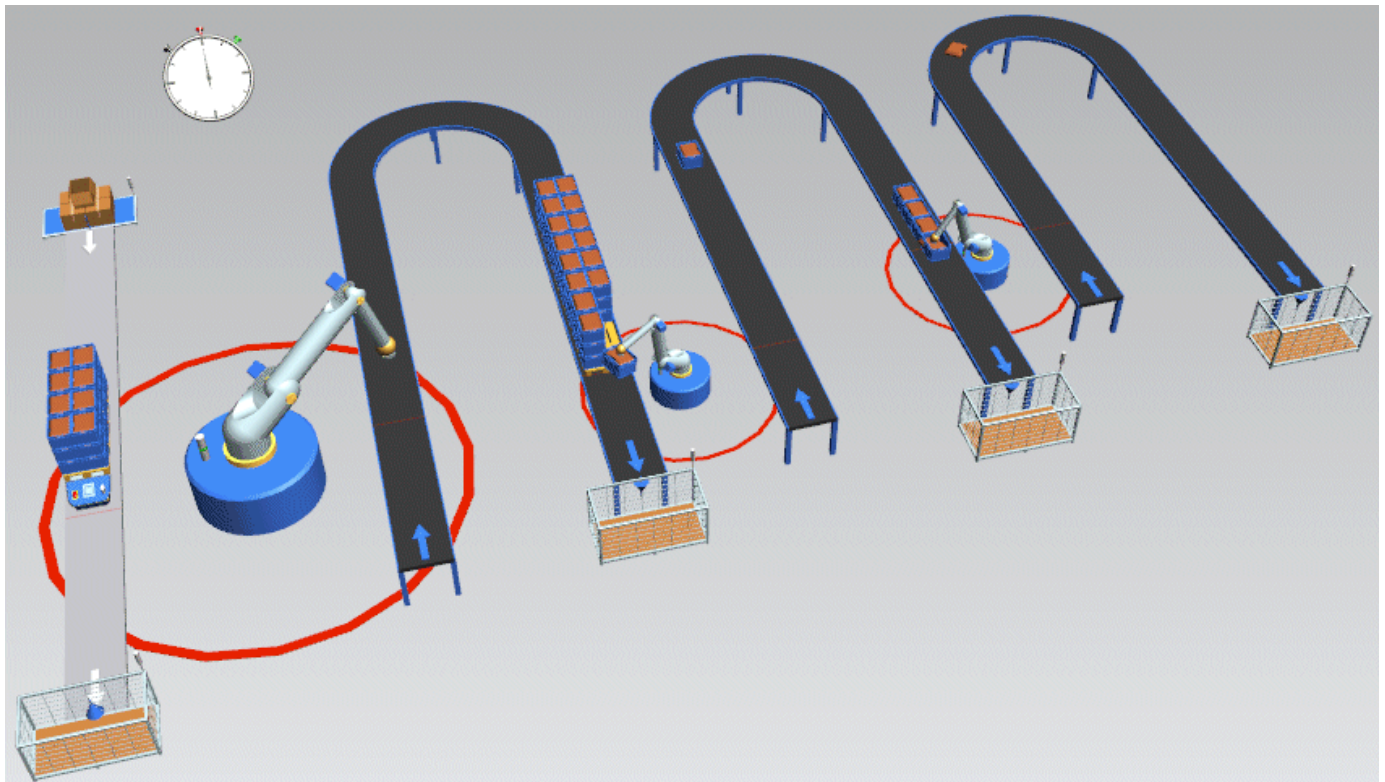
Configure the robots with the same settings. Enter a **Loading Time** and an **Unloading Time** of 2 seconds each. The robot places the parts at the respective first sensor onto the succeeding *Conveyor*.



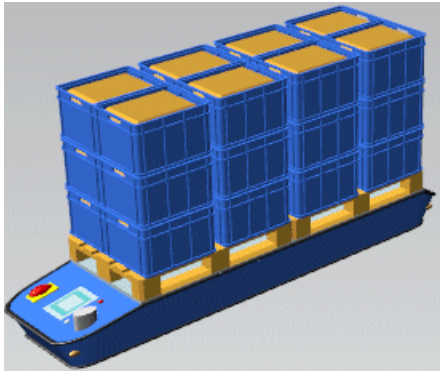
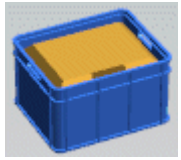
Double the size of the graphic of the first robot by selecting a **Uniform Scaling Factor** of 2.



When you now run the simulation, you can watch how the robots unload the stacked parts and singularize them.



The stacked parts look like this:

| Transporter                                                                       | Pallet                                                                            | Box                                                                                 |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|  |  |  |

[Back to Unload Stacked Parts](#)

## Animate the Robot Arm in 3D

### Animate the Robot Arm

On the tab **Robot Arm Animation** you can set how the arm of the *PickAndPlace robot* will be animated. We demonstrate this with a simple sample model.

We will:

- [Configure the Material Flow](#)
- [Exchange the Graphic of the Robot](#)
- [Configure the Movement of the Robot Arm](#)

[Back to Pick and Place Parts with the PickAndPlace Robot](#)

### Configure the Material Flow

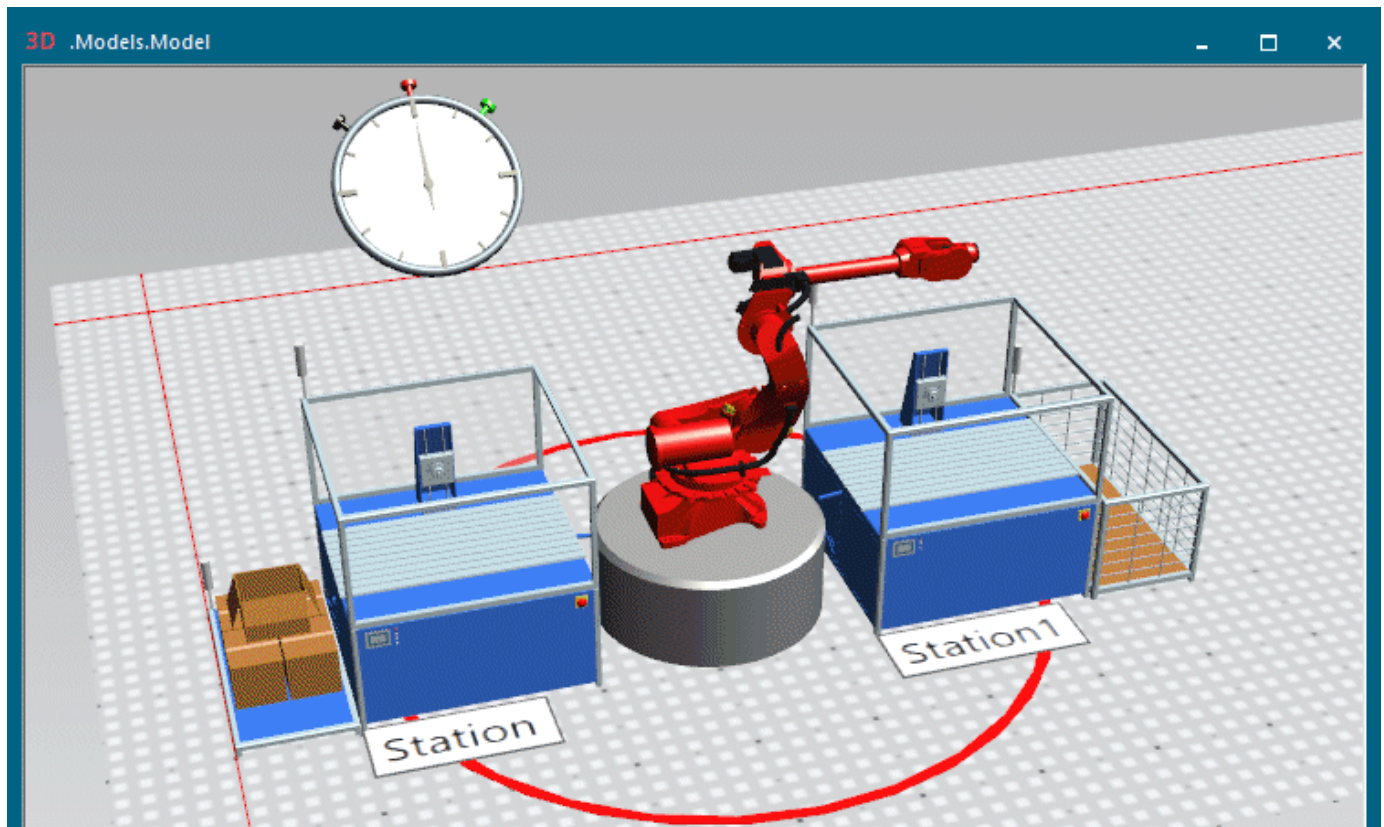
In our first step we model the material flow. To so, we place the individual material flow objects next to each other to make *Plant Simulation* connect them automatically with *Connectors*.

#### Procedure

1. Our material flow consists of a *Source* which produces the parts.
2. The first *Station* processes the parts.
3. After the parts are processed all the way, the *Robot* picks them up.
4. It then rotates to the seconds *Station* and places them there. *Station1* processes the parts as well.
5. The *Drain* removes the parts from the installation.

We use the material flow objects with their default settings meaning we do not change anything.

The finished simulation model looks like this:

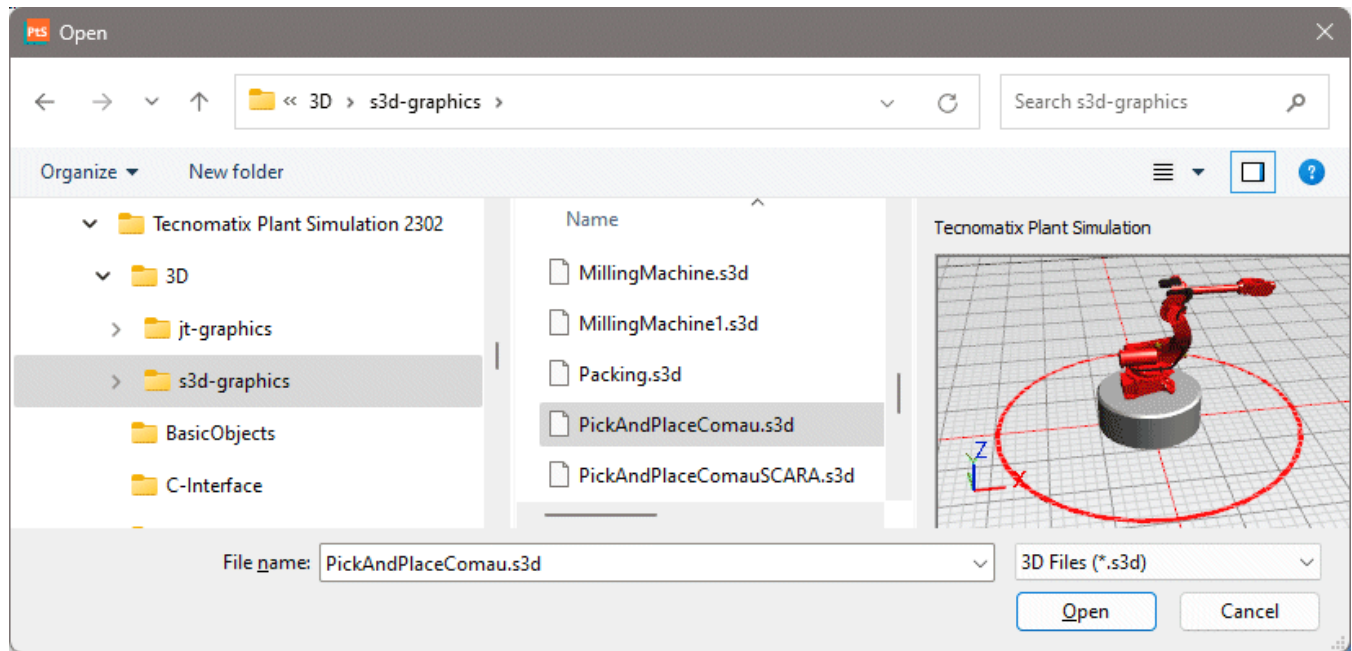


Back to [Animate the Robot Arm](#)

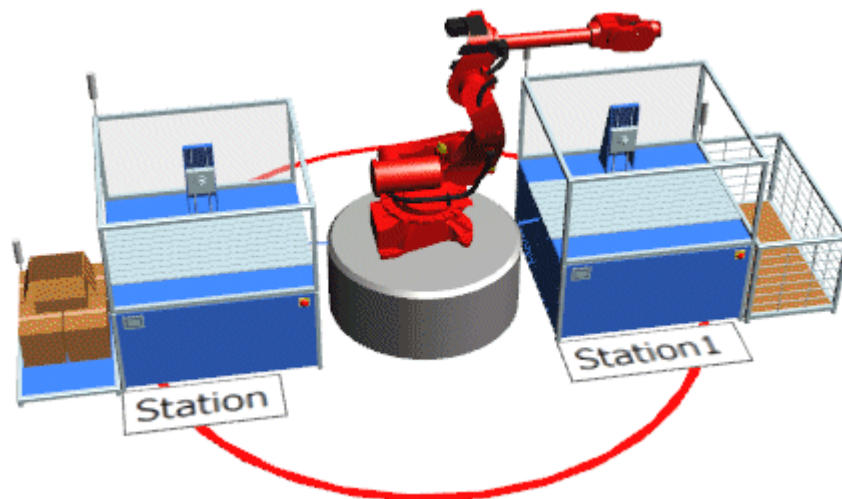
## Exchange the Graphic of the Robot

As we do not want to use the graphic of the built-in *PickAndPlace* robot, we will replace its graphic.

- To do so, we click the robot with the right mouse button and select **Exchange Graphics**.
- We use the *PickAndPlace* robot by Comau.



Our model now looks like this.



Back to [Animate the Robot Arm](#)

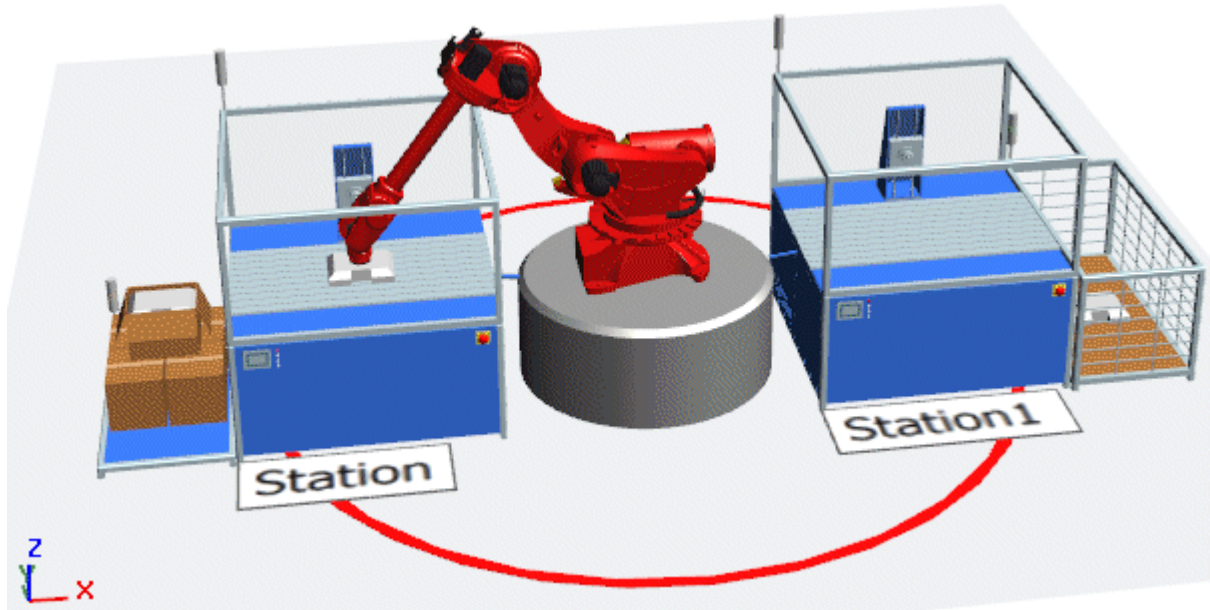
[Video on YouTube](#)

<https://youtu.be/K-NEPvShU10?si=gOBbox6eKxZQNEi8&t=730>

## Configure the Movement of the Robot Arm

When you run the simulation with the standard settings of the *EventController*, it runs so fast that it is hard to recognize the motion sequence. To prevent this, enter 1 as the **Real-time Factor**.

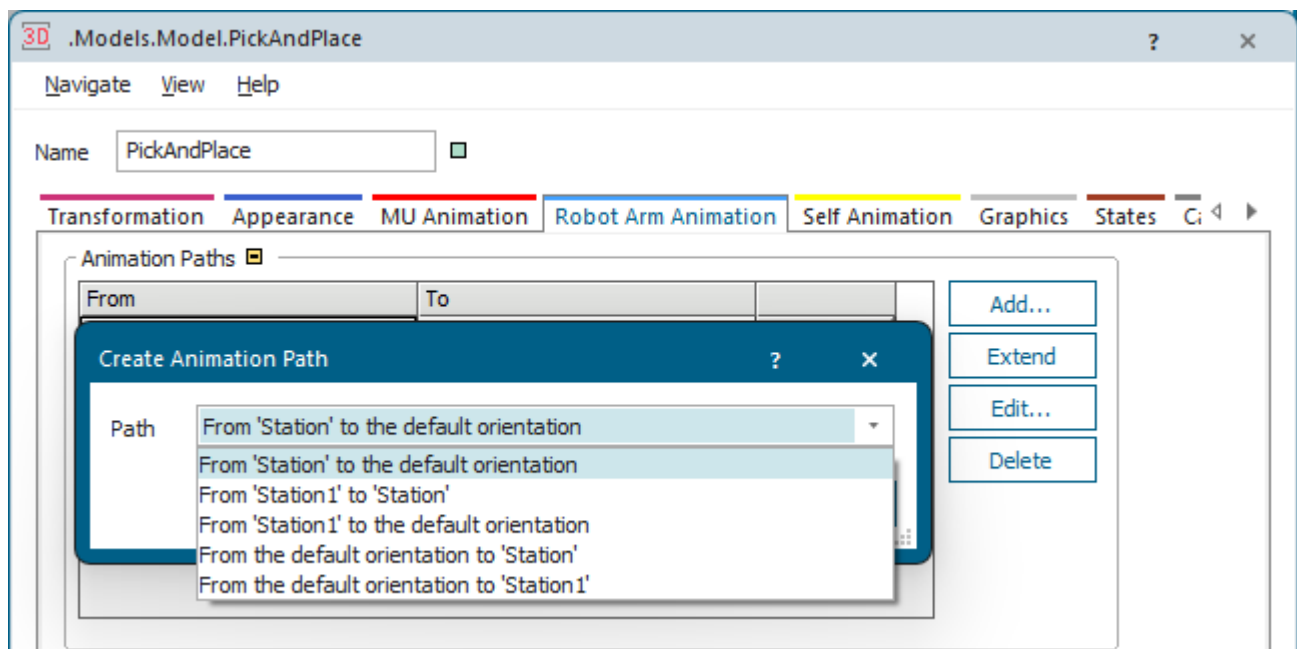
Now you can watch the robot move from the default orientation to the *Station*, lower the robot arm, pick the part up, raise the robot arm, rotate to the second *Station*, and place the part there.



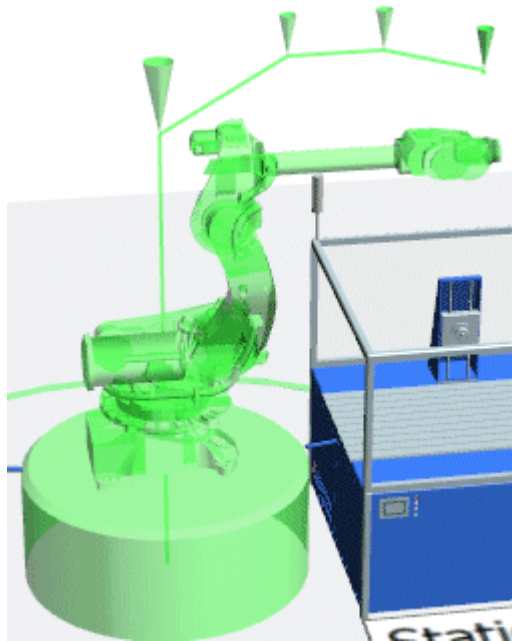
You will notice though that the robot arm moves through the protective cage and the parts holder. To prevent this from happening, change the movement of the robot arm so that it moves above both.

To do so, press the **spacebar** to open the dialog **Edit 3D Properties** and change to the tab **Robot Arm Animation** [\[PickAndPlace\]](#).

**Add** the animation path **From 'Station' to 'Station1'**.

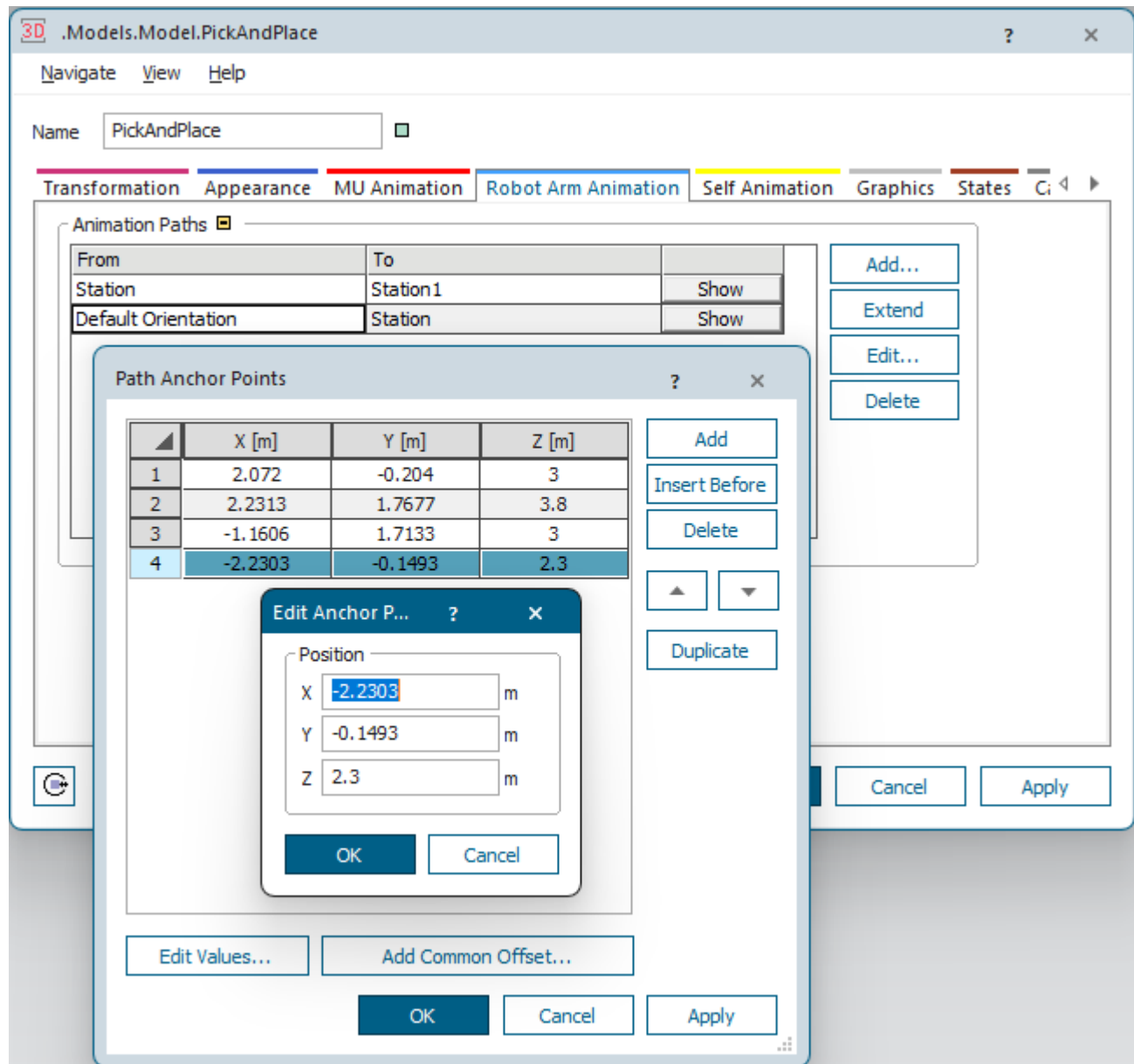


Then, click **Show** to show the animation path in the model window.

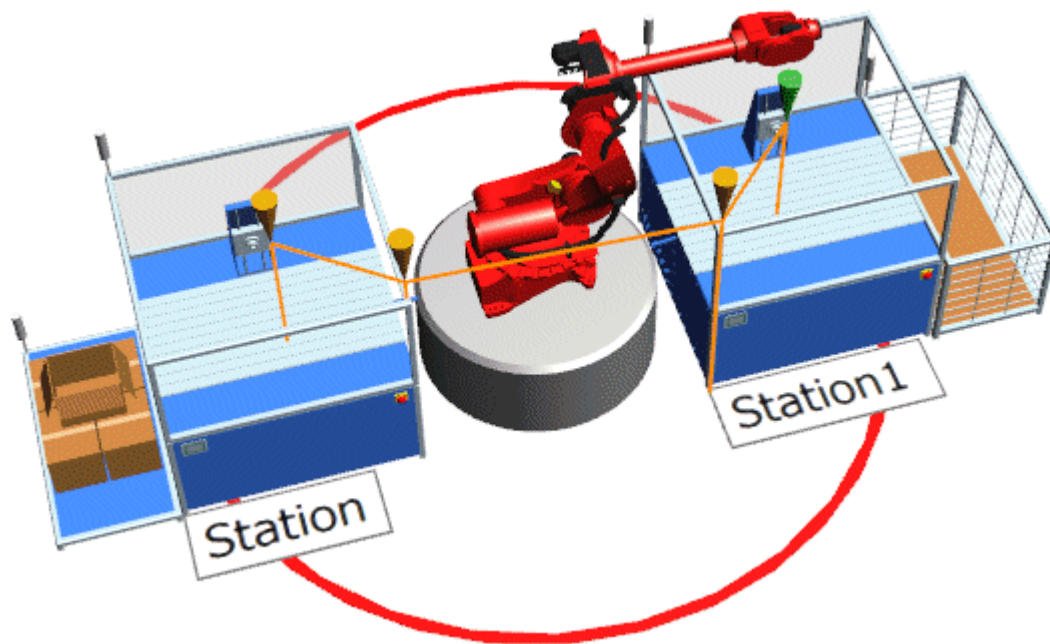


Next, click **Extend** and add three path anchor points.

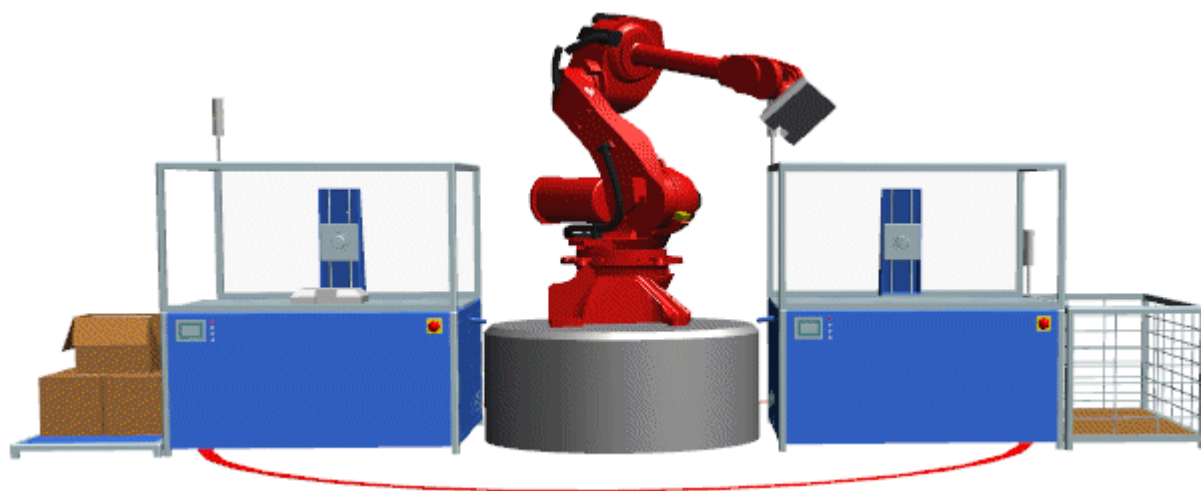
Then adjust the elevation of the anchor points: Click **Edit** and enter precise values. To do so, double-click the row of the respective anchor point in the dialog **Path Anchor Points** and type in an elevation of 2.2 for all anchor points. As our stations are 2 meters high, 2.20 meters should provide enough room for the robot arm to move across them.



As the last step you have to move the anchor points to the correction positions on the stations at which the robot picks the parts up, exits the station, enters the next station and places the part. To do so, click the respective anchor point and drag it to the correct position.



When you run the simulation you see that the robot lifts the part above the protective cage while moving from the *Station* to *Station1*.



As our robot arm still moves through the parts holder and the protective cage from the default orientation to the *Station*, add the path **Default Orientation to Station**.

To do so, repeat the steps described above. To prevent the robot arm from touching the indicator light in the left corner of the cage, we entered 3.8 meters as the elevation. It then looks like this: